



DOCUMENTO DE ARQUITECTURA

SIRHA - Sistema de Reasignación de Horarios Académicos.

Equipo Paw Patrol

Juan Pablo Caballero.

Oscar Sánchez.

Robinson Núñez.

David Santiago Palacios.

Diego Fernando Chavarro.

PROGRAMA

Ingeniería de Sistemas

ASIGNATURA

Ciclos de Vida del Desarrollo de Software (CVDS)

Desarrollo de Operaciones de Software (DOSW)

PROFESORES

Ing. Andrés Martín Cantor Urrego G1-G2

Ing. Pedro Andrés Melo García G3

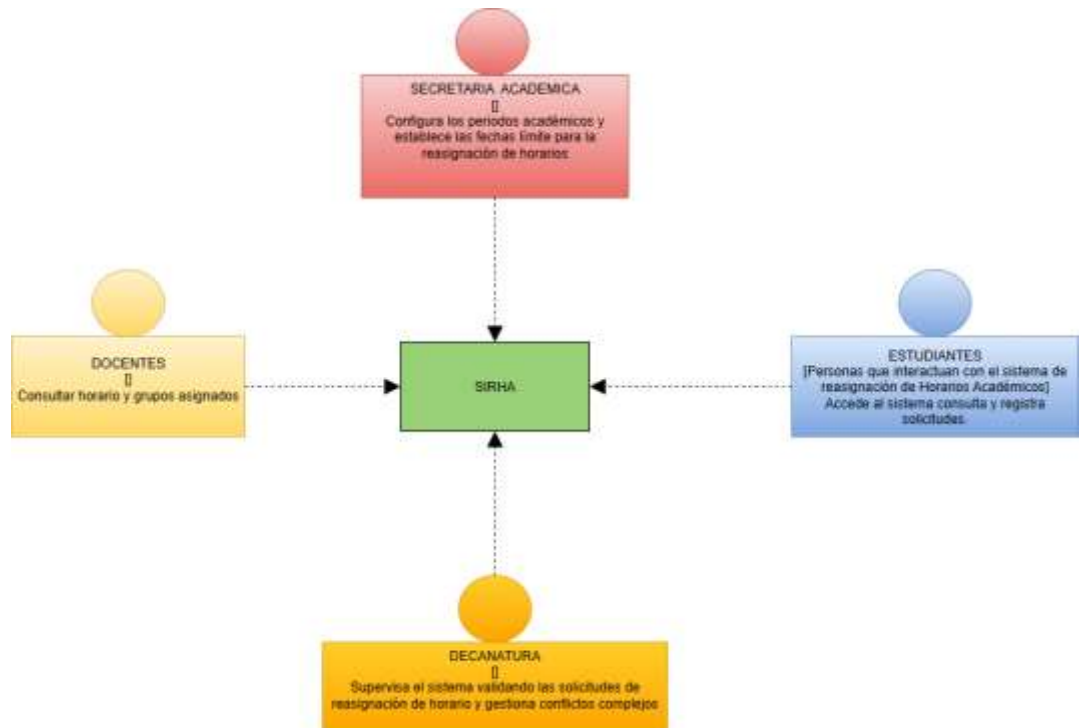
BOGOTÁ DC, COLOMBIA

30 de 09 de 2025

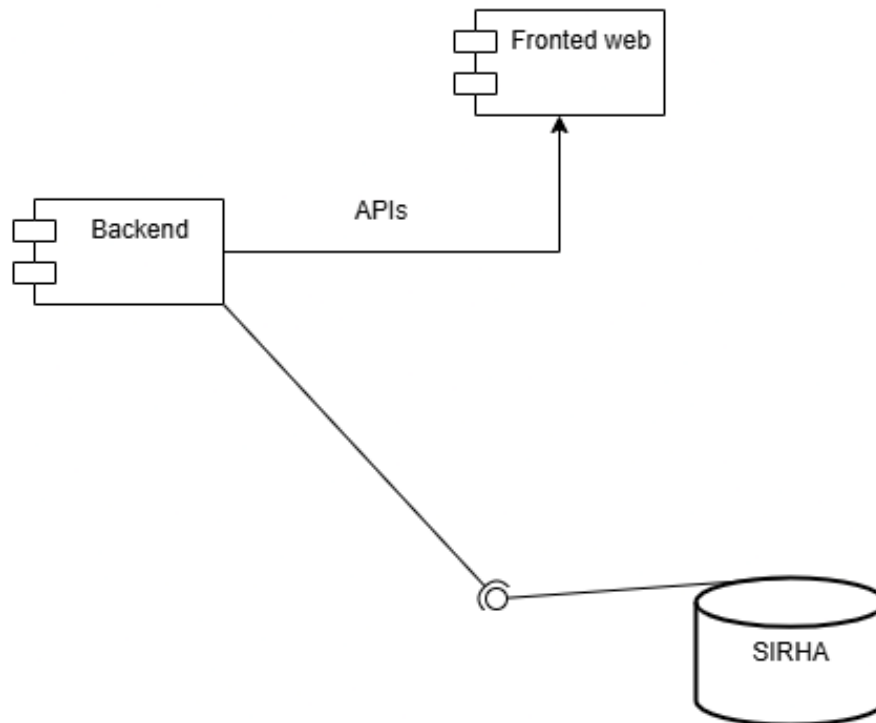
SIRHA

- introducción de la solución:

El sistema SIRHA tiene como finalidad facilitar y optimizar la gestión de solicitudes de cambio de materias y grupos de estudiantes, con trazabilidad, control de capacidad y priorización, ofreciendo a la decanatura y a los profesores herramientas para procesar y valorar las solicitudes.



- Diagrama de Componentes General:



El sistema SIRHA opera mediante una arquitectura de tres capas que asigna responsabilidades específicas:

1. **Capa de Presentación (Frontend):** Es la capa de presentación, una interfaz grafica en la cual interactúan los usuarios del sistema, en este módulo no hay lógica compleja, solo es el encargado de mostrar la información y capturar las acciones de los usuarios, comunicándose con backend a través de las APIs.
2. **Capa de Lógica (Backend):** Es el núcleo del sistema, es el encargado de toda la lógica de negocio. En esta capa se cargan todas las solicitudes de los estudiantes, se les aplican restricciones del negocio, se gestionan y se generan reportes de las mismas. El backend actúa como intermediario entre el frontend y la base de datos, asegurando que las reglas de negocio se cumplan.
3. **Capa de Datos (Base de Datos):** Es la encargada de la persistencia del sistema, de almacenar toda la información pertinente, esta información incluye datos de estudiantes, materias, grupos, horarios, solicitudes, etc. El acceso a esta base de datos se da únicamente a backend, lo que nos garantiza la consistencia y la seguridad de los datos.

Tecnologías utilizadas

- Tecnologías utilizadas:

SIRHA

Para el desarrollo del proyecto usamos las siguientes tecnologías:

Lenguaje de Programación

- **Java 21** - Versión más reciente del lenguaje de programación

Framework Principal

- **Spring Boot 3.5.3** - Framework para desarrollo de aplicaciones Java empresariales

Gestión de Dependencias y Build

- **Maven** - Herramienta para la gestión y construcción del proyecto

Base de Datos

- **MongoDB** - Base de datos NoSQL no relaciona
- **Spring Data MongoDB** - Integración con MongoDB para operaciones de datos
- **H2 Database** - Base de datos en memoria para entornos de prueba

Desarrollo Web y API

- **Spring Web MVC** - Framework para aplicaciones web y REST APIs
- **SpringDoc OpenAPI 2.6.0** - Documentación automática de APIs (Swagger UI)
- **Hibernate Validator** - Validación de datos y beans

Seguridad y Autenticación

- **JSON Web Token (JWT)** - Para autenticación y autorización
 - jjwt-api 0.11.5
 - jjwt-impl 0.11.5
 - jjwt-jackson 0.11.5

Herramientas de Desarrollo

- **Lombok 1.18.32** - Reducción de código boilerplate mediante anotaciones
- **Spring Boot DevTools** - Herramientas para desarrollo

Testing y Calidad de Código

- **JUnit 5** - Framework de testing
- **Mockito** - Framework para mocking en tests

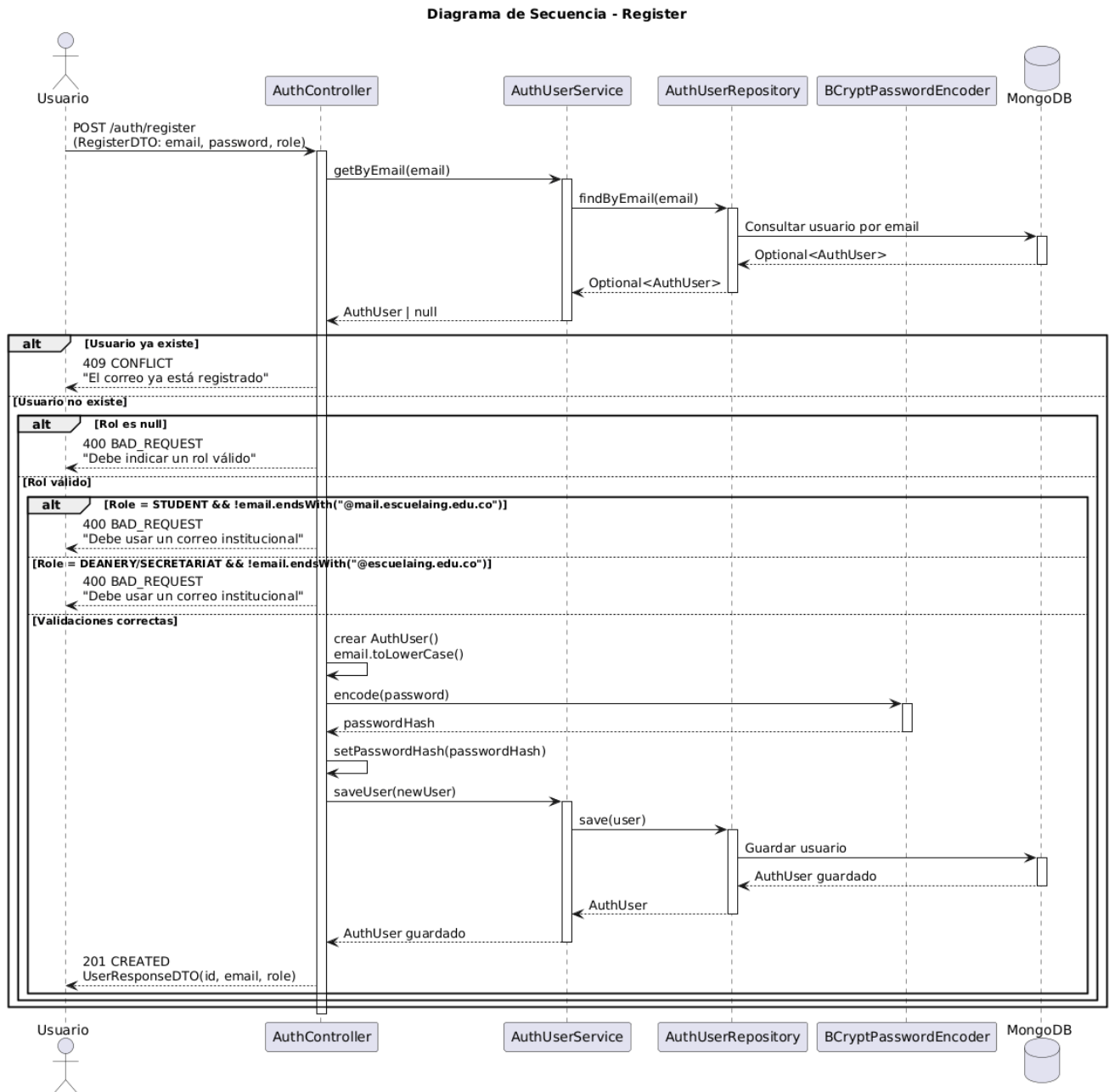
- **JaCoCo 0.8.13** - Análisis de cobertura de código
- **SonarQube** - Análisis estático de código y métricas de calidad

Funcionalidades del Sistema

1. Funcionalidad: Registrar Nuevo Usuario

- **descripción de la Funcionalidad:** Permite registrar un nuevo usuario en el sistema SIRHA con un rol específico (STUDENT, DEANERY, SECRETARIAT, ADMIN). El sistema valida que el email no esté previamente registrado, verifica que se proporcione un rol válido, valida que el formato del correo coincida con el dominio institucional según el rol (estudiantes deben usar @mail.escuelaing.edu.co, personal administrativo @escuelaing.edu.co), convierte el email a minúsculas para normalización, hashea la contraseña usando BCrypt para almacenamiento seguro, persiste el usuario en la base de datos y retorna la información del usuario creado sin incluir la contraseña. Retorna código 201 Created al completarse exitosamente.

- **Diagramas:**



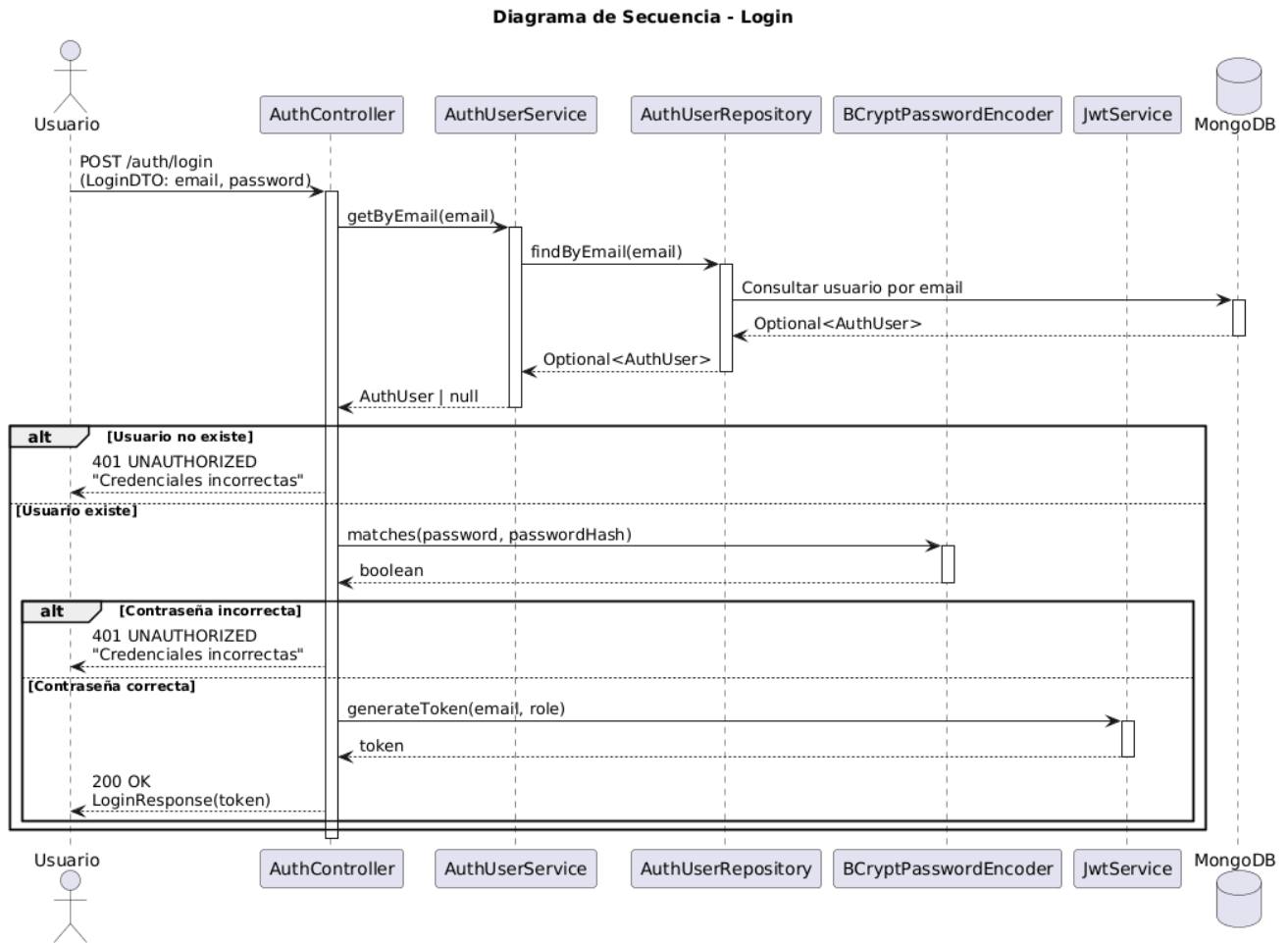
El cliente envía una petición POST con el cuerpo RegisterDTO conteniendo email, password y role, el sistema busca si ya existe un usuario con ese email mediante getByEmail(), si existe lanza excepción 409 Conflict con mensaje "El correo ya está registrado", valida que el campo role no sea null, si es null lanza excepción 400 Bad Request con mensaje "Debe indicar un rol válido", normaliza el email convirtiéndolo a minúsculas con toLowerCase(), valida el dominio del email según el

rol: si es STUDENT verifica que termine en "[@mail.escuelaing.edu.co](mailto:mail.escuelaing.edu.co)" lanzando 400 si no cumple, si es DEANERY o SECRETARIAT verifica que termine en "[@escuelaing.edu.co](mailto:escuelaing.edu.co)" lanzando 400 si no cumple, crea una nueva instancia de AuthUser, establece el email normalizado, establece el role proporcionado, hashea la contraseña con `passwordEncoder.encode()` y la establece en `passwordHash`, persiste el usuario con `saveUser()` generando ID automáticamente, mapea el usuario guardado a `UserResponseDTO` conteniendo solo id, email y role (sin password), y retorna el DTO con estado HTTP 201.

- **Manejo de Errores:** 409 Conflict: Si el email ya está registrado en el sistema. Mensaje: "El correo ya está registrado". 400 Bad Request: Si no se proporciona el campo role o es null. Mensaje: "Debe indicar un rol válido". 400 Bad Request: Si el rol es STUDENT y el email no termina en "[@mail.escuelaing.edu.co](mailto:mail.escuelaing.edu.co)". Mensaje: "Debe usar un correo institucional". 400 Bad Request: Si el rol es DEANERY o SECRETARIAT y el email no termina en "[@escuelaing.edu.co](mailto:escuelaing.edu.co)". Mensaje: "Debe usar un correo institucional". 400 Bad Request: Si faltan campos obligatorios (email, password, role) o están malformados. 400 Bad Request: Si el valor de role no corresponde a un valor válido del enum Role (STUDENT, DEANERY, SECRETARIAT, ADMIN). 500 Internal Server Error: Si ocurre un error durante el hashéo de la contraseña o persistencia del usuario.
- **Link Swagger:** POST /auth/register

2. Funcionalidad: Iniciar Sesión

- **descripción de la Funcionalidad:** Permite a un usuario autenticarse en el sistema SIRHA proporcionando su correo electrónico y contraseña. El sistema valida las credenciales contra la base de datos de usuarios autorizados, verifica que la contraseña coincida usando bcrypt, y si la autenticación es exitosa genera un token JWT (JSON Web Token) válido por 1 hora que incluye el email del usuario y su rol. El token debe usarse en el header Authorization: Bearer {token} para acceder a endpoints protegidos del sistema.
- **Diagramas:**



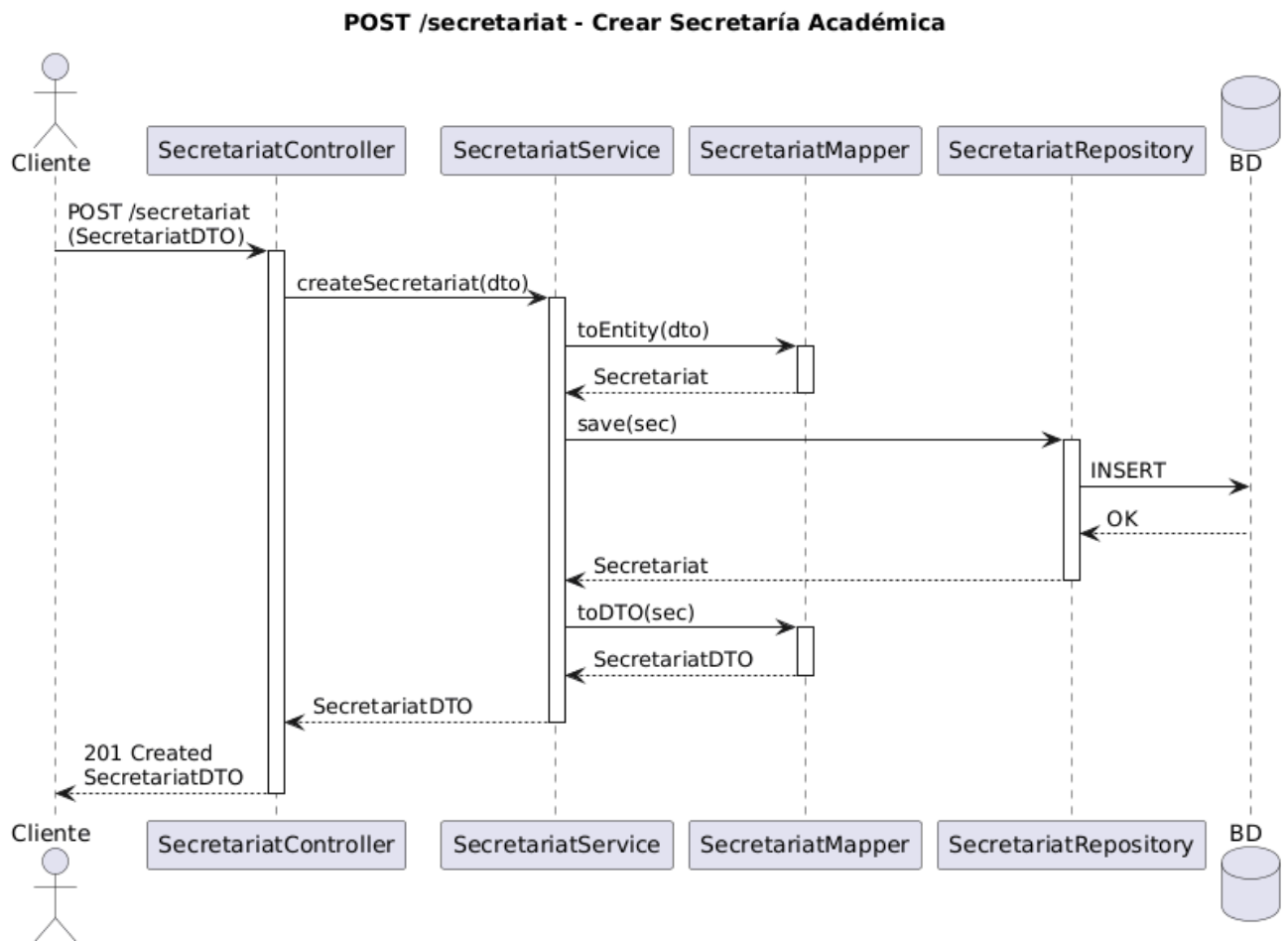
El cliente envía una petición POST con el cuerpo LoginDTO conteniendo email y password en texto plano, el sistema busca el usuario por email en el repositorio mediante getByEmail(), si el usuario no existe retorna null y lanza excepción 401 Unauthorized, si existe verifica la contraseña usando passwordEncoder.matches() comparando el password del request con el passwordHash almacenado en la base de datos,

si las contraseñas no coinciden lanza excepción 401 Unauthorized con mensaje "Credenciales incorrectas", si coinciden invoca el servicio JWT con generateToken(email, role) que crea un token firmado con HS256 incluyendo el email como subject, el rol en los claims, fecha de emisión y fecha de expiración (1 hora), retorna un objeto LoginResponse conteniendo únicamente el token JWT generado.

- **Manejo de Errores:** 401 Unauthorized: Si el email no existe en el sistema. Mensaje: "Credenciales incorrectas". 401 Unauthorized: Si la contraseña proporcionada no coincide con el hash almacenado. Mensaje: "Credenciales incorrectas". 400 Bad Request: Si el cuerpo del request está malformado o faltan campos obligatorios (email o password). 500 Internal Server Error: Si ocurre un error durante la generación del token o consulta a la base de datos.
- **Link Swagger:** POST /auth/login

3. Funcionalidad: Crear Nueva Secretaría Académica

- **descripción de la Funcionalidad:** Permite registrar una nueva secretaria académica en el sistema con sus datos básicos (nombre, email, fechas de inicio y fin del periodo de solicitudes). La secretaria es la entidad responsable de gestionar los periodos académicos y procesar solicitudes de cambios. Retorna la secretaria creada con su ID autogenerado y código de estado 201 Created.
- **Diagramas:**

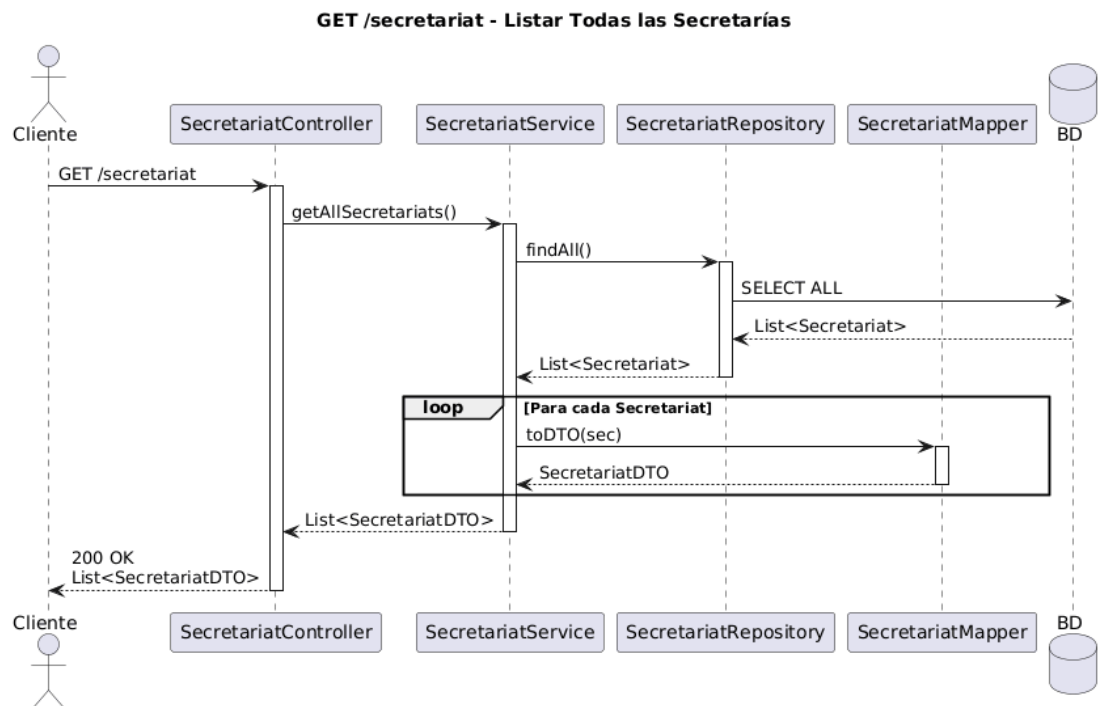


El cliente envía una petición POST con el cuerpo JSON conteniendo los datos de la secretaria (nombre, email, requestStartDate, requestEndDate), el sistema mapea el SecretariatDTO a entidad Secretariat mediante el mapper, persiste en la base de datos con save() generando el ID automáticamente, mapea la entidad guardada a SecretariatDTO y retorna con estado HTTP 201.

- **Manejo de Errores:** 400 Bad Request: Si faltan campos obligatorios (nombre o email). Mensaje: "Validation failed" con detalles de los campos. 400 Bad Request: Si el formato del email es inválido. 400 Bad Request: Si las fechas son inválidas o la fecha de fin es anterior a la fecha de inicio. 500 Internal Server Error: Si ocurre un error inesperado durante la persistencia.
- **Link Swagger:** POST /secretariat

4. Funcionalidad: Listar Todas las Secretarías Académicas

- **descripción de la Funcionalidad:** Permite obtener una lista completa de todas las secretarías académicas registradas en el sistema. Retorna información básica de cada secretaría incluyendo ID, nombre, email y fechas del periodo de solicitudes configurado.
- **Diagramas:**

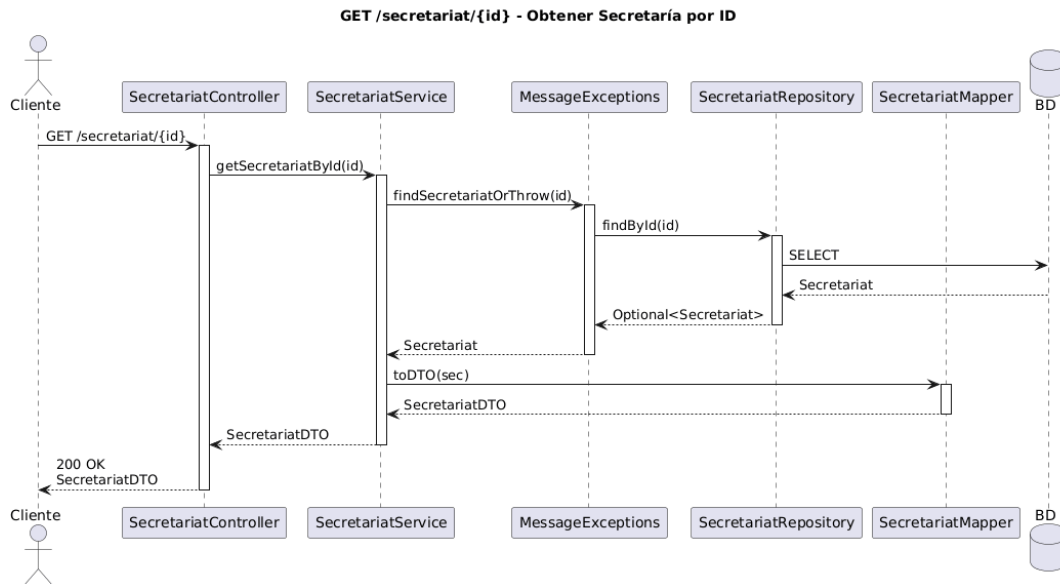


El cliente realiza una petición GET sin parámetros, el sistema consulta el repositorio de secretarías mediante findAll(), mapea cada entidad Secretariat a SecretariatDTO usando stream y el mapper, y retorna la lista completa en formato JSON.

- **Manejo de Errores:** 500 Internal Server Error: Si ocurre un error inesperado en el servidor durante la consulta. Retorna una lista vacía [] si no hay secretarías registradas (no es un error).
- **Link Swagger:** GET /secretariat

5. Funcionalidad: Obtener Secretaría por ID

- **descripción de la Funcionalidad:** Recupera la información detallada de una secretaría académica específica mediante su identificador único numérico. Retorna todos los datos de la secretaría incluyendo nombre, email y las fechas configuradas para el periodo de solicitudes (requestStartDate, requestEndDate).
- **Diagramas:**

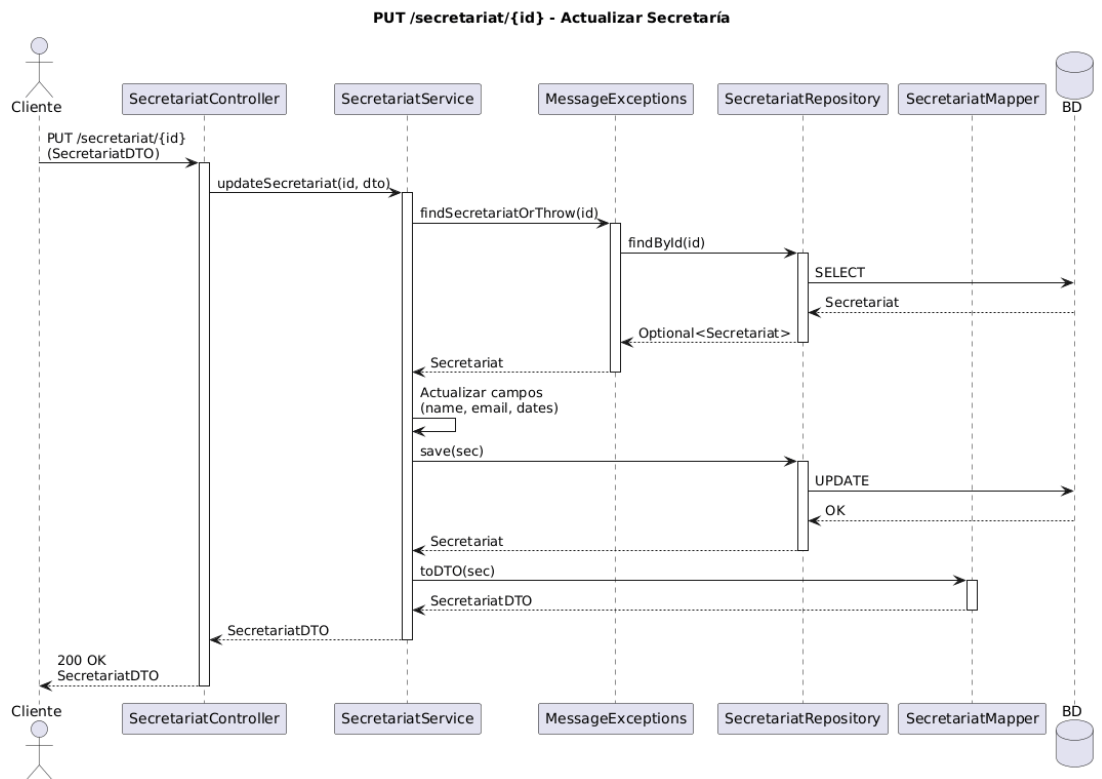


El cliente envía una petición GET con el ID de la secretaría como parámetro de ruta, el sistema invoca `findSecretariatOrThrow(id)` que busca en el repositorio mediante `findById()`, valida la existencia de la secretaría lanzando excepción 404 si no existe, mapea la entidad `Secretariat` a `SecretariatDTO` y retorna el DTO completo.

- **Manejo de Errores:** 404 Not Found: Si la secretaría con el ID especificado no existe. Mensaje: "Secretari@ no encontrad@ con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la operación.
- **Link Swagger:** GET /secretariat/{id}

6. Funcionalidad: Actualizar Secretaría Académica

- **descripción de la Funcionalidad:** Permite actualizar parcialmente los campos de una secretaría académica existente. Solo actualiza los campos que están presentes y no son null en el request body (nombre, email, requestStartDate, requestEndDate). Los campos ausentes o null mantienen su valor actual en la base de datos.
- **Diagramas:**

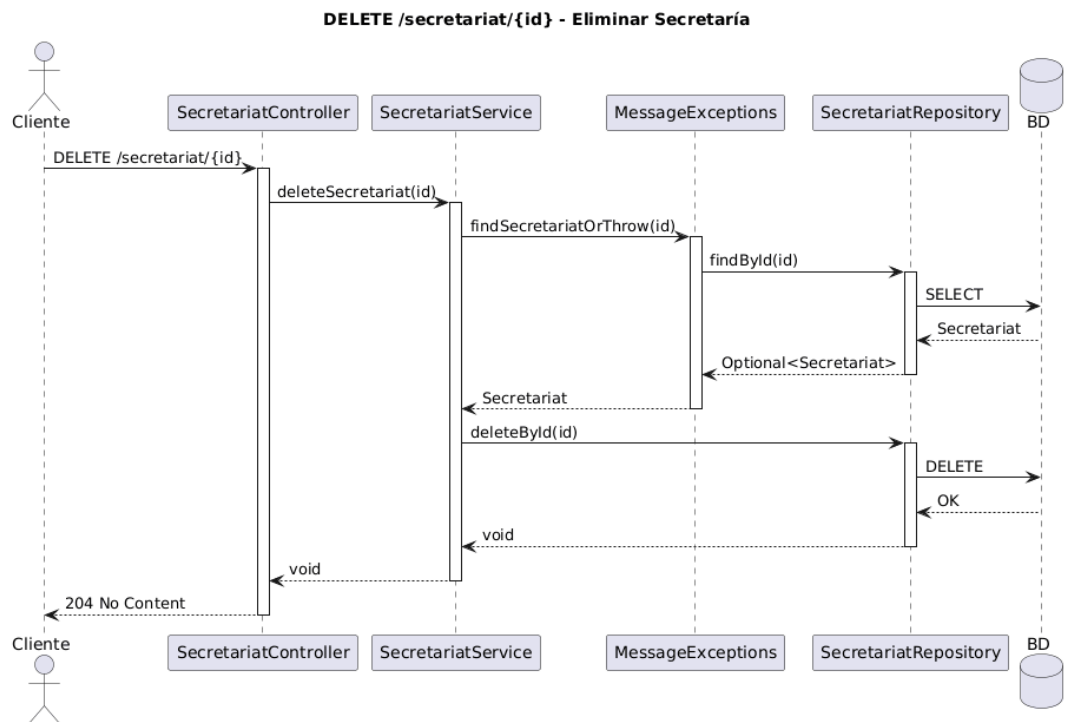


El cliente envía una petición PUT con el ID en la ruta y el cuerpo con los campos a actualizar, el sistema busca la secretaría existente con `findSecretariatOrThrow(id)`, verifica campo por campo si el DTO contiene valores no nulos (name, email, requestStartDate, requestEndDate), actualiza solo los campos presentes manteniendo los demás sin cambios, persiste con `save()` y retorna el `SecretariatDTO` completo actualizado.

- **Manejo de Errores:** 404 Not Found: Si la secretaria con el ID especificado no existe. Mensaje: "Secretari@ no encontrad@ con id: {id}". 400 Bad Request: Si el formato del email proporcionado es inválido. 400 Bad Request: Si las fechas proporcionadas son inválidas. 500 Internal Server Error: Si ocurre un error durante la actualización.
- **Link Swagger:** PUT /secretariat/{id}

7. Funcionalidad: Eliminar Secretaría Académica

- **descripción de la Funcionalidad:** Elimina permanentemente una secretaria académica del sistema mediante su ID. Antes de eliminar, valida la existencia de la secretaria. Retorna código de estado 204 No Content al completarse exitosamente la eliminación sin cuerpo en la respuesta.
- **Diagramas:**

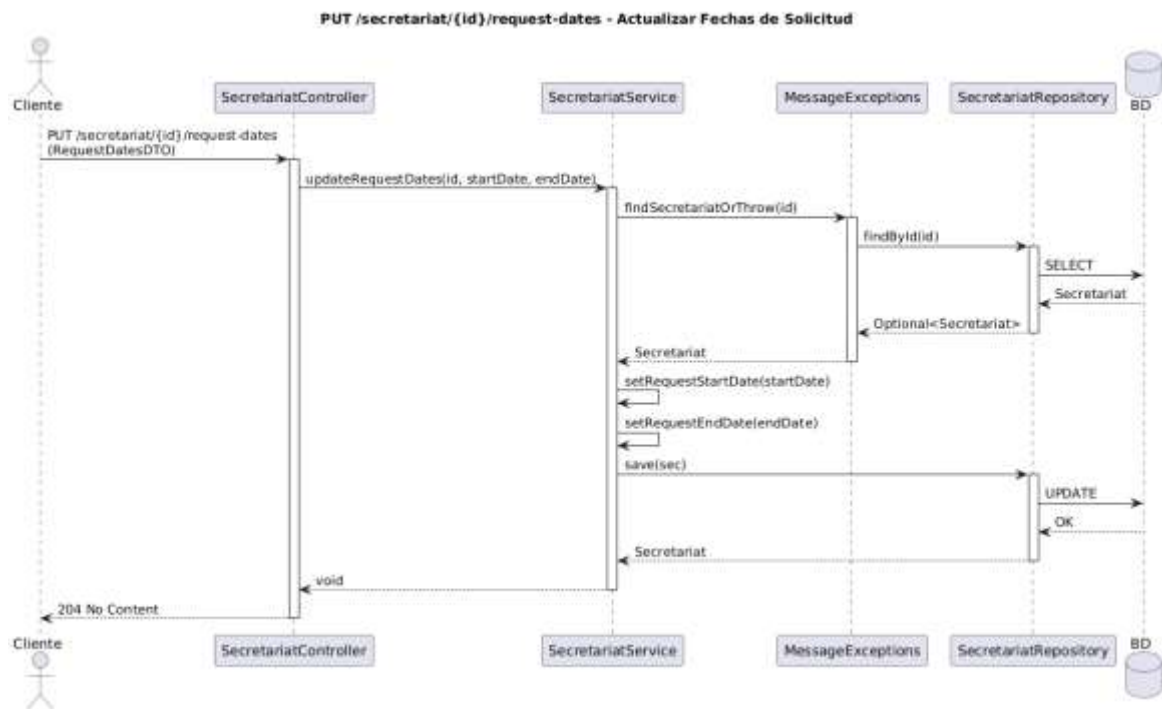


El cliente envía una petición DELETE con el ID de la secretaria en la ruta, el sistema busca y valida la existencia de la secretaria mediante findSecretariatOrThrow(id), si existe procede a eliminarla del repositorio con deleteById(id), confirma la eliminación y retorna estado HTTP 204 sin contenido en el body.

- **Manejo de Errores:** 404 Not Found: Si la secretaría con el ID especificado no existe. Mensaje: "Secretari@ no encontrad@ con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la eliminación (por ejemplo, restricciones de integridad referencial).
- **Link Swagger:** DELETE /secretariat/{id}

8. Funcionalidad: Actualizar Fechas del Periodo de Solicitudes

- **descripción de la Funcionalidad:** Permite actualizar específicamente las fechas de inicio y fin del periodo habilitado para realizar solicitudes de cambio. Esta operación es crítica para habilitar o deshabilitar ventanas de tiempo en las que los estudiantes pueden crear solicitudes. No retorna contenido, solo código 204 No Content al completarse exitosamente.
- **Diagramas:**



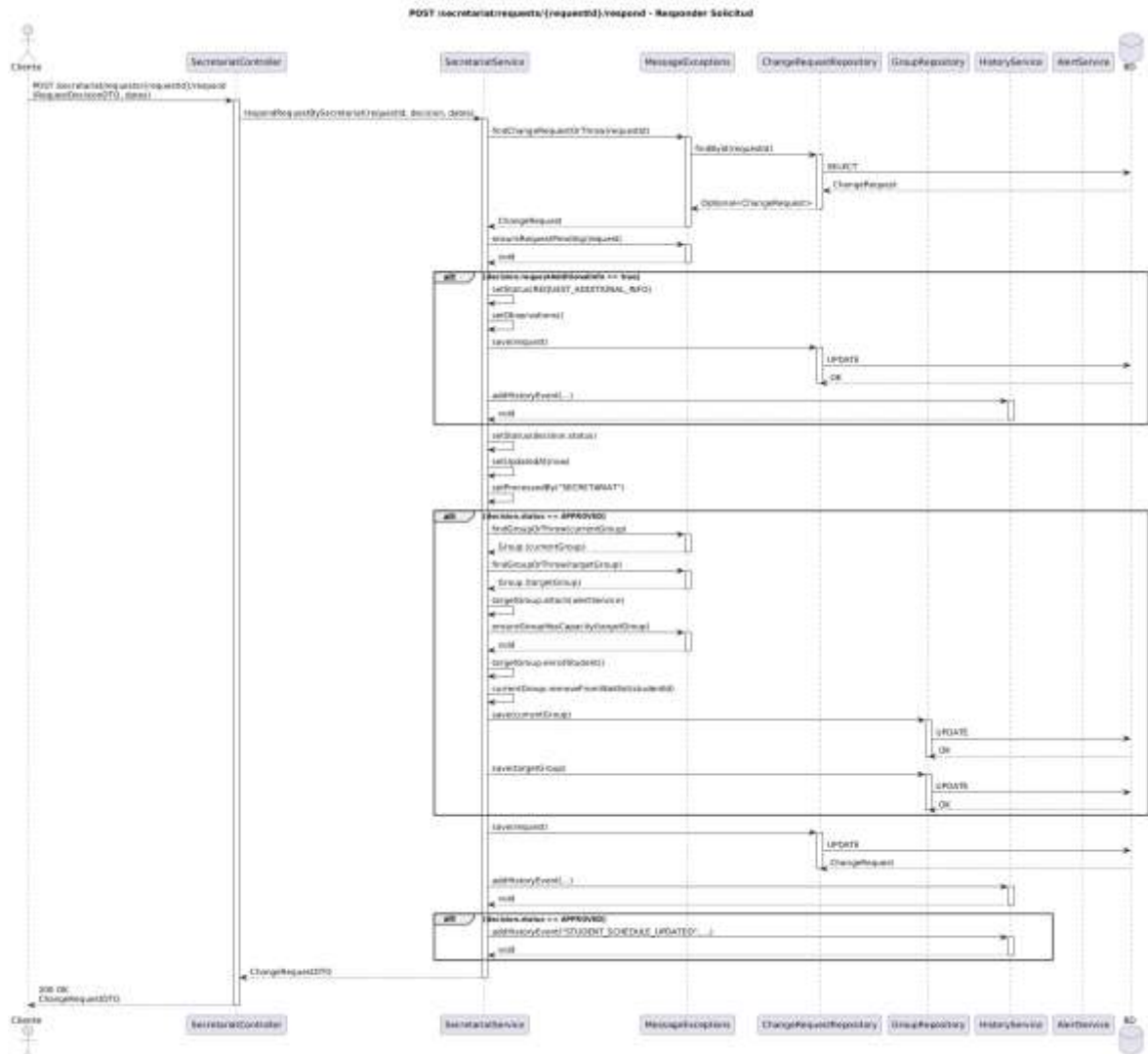
El cliente envía una petición PUT con el ID de la secretaría en la ruta y el cuerpo RequestDatesDTO conteniendo startDate y endDate, el sistema busca la secretaría con findSecretariatOrThrow(id), actualiza los campos requestStartDate y requestEndDate con las nuevas fechas,

persiste los cambios con `save()` y retorna estado HTTP 204 sin contenido.

- **Manejo de Errores:** 404 Not Found: Si la secretaría con el ID especificado no existe. Mensaje: "Secretari@ no encontrad@ con id: {id}". 400 Bad Request: Si las fechas no son proporcionadas o son inválidas. 400 Bad Request: Si la fecha de fin es anterior a la fecha de inicio. 500 Internal Server Error: Si ocurre un error durante la actualización de fechas.
- **Link Swagger:** PUT /secretariat/{id}/request-dates

9. Funcionalidad: Responder Solicitud como Secretaría

- descripción de la Funcionalidad:** Permite a la secretaría académica procesar y responder una solicitud de cambio pendiente. Puede aprobar la solicitud (actualizando capacidades de grupos y horarios del estudiante), rechazarla o solicitar información adicional. Valida que la solicitud esté en estado PENDING y que la operación se realice dentro del periodo habilitado. Al aprobar, actualiza automáticamente las capacidades de los grupos origen y destino, gestiona listas de espera y registra el cambio en el historial.
- Diagramas:**



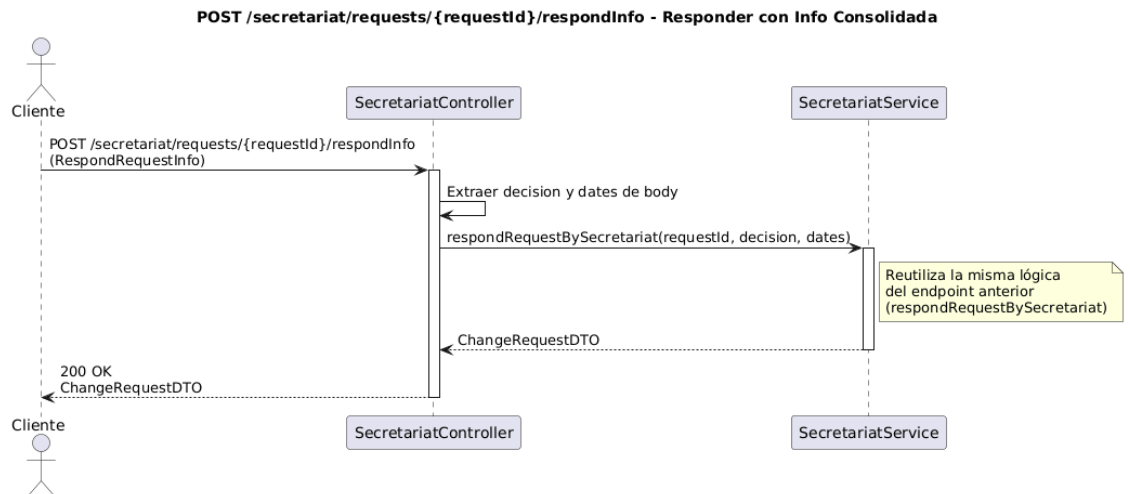
El cliente envía una petición POST con el requestId en la ruta, el cuerpo RequestDecisionDTO conteniendo la decisión (status, observations, requestAdditionalInfo, additionalInfoRequestMessage, infoDueDate) y opcionalmente parámetros query startDate y endDate, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté en

estado PENDING mediante `ensureRequestPending()`, valida que la operación esté dentro de las fechas permitidas con `ensureNowWithinDatesIfPresent()`, si se solicita información adicional cambia el estado a `REQUEST_ADDITIONAL_INFO` y agrega el mensaje a observaciones, si la decisión es `APPROVED` busca los grupos origen y destino, valida capacidad disponible del grupo destino con `ensureGroupHasCapacity()`, incrementa la capacidad actual del grupo destino con `enrollStudent()`, elimina al estudiante de la lista de espera del grupo origen si existe, persiste los cambios en ambos grupos y en la solicitud, actualiza el estado de la solicitud, marca `processedBy` como "SECRETARIAT", actualiza el timestamp, persiste la solicitud actualizada, registra el evento en el historial mediante `addHistoryEvent()` y retorna el `ChangeRequestDTO` actualizado.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 404 Not Found: Si el grupo origen o destino no existe. Mensaje: "Grupo no encontrado: {groupId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el grupo destino no tiene capacidad disponible al aprobar. Mensaje: "El grupo ya alcanzó su capacidad máxima". 403 Forbidden: Si la operación se realiza fuera del periodo habilitado. Mensaje: "No se pueden gestionar solicitudes fuera del periodo académico habilitado". 400 Bad Request: Si faltan campos obligatorios en la decisión. 500 Internal Server Error: Si ocurre un error durante el procesamiento de la solicitud.
- **Link Swagger:** POST
`/secretariat/requests/{requestId}/respond?startDate={LocalDateTime}&endDate={LocalDateTime}`

10.Funcionalidad: Responder Solicitud (Info Consolidada)

- **descripción de la Funcionalidad:** Versión alternativa del endpoint de respuesta que acepta toda la información consolidada en un solo objeto RespondRequestInfo en el body (decisión y fechas juntas), en lugar de parámetros query separados. Internamente invoca el mismo método de servicio que el endpoint anterior, proporcionando la misma funcionalidad con diferente formato de entrada.
- **Diagramas:**



El cliente envía una petición POST con el requestId en la ruta y el cuerpo RespondRequestInfo conteniendo un objeto RequestDecisionDTO (decision) y un objeto RequestDatesDTO (dates), el sistema extrae la decisión y las fechas del body, invoca internamente respondRequestBySecretariat() con los mismos parámetros, ejecuta el mismo flujo de validaciones y procesamiento descrito en el endpoint anterior y retorna el ChangeRequestDTO actualizado.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 404 Not Found: Si el grupo origen o destino no existe. Mensaje: "Grupo no encontrado: {groupId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el grupo destino no tiene capacidad disponible al aprobar. Mensaje: "El grupo ya alcanzó su capacidad máxima". 403 Forbidden: Si la operación se realiza fuera del periodo habilitado. Mensaje: "No se pueden gestionar solicitudes fuera del periodo académico habilitado". 400 Bad Request: Si faltan campos obligatorios en la decisión o fechas. 500 Internal Server Error: Si ocurre un error durante el procesamiento.
- **Link Swagger:** POST /secretariat/requests/{requestId}/respondInfo

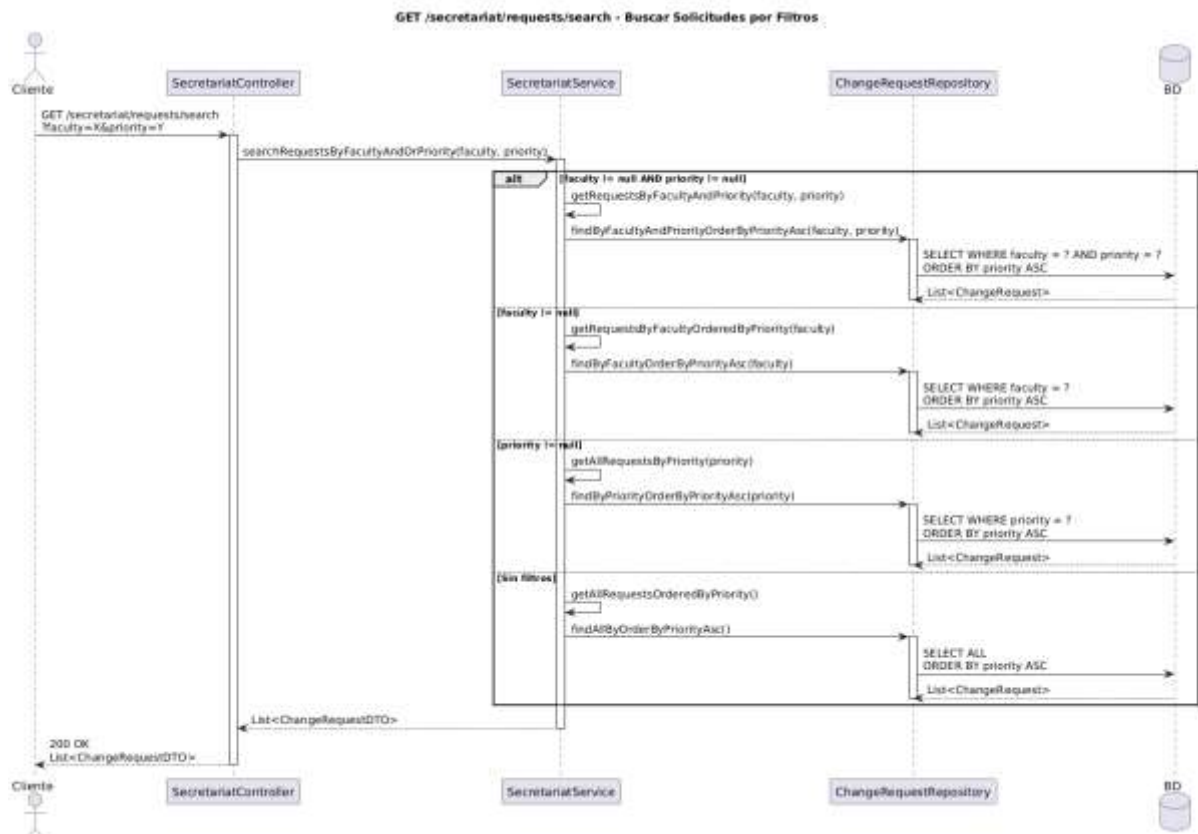
11. Funcionalidad: Obtener Solicitudes por Facultad y Estado

- **descripción de la Funcionalidad:** Permite filtrar y obtener todas las solicitudes de cambio que pertenecen a una facultad específica y tienen un estado determinado (PENDING, APPROVED, REJECTED, etc.). Útil para que la secretaría gestione solicitudes segmentadas por facultad y estado, permitiendo un procesamiento organizado por áreas académicas.
- **Diagramas:**

El cliente envía una petición GET con la facultad y el estado como parámetros de ruta (Faculty enum y RequestStatus enum), el sistema consulta el repositorio mediante findByFacultyAndStatus(faculty, status) que filtra por ambos criterios, mapea cada ChangeRequest a ChangeRequestDTO usando stream y el mapper, y retorna la lista filtrada en formato JSON.
- **Manejo de Errores:** 400 Bad Request: Si el valor de faculty no corresponde a un valor válido del enum Faculty (ENGINEERING, SCIENCES, ARTS, etc.). 400 Bad Request: Si el valor de status no corresponde a un valor válido del enum RequestStatus (PENDING, APPROVED, REJECTED, INFO_REQUESTED, APPROVED_BY_DEANERY). 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna una lista vacía [] si no hay solicitudes que coincidan con los filtros (no es un error).
- **Link Swagger:** GET /secretariat/requests/faculty/{faculty}/status/{status}

12.Funcionalidad: Obtener Solicitudes por Facultad Ordenadas por Prioridad

- **descripción de la Funcionalidad:** Retorna todas las solicitudes de cambio de una facultad específica ordenadas ascendentemente por su nivel de prioridad (1 es más prioritario, números mayores son menos prioritarios). Permite a la secretaría procesar solicitudes de una facultad en el orden de importancia establecido, facilitando la gestión eficiente de casos urgentes o críticos.
- **Diagramas:**

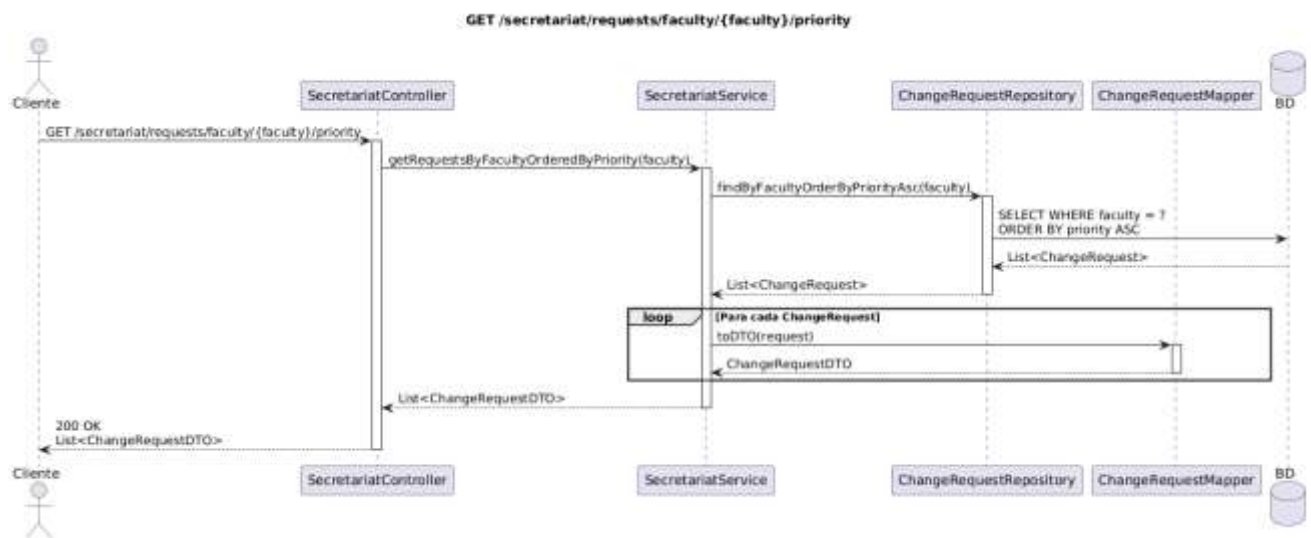


El cliente envía una petición GET con la facultad como parámetro de ruta (Faculty enum), el sistema consulta el repositorio mediante `findByFacultyOrderByPriorityAsc(faculty)` que filtra por facultad y ordena por el campo `priority` de forma ascendente, mapea cada `ChangeRequest` a `ChangeRequestDTO` y retorna la lista ordenada.

- **Manejo de Errores:** 400 Bad Request: Si el valor de faculty no corresponde a un valor válido del enum Faculty. 500 Internal Server Error: Si ocurre un error durante la consulta u ordenamiento. Retorna una lista vacía [] si la facultad no tiene solicitudes (no es un error).
- **Link Swagger:** GET /secretariat/requests/faculty/{faculty}/priority

13.Funcionalidad: Buscar Solicitudes por Filtros

- **descripción de la Funcionalidad:** Endpoint flexible que permite buscar solicitudes combinando opcionalmente filtros de facultad y/o prioridad. Soporta cuatro escenarios: filtrar por ambos criterios, solo por facultad, solo por prioridad, o listar todas las solicitudes. En todos los casos las solicitudes se ordenan ascendentemente por prioridad. Es el endpoint más versátil para consultas de solicitudes.
- **Diagramas:**



El cliente envía una petición GET con parámetros query opcionales faculty y/o priority, el sistema evalúa qué filtros fueron proporcionados mediante condicionales, si ambos están presentes invoca getRequestsByFacultyAndPriority() consultando con findByFacultyAndPriorityOrderByPriorityAsc(), si solo faculty invoca getRequestsByFacultyOrderedByPriority() consultando con findByFacultyOrderByPriorityAsc(), si solo priority invoca getAllRequestsByPriority() consultando con

findByPriorityOrderByPriorityAsc(), si ninguno está presente invoca getAllRequestsOrderedByPriority() consultando con findAllByOrderByPriorityAsc(), mapea los resultados a ChangeRequestDTO y retorna la lista filtrada y ordenada.

- **Manejo de Errores:** 400 Bad Request: Si el valor de faculty no corresponde a un valor válido del enum Faculty. 400 Bad Request: Si el valor de priority no es un número entero válido. 500 Internal Server Error: Si ocurre un error durante la búsqueda. Retorna una lista vacía [] si no hay solicitudes que coincidan con los filtros (no es un error).

Parámetros Query:

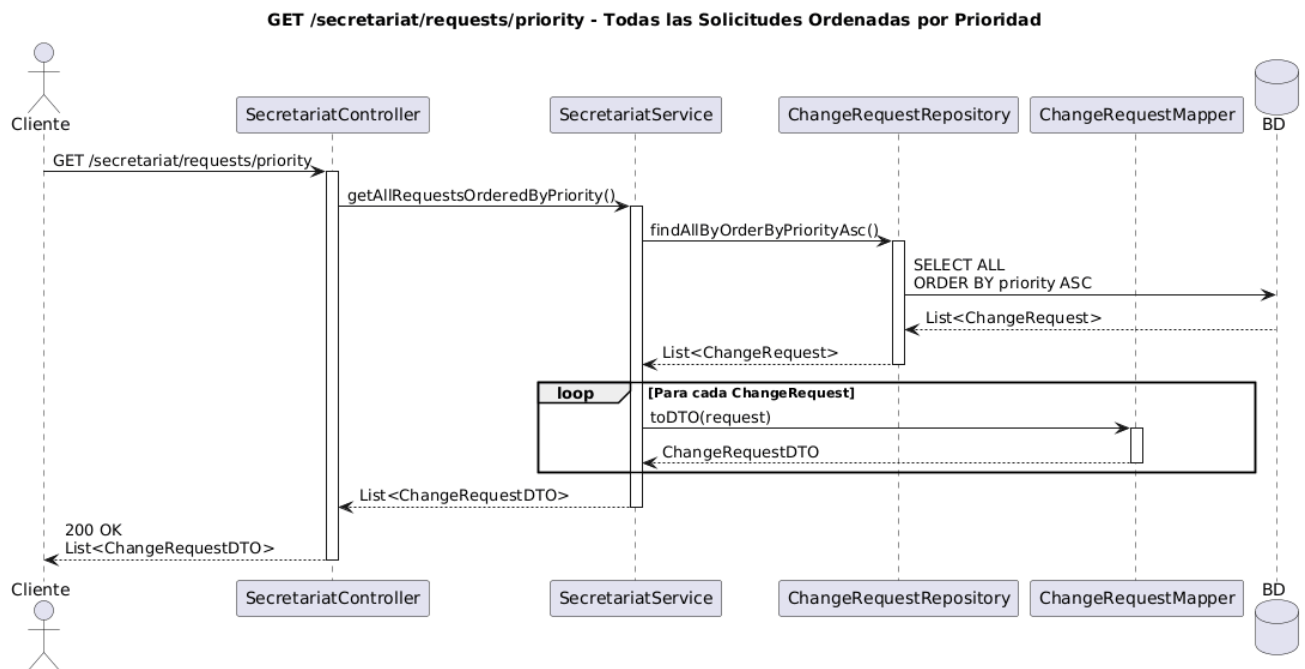
faculty (opcional): Enum Faculty para filtrar por facultad.

priority (opcional): Número entero para filtrar por nivel de prioridad específico.

- **Link Swagger:** GET
`/secretariat/requests/search?faculty={Faculty}&priority={integer}`

14.Funcionalidad: Listar Todas las Solicitudes Ordenadas por Prioridad

- **descripción de la Funcionalidad:** Retorna todas las solicitudes de cambio del sistema completo ordenadas ascendentemente por su nivel de prioridad, independientemente de la facultad o estado. Permite a la secretaría tener una vista global de todas las solicitudes priorizadas, útil para reportes generales o gestión centralizada.
- **Diagramas:**

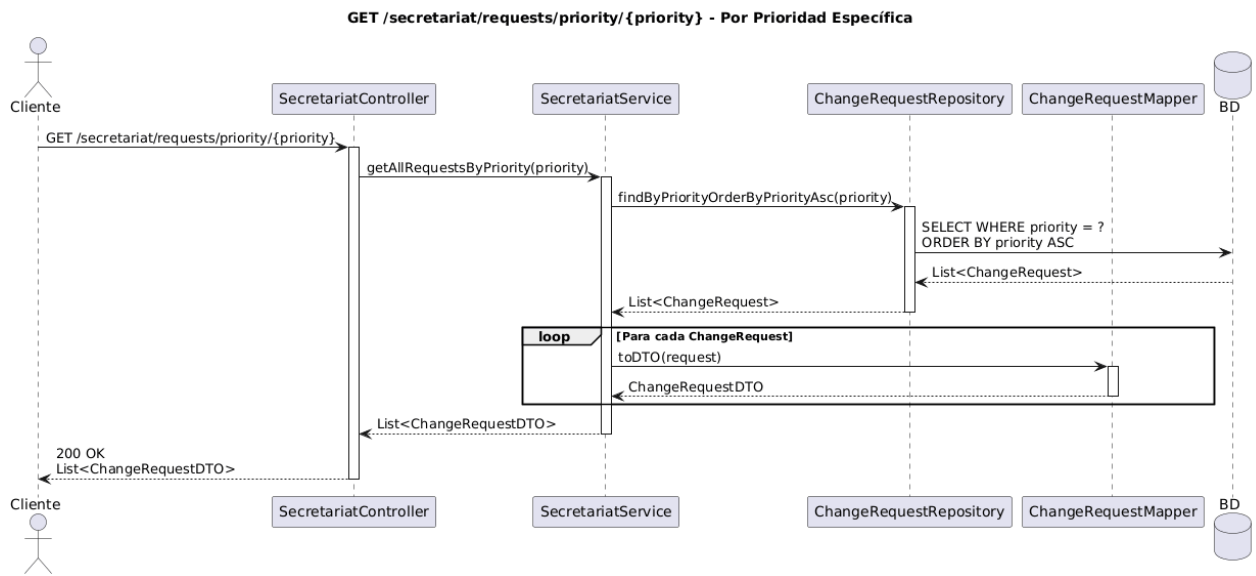


El cliente envía una petición GET sin parámetros, el sistema consulta el repositorio mediante `findAllByOrderByPriorityAsc()` que obtiene todas las solicitudes ordenadas por el campo `priority` de forma ascendente, mapea cada `ChangeRequest` a `ChangeRequestDTO` usando `stream` y el `mapper`, y retorna la lista completa ordenada en formato JSON.

- **Manejo de Errores:** 500 Internal Server Error: Si ocurre un error durante la consulta u ordenamiento. Retorna una lista vacía `[]` si no hay solicitudes en el sistema (no es un error).
- **Link Swagger:** `GET /secretariat/requests/priority`

15.Funcionalidad: Listar Solicitudes por Prioridad Específica

- **descripción de la Funcionalidad:** Retorna todas las solicitudes que tienen exactamente el nivel de prioridad especificado, ordenadas ascendentemente. Permite filtrar solicitudes por un nivel de urgencia específico, útil para procesar lotes de solicitudes con la misma criticidad o para análisis estadísticos por nivel de prioridad.
- **Diagramas:**

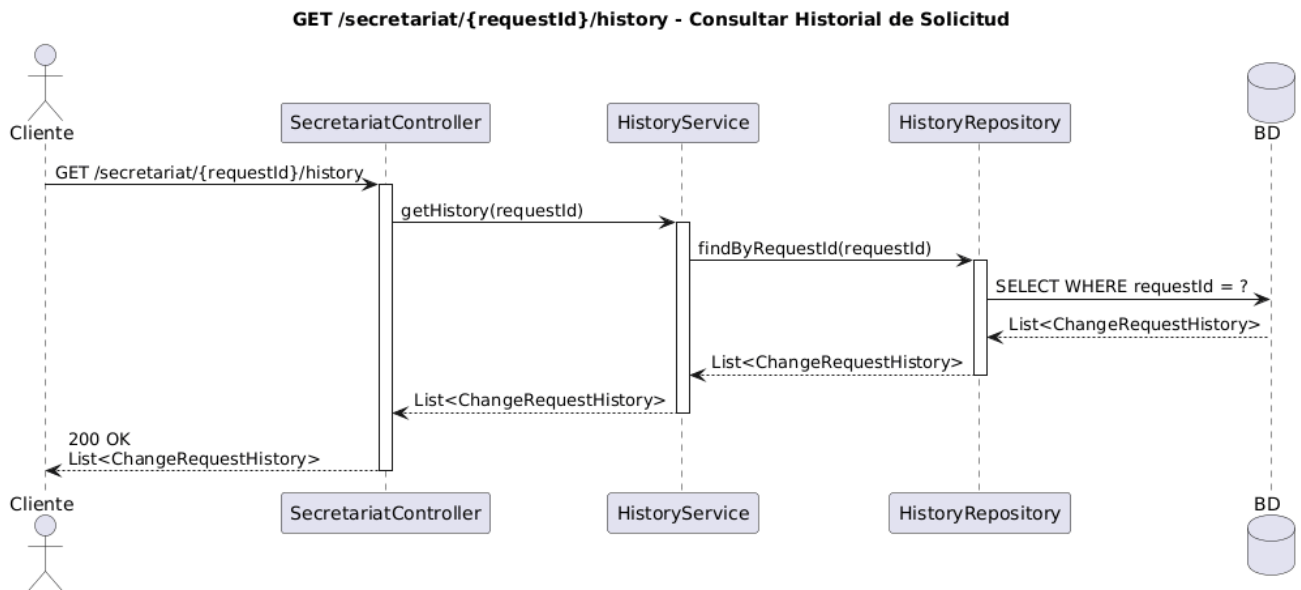


El cliente envía una petición GET con el nivel de prioridad como parámetro de ruta (número entero), el sistema consulta el repositorio mediante `findByPriorityOrderByPriorityAsc(priority)` que filtra por el campo `priority` exacto y ordena ascendentemente, mapea cada `ChangeRequest` a `ChangeRequestDTO` y retorna la lista filtrada.

- **Manejo de Errores:** 400 Bad Request: Si el valor de `priority` no es un número entero válido. 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna una lista vacía `[]` si no hay solicitudes con esa prioridad (no es un error).
- **Link Swagger:** `GET /secretariat/requests/priority/{priority}`

16.Funcionalidad: Consultar Historial de Solicitud

- **descripción de la Funcionalidad:** Retorna el historial completo de eventos y cambios de estado de una solicitud específica. Incluye todos los registros de quién procesó la solicitud, qué cambios se realizaron, cuándo ocurrieron y las observaciones asociadas a cada evento. Permite trazabilidad completa del ciclo de vida de una solicitud desde su creación hasta su resolución final.
- **Diagramas:**

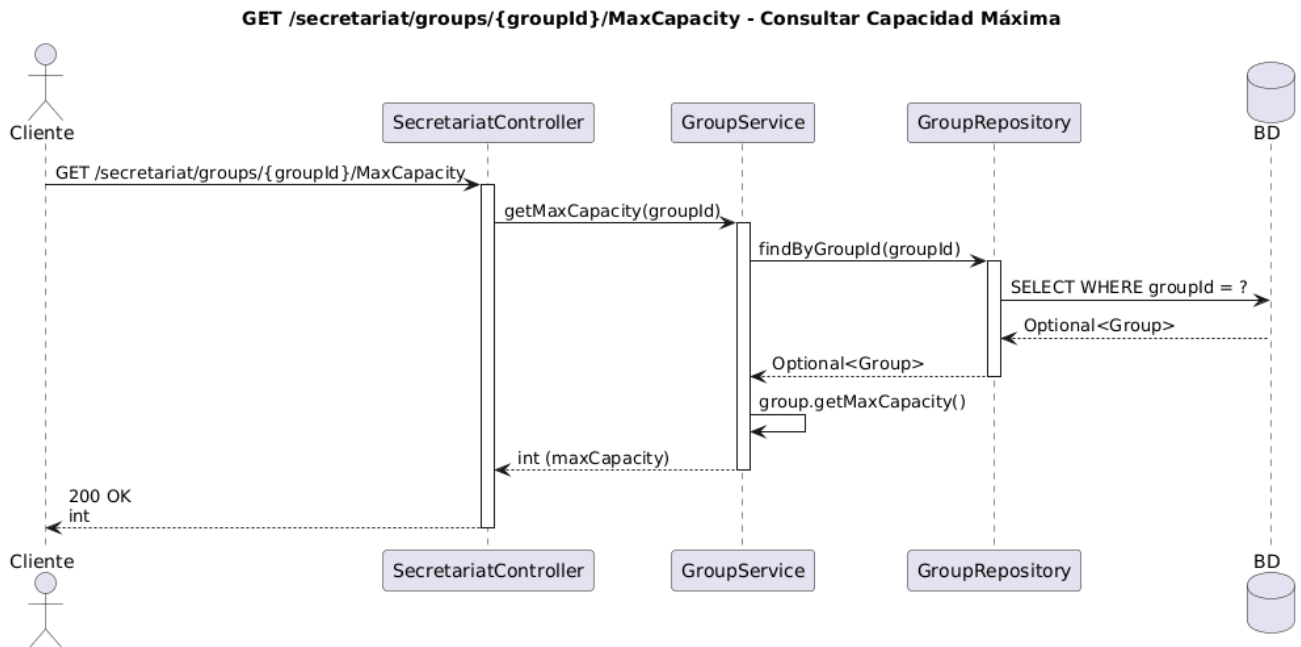


El cliente envía una petición GET con el requestId (UUID) como parámetro de ruta, el sistema invoca el servicio de historial con `getHistory(requestId)` que consulta la tabla de `ChangeRequestHistory` filtrando por el ID de solicitud, ordena los eventos cronológicamente y retorna la lista completa de objetos `ChangeRequestHistory` con toda la información de auditoría.

- **Manejo de Errores:** 400 Bad Request: Si el requestId no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la consulta del historial. Retorna una lista vacía `[]` si la solicitud no tiene eventos registrados (no es un error).
- **Link Swagger:** `GET /secretariat/{requestId}/history`

17.Funcionalidad: Consultar Capacidad Máxima de Grupo

- **descripción de la Funcionalidad:** Retorna la capacidad máxima configurada para un grupo específico, es decir, el número total de estudiantes que el grupo puede aceptar. Este endpoint permite verificar el límite de cupos antes de aprobar solicitudes de cambio o gestionar inscripciones.
- **Diagramas:**

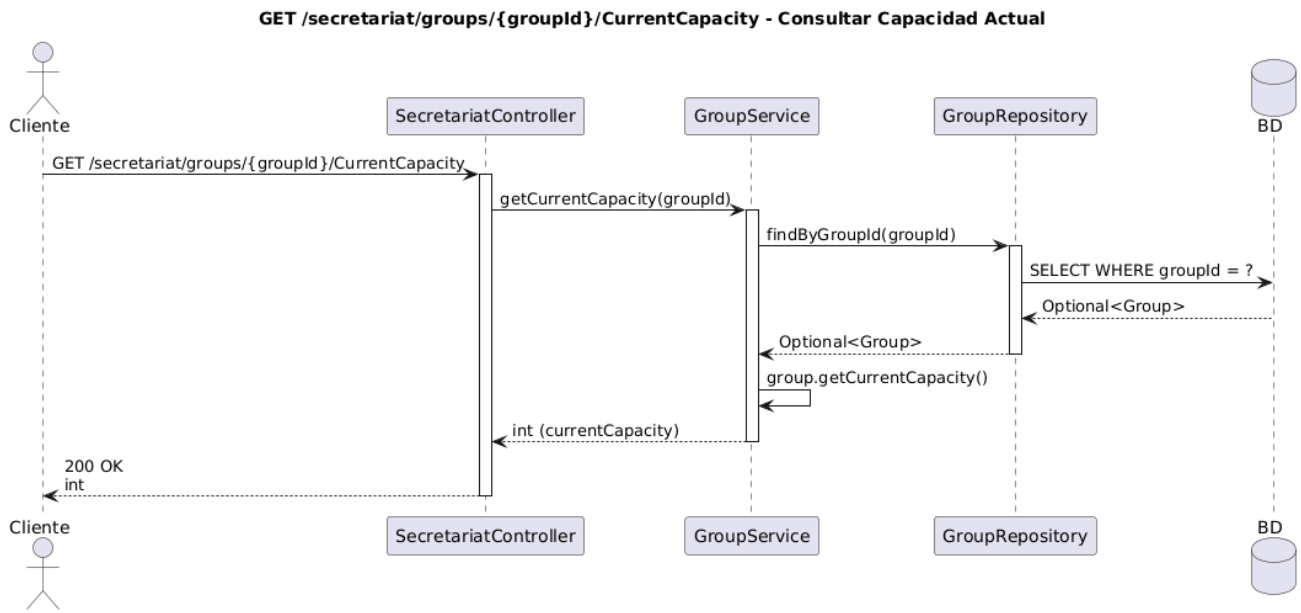


El cliente envía una petición GET con el groupId como parámetro de ruta, el sistema invoca el servicio de grupos con `getMaxCapacity(groupId)` que busca el grupo en el repositorio, valida su existencia y retorna el valor del campo `maximumCapacity` como un entero.

- **Manejo de Errores:** 404 Not Found: Si el grupo con el ID especificado no existe. Mensaje: "Grupo no encontrado: {groupId}". 500 Internal Server Error: Si ocurre un error durante la consulta.
- **Link Swagger:** [GET /secretariat/groups/{groupId}/MaxCapacity](#)

18.Funcionalidad: Consultar Capacidad Actual de Grupo

- **descripción de la Funcionalidad:** Retorna el número actual de estudiantes inscritos en un grupo específico. Permite conocer cuántos cupos están ocupados en tiempo real, útil para determinar si hay espacio disponible antes de aprobar solicitudes de cambio o realizar nuevas inscripciones.
- **Diagramas:**

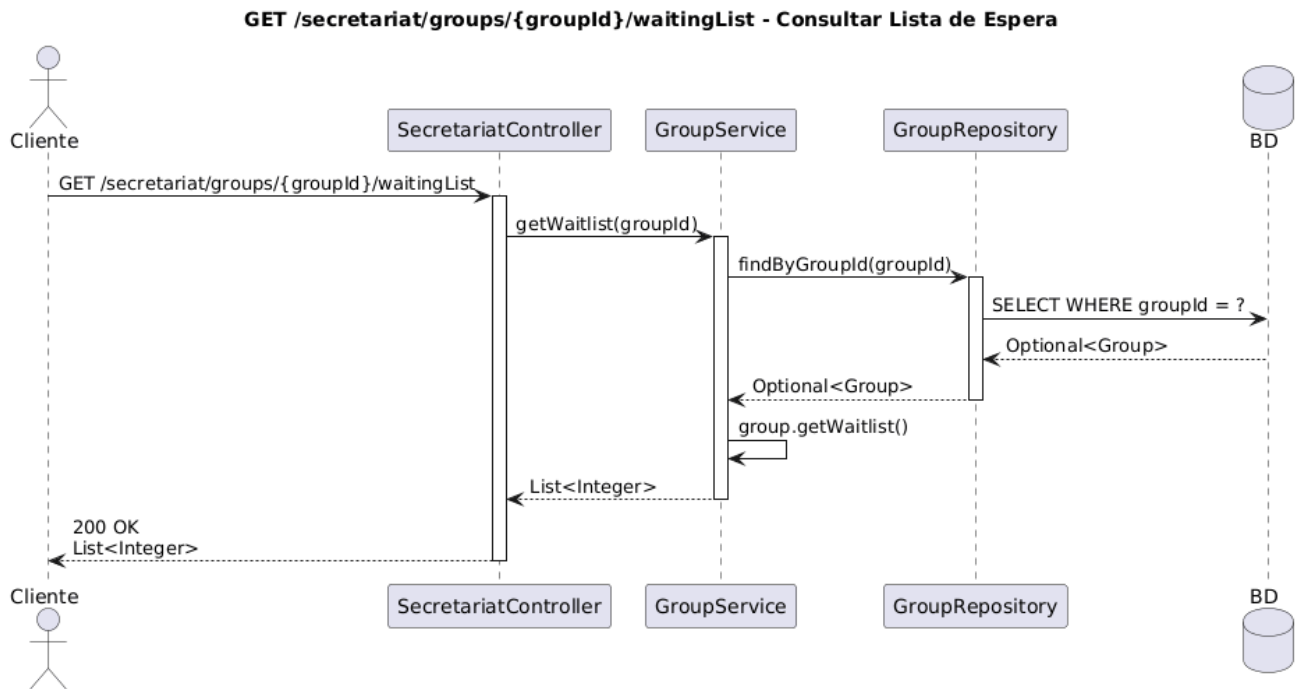


El cliente envía una petición GET con el groupId como parámetro de ruta, el sistema invoca el servicio de grupos con `getCurrentCapacity(groupId)` que busca el grupo en el repositorio, valida su existencia y retorna el valor del campo `currentCapacity` como un entero.

- **Manejo de Errores:** 404 Not Found: Si el grupo con el ID especificado no existe. Mensaje: "Grupo no encontrado: {groupId}". 500 Internal Server Error: Si ocurre un error durante la consulta.
- **Link Swagger:** `GET /secretariat/groups/{groupId}/CurrentCapacity`

19.Funcionalidad: Consultar Lista de Espera de Grupo

- **descripción de la Funcionalidad:** Retorna la lista de IDs de estudiantes que están en lista de espera para un grupo específico que ha alcanzado su capacidad máxima. Los estudiantes en esta lista tienen solicitudes pendientes para ingresar al grupo cuando se liberen cupos. La lista se mantiene en el orden en que los estudiantes solicitaron el cambio.
- **Diagramas:**

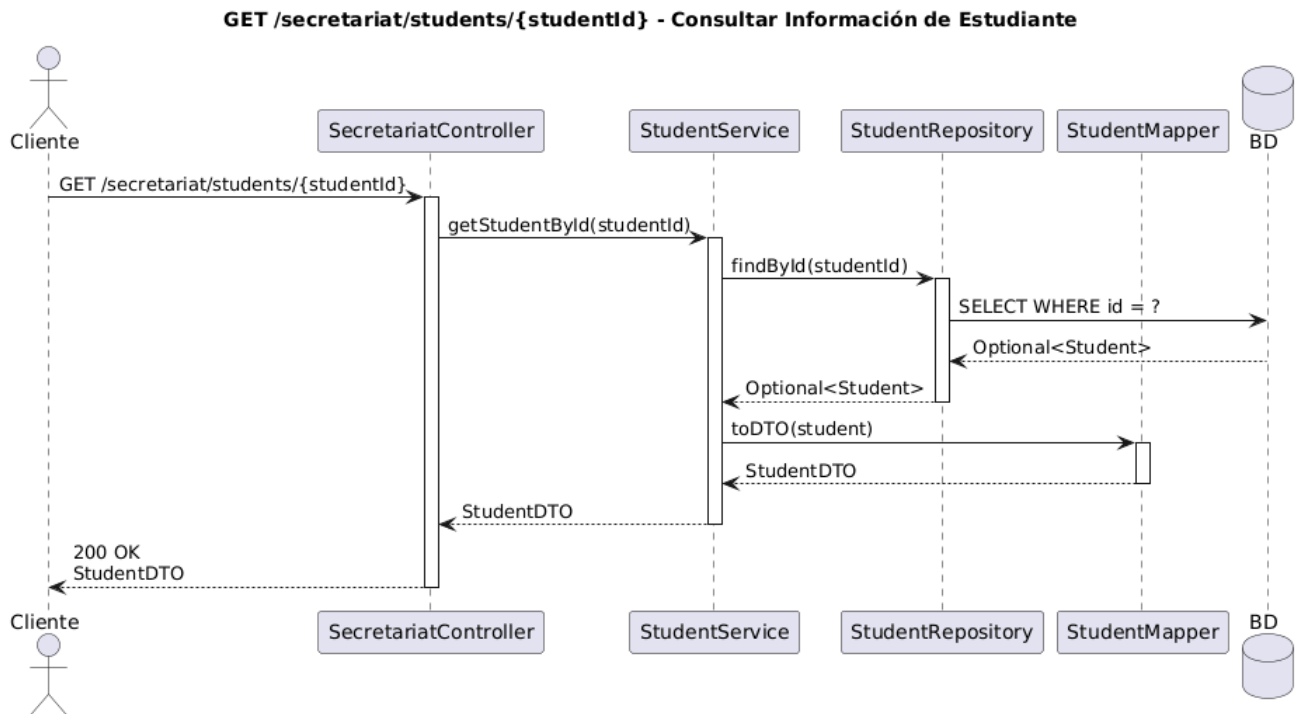


El cliente envía una petición GET con el `groupId` como parámetro de ruta, el sistema invoca el servicio de grupos con `getWaitlist(groupId)` que busca el grupo en el repositorio, valida su existencia, accede al campo `waitlist` (lista de enteros) y retorna la lista completa de IDs de estudiantes en espera.

- **Manejo de Errores:** 404 Not Found: Si el grupo con el ID especificado no existe. Mensaje: "Grupo no encontrado: {groupId}". 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna una lista vacía `[]` si el grupo no tiene estudiantes en lista de espera (no es un error).
- **Link Swagger:** `GET /secretariat/groups/{groupId}/waitingList`

20.Funcionalidad: Consultar Información de Estudiante

- **descripción de la Funcionalidad:** Retorna la información completa y detallada de un estudiante específico incluyendo datos personales, información académica (carrera, semestre, currículum, semáforo académico), materias aprobadas e inscritas, horario actual y solicitudes asociadas. Permite a la secretaría consultar el perfil completo del estudiante al procesar sus solicitudes.
- **Diagramas:**

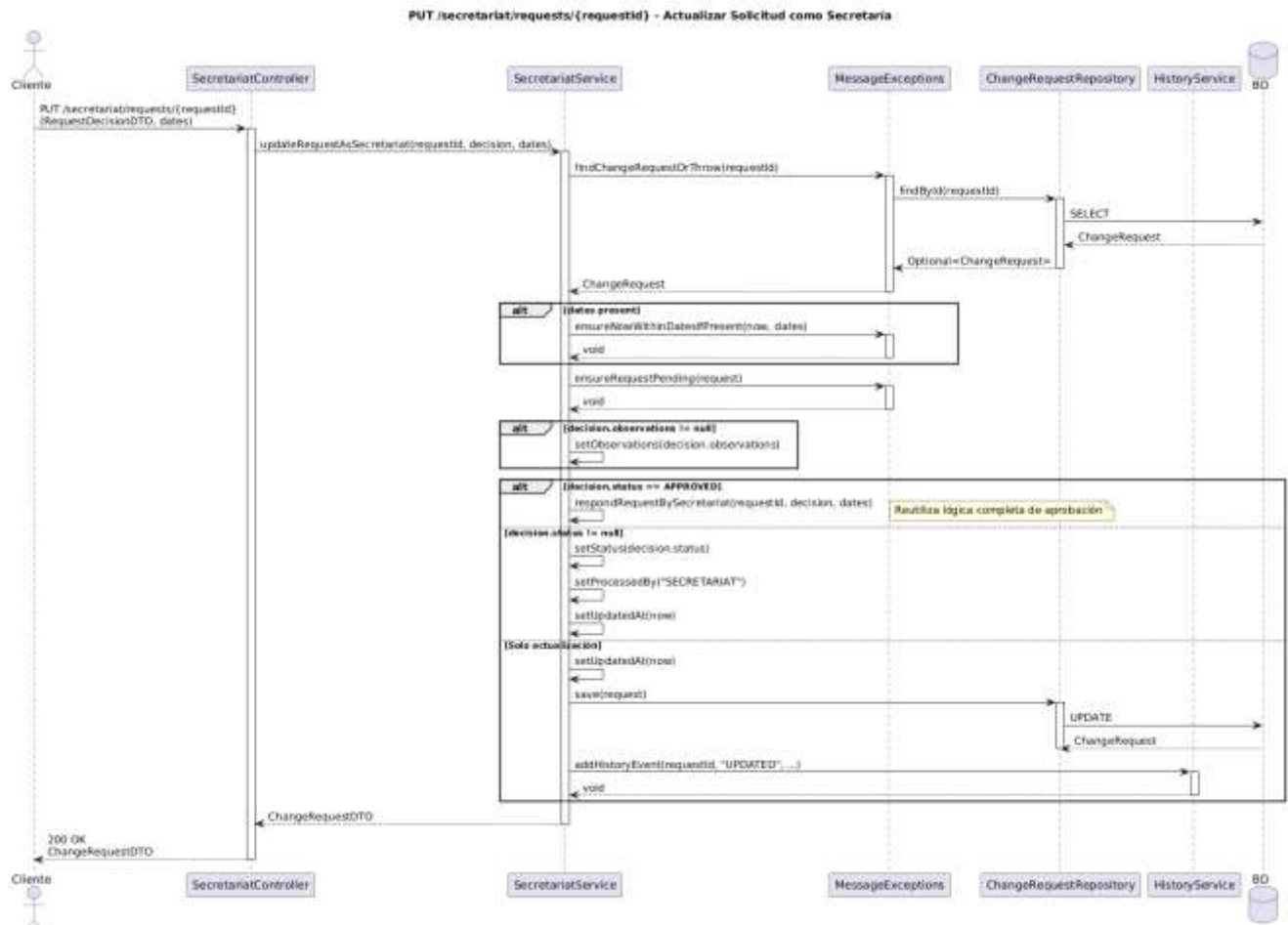


El cliente envía una petición GET con el `studentId` como parámetro de ruta, el sistema invoca el servicio de estudiantes con `getStudentById(studentId)` que busca el estudiante mediante `findById(studentId)`, valida su existencia, mapea toda la información de la entidad `Student` a `StudentDTO` incluyendo relaciones y retorna el DTO completo.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error durante la consulta.
- **Link Swagger:** `GET /secretariat/students/{studentId}`

21.Funcionalidad: Actualizar Solicitud como Secretaría

- **descripción de la Funcionalidad:** Permite a la secretaría modificar los detalles de una solicitud pendiente sin cambiarle el estado definitivamente, o cambiar el estado si es necesario. Valida que la solicitud esté en estado PENDING y que la operación se realice dentro del periodo habilitado. Si se cambia el estado a APPROVED, invoca automáticamente el flujo completo de aprobación de solicitudes. Si solo se actualizan observaciones sin cambiar estado, registra el evento como "UPDATED".
- **Diagramas:**



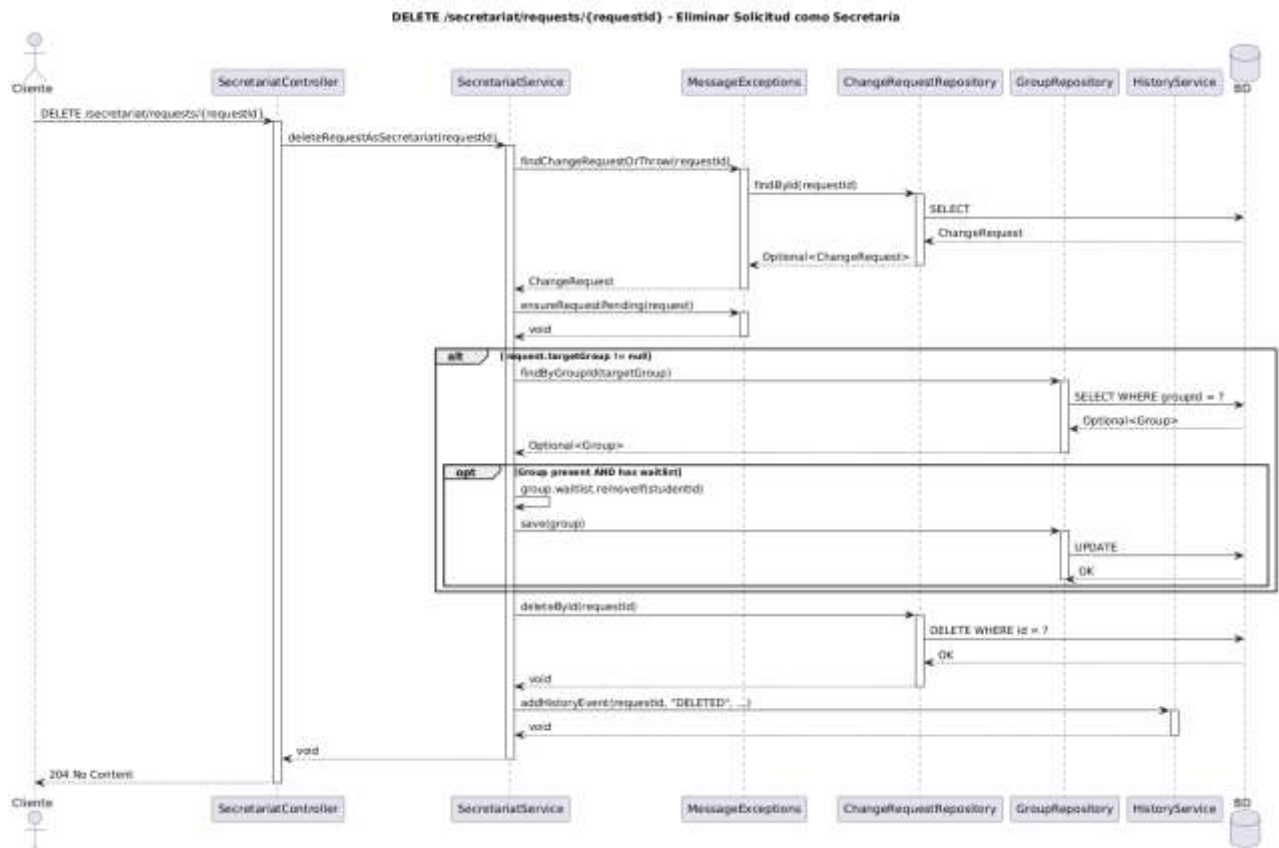
El cliente envía una petición PUT con el requestId en la ruta, el cuerpo RequestDecisionDTO con los campos a actualizar y opcionalmente parámetros query startDate y endDate, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté dentro de las fechas permitidas con ensureNowWithinDatesIfPresent(), valida que esté en estado PENDING mediante ensureRequestPending(), si el DTO contiene observaciones actualiza el campo observations, si el DTO

contiene un nuevo status actualiza el estado y marca processedBy como "SECRETARIAT", si el nuevo estado es APPROVED invoca internamente respondRequestBySecretariat() para ejecutar el flujo completo de aprobación, si no hay cambio de estado solo actualiza timestamp y persiste, registra el evento en el historial y retorna el ChangeRequestDTO actualizado.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 403 Forbidden: Si la operación se realiza fuera del periodo habilitado. Mensaje: "No se pueden gestionar solicitudes fuera del periodo académico habilitado". 400 Bad Request: Si al aprobar, el grupo destino no tiene capacidad. Mensaje: "El grupo ya alcanzó su capacidad máxima". 500 Internal Server Error: Si ocurre un error durante la actualización.
 - Parámetros Query:
 - startDate (opcional): Fecha de inicio del periodo válido para actualizar la solicitud.
 - endDate (opcional): Fecha de fin del periodo válido para actualizar la solicitud.
- **Link Swagger:** PUT
/secretariat/requests/{requestId}?startDate={LocalDateTime}&endDate={LocalDateTime}

22.Funcionalidad: Eliminar Solicitud como Secretaria

- **descripción de la Funcionalidad:** Permite a la secretaria eliminar permanentemente una solicitud pendiente del sistema. Valida que la solicitud esté en estado PENDING antes de eliminarla. Si el estudiante estaba en lista de espera del grupo destino, lo remueve automáticamente. Registra el evento de eliminación en el historial antes de borrar la solicitud. Retorna código 204 No Content al completarse exitosamente.
- **Diagramas:**

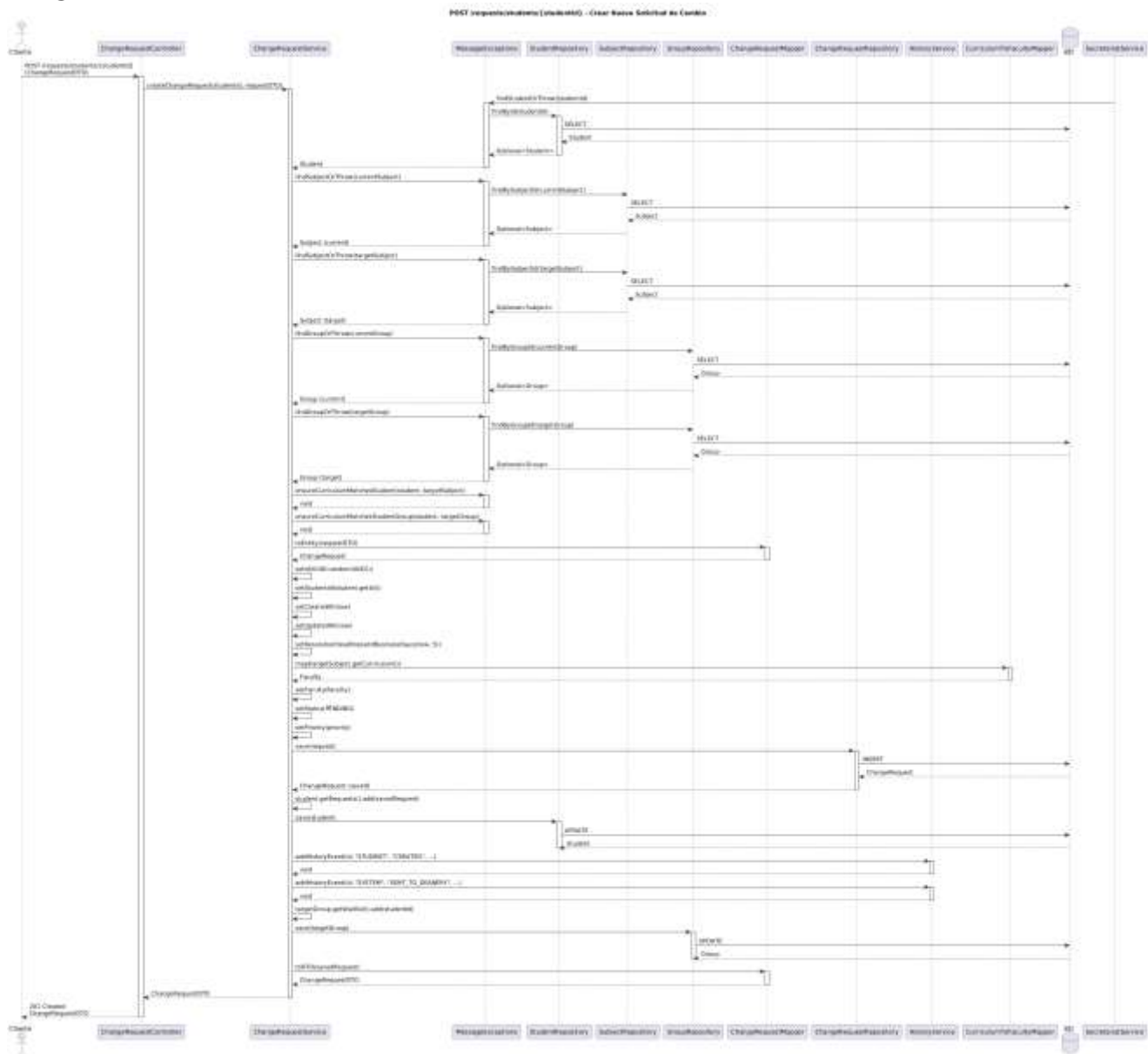


El cliente envía una petición DELETE con el requestId como parámetro de ruta, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté en estado PENDING mediante ensureRequestPending(), si la solicitud tiene un grupo destino busca el grupo con findGroupById(), si el grupo tiene lista de espera remueve el studentId de la lista usando removeIf(), persiste los cambios del grupo si hubo modificaciones, registra el evento "DELETED" en el historial con addHistoryEvent(), elimina la solicitud del repositorio con deleteById() y retorna estado HTTP 204 sin contenido.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el requestId no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la eliminación.
- **Link Swagger:** DELETE /secretariat/requests/{requestId}

23.Funcionalidad: Crear Nueva Solicitud de Cambio

- **descripción de la Funcionalidad:** Permite a un estudiante crear una nueva solicitud de cambio de grupo o materia. El sistema valida que las materias y grupos objetivo pertenezcan al currículum del estudiante, calcula automáticamente la prioridad basada en el número de solicitudes previas del estudiante, establece una fecha límite de resolución de 5 días hábiles, asigna el estado inicial como PENDING, envía la solicitud a decanatura (SENT_TO_DEANERY), agrega al estudiante a la lista de espera del grupo destino y registra todos los eventos en el historial. Retorna la solicitud creada con código 201 Created.
- **Diagramas:**

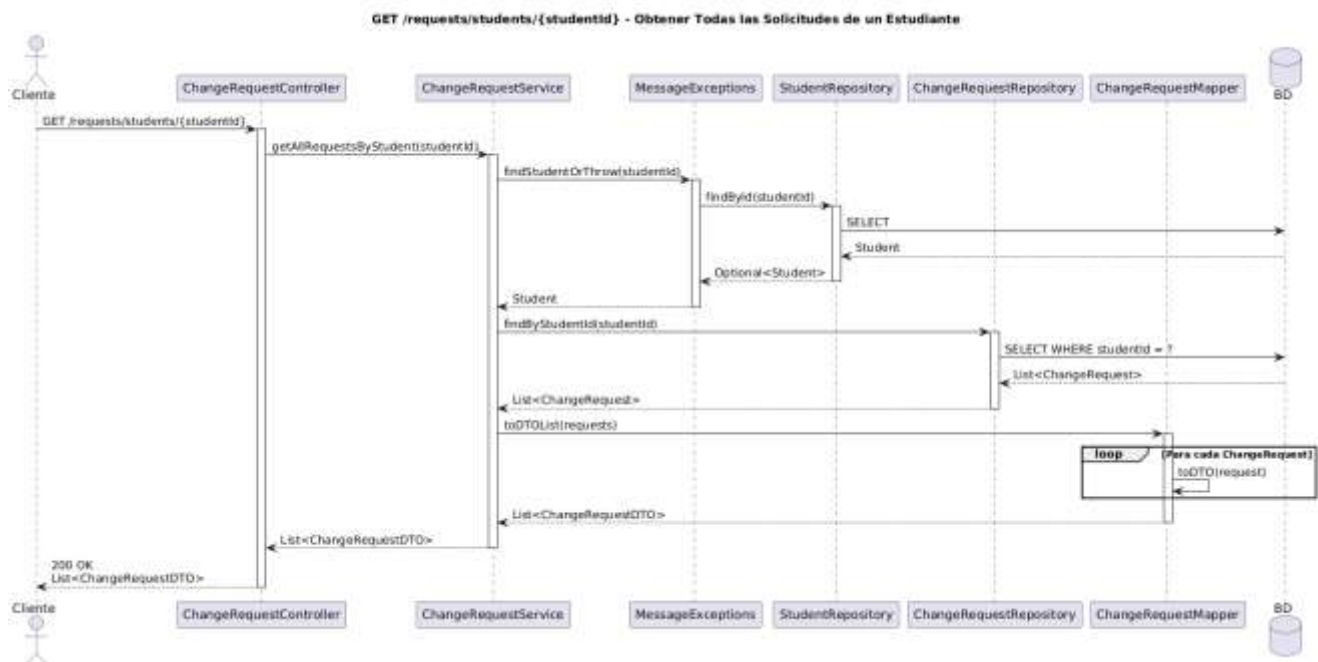


El cliente envía una petición POST con el `studentId` en la ruta y el cuerpo `ChangeRequestDTO` validado con `@Valid` conteniendo `currentSubject`, `currentGroup`, `targetSubject`, `targetGroup` y observaciones opcionales, el sistema busca y valida la existencia del estudiante con `findStudentOrThrow()`, busca y valida las materias origen y destino con `findSubjectOrThrow()`, busca y valida los grupos origen y destino con `findGroupOrThrow()`, valida que la materia destino pertenezca al currículum del estudiante mediante `ensureCurriculumMatchesStudent()`, valida que el grupo destino pertenezca al currículum del estudiante mediante `ensureCurriculumMatchesStudentGroup()`, mapea el DTO a entidad `ChangeRequest`, genera un UUID único para la solicitud, calcula la prioridad como número de solicitudes previas + 1, establece timestamps (`createdAt`, `updatedAt`), calcula `resolutionDeadline` sumando 5 días hábiles con `addBusinessDays()`, mapea el currículum a facultad mediante `curriculumToFacultyMapper`, establece estado inicial como `PENDING` (aunque también se marca `SENT_TO_DEANERY` para histórico), persiste la solicitud con `save()`, agrega la solicitud a la lista de requests del estudiante, persiste el estudiante actualizado, agrega al estudiante a la waitlist del grupo destino si no está presente, persiste el grupo actualizado, registra dos eventos en historial: "CREATED" por el estudiante y "SENT_TO_DEANERY" por el sistema, y retorna el `ChangeRequestDTO` de la solicitud creada.

- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 404 Not Found: Si alguna materia (origen o destino) no existe. Mensaje: "Materia no encontrada: {subjectId}". 404 Not Found: Si algún grupo (origen o destino) no existe. Mensaje: "Grupo no encontrado: {groupId}". 400 Bad Request: Si la materia destino no pertenece al currículo del estudiante. Mensaje: "Materia objetivo del pensum no coincide con el pensum del estudiante". 400 Bad Request: Si el grupo destino no pertenece al currículo del estudiante. Mensaje: "Grupo objetivo del pensum no coincide con el pensum del estudiante". 400 Bad Request: Si faltan campos obligatorios en el DTO (currentSubject, currentGroup, targetSubject, targetGroup). Mensaje: "Validation failed" con detalles. 500 Internal Server Error: Si ocurre un error durante la creación o persistencia.
- **Link Swagger:** POST /requests/students/{studentId}

24.Funcionalidad: Obtener Todas las Solicitudes del Estudiante

- **descripción de la Funcionalidad:** Retorna el listado completo de todas las solicitudes de cambio creadas por un estudiante específico, independientemente de su estado o tipo. Incluye solicitudes normales y excepcionales, en cualquier estado (PENDING, APPROVED, REJECTED, etc.). Útil para que el estudiante visualice todo su historial de solicitudes.
- **Diagramas:**



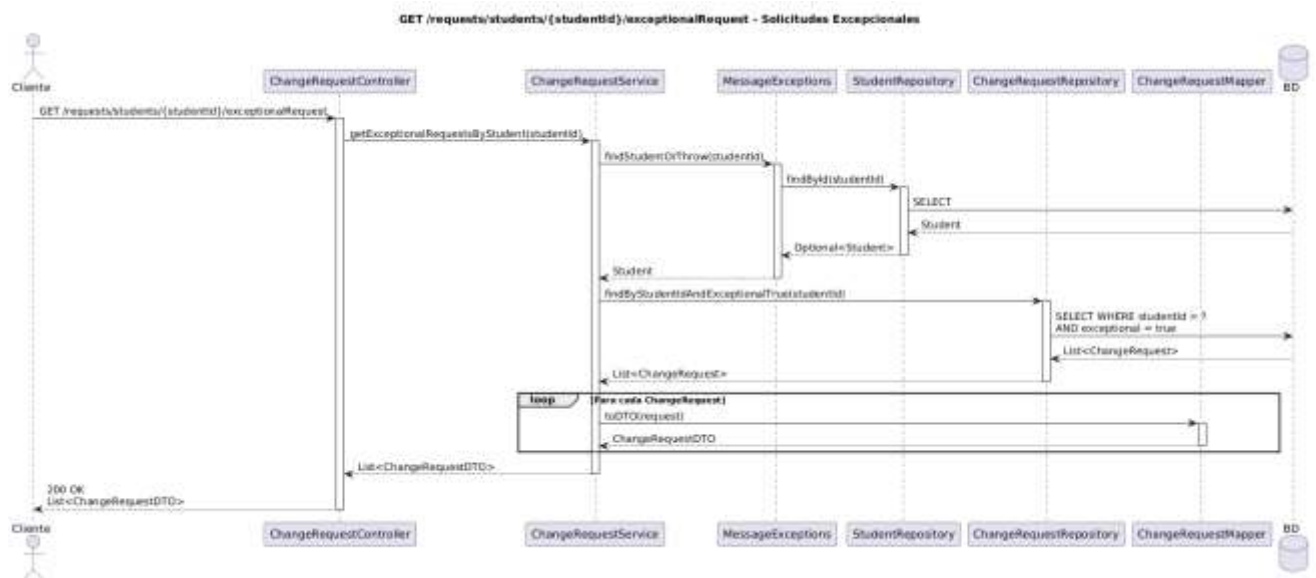
El cliente envía una petición GET con el studentId como parámetro de ruta, el sistema valida la existencia del estudiante mediante findStudentOrThrow(), consulta el repositorio filtrando por studentId

con `findByStudentId()`, mapea cada `ChangeRequest` a `ChangeRequestDTO` mediante `toDTOList()` y retorna la lista completa en formato JSON.

- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el studentId no es un número válido. 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna una lista vacía [] si el estudiante no tiene solicitudes (no es un error).
- **Link Swagger:** `GET /requests/students/{studentId}`

25.Funcionalidad: Obtener Solicitudes Excepcionales del Estudiante

- **descripción de la Funcionalidad:** Retorna únicamente las solicitudes de cambio que han sido marcadas como excepcionales (`exceptional: true`) para un estudiante específico. Las solicitudes excepcionales requieren aprobación de la decanatura antes de ser procesadas por secretaría, generalmente porque involucran casos especiales fuera de las reglas normales de cambio.
- **Diagramas:**



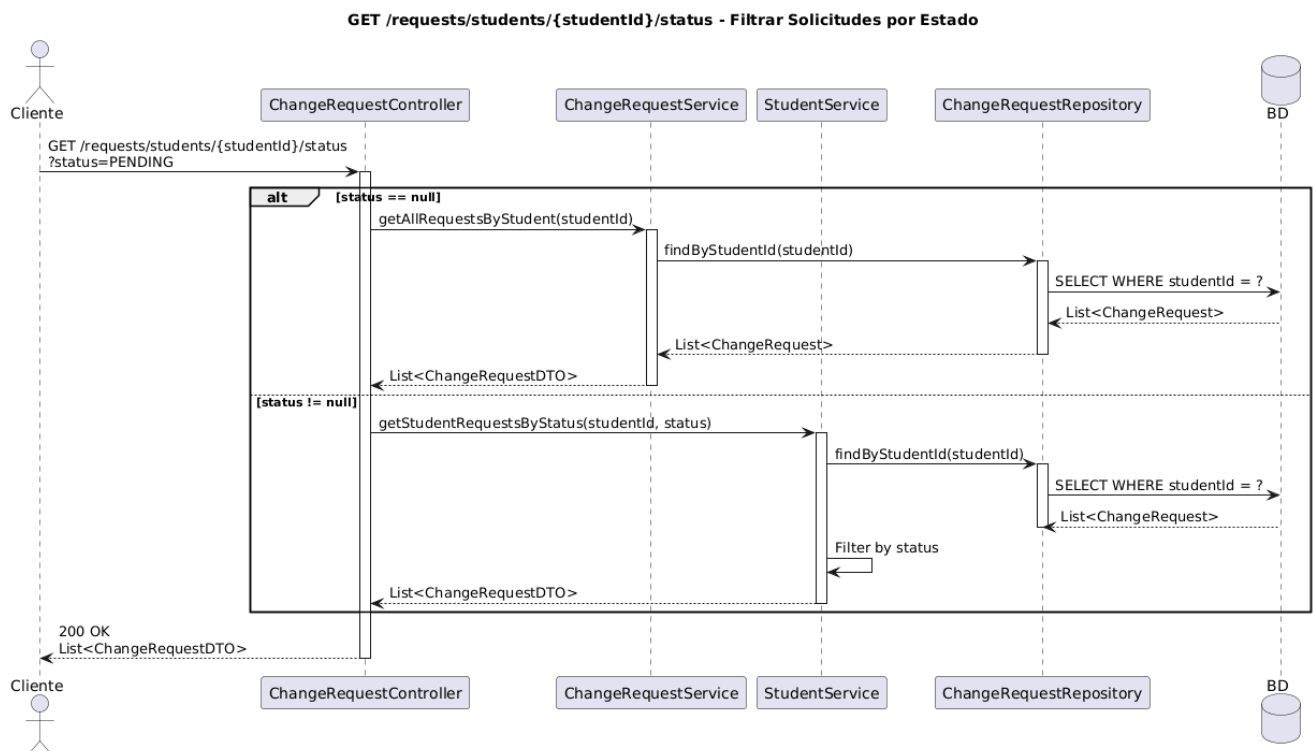
El cliente envía una petición GET con el `studentId` como parámetro de ruta, el sistema valida la existencia del estudiante mediante `findStudentOrThrow()`, consulta el repositorio filtrando por `studentId` y `exceptional=true` con `findByStudentIdAndExceptionalTrue()`, mapea

cada ChangeRequest a ChangeRequestDTO usando stream y mapper, y retorna la lista filtrada.

- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el studentId no es un número válido. 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna una lista vacía [] si el estudiante no tiene solicitudes excepcionales (no es un error).
- **Link Swagger:** GET /requests/students/{studentId}/exceptionalRequest

26.Funcionalidad: Filtrar Solicitudes por Estado

- **descripción de la Funcionalidad:** Permite obtener las solicitudes de un estudiante filtradas opcionalmente por su estado (PENDING, APPROVED, REJECTED, etc.). Si se proporciona el parámetro query status, retorna solo las solicitudes en ese estado; si no se proporciona o es null, retorna todas las solicitudes del estudiante sin filtro. Es similar al endpoint general de solicitudes del estudiante pero con capacidad de filtrado.
- **Diagramas:**



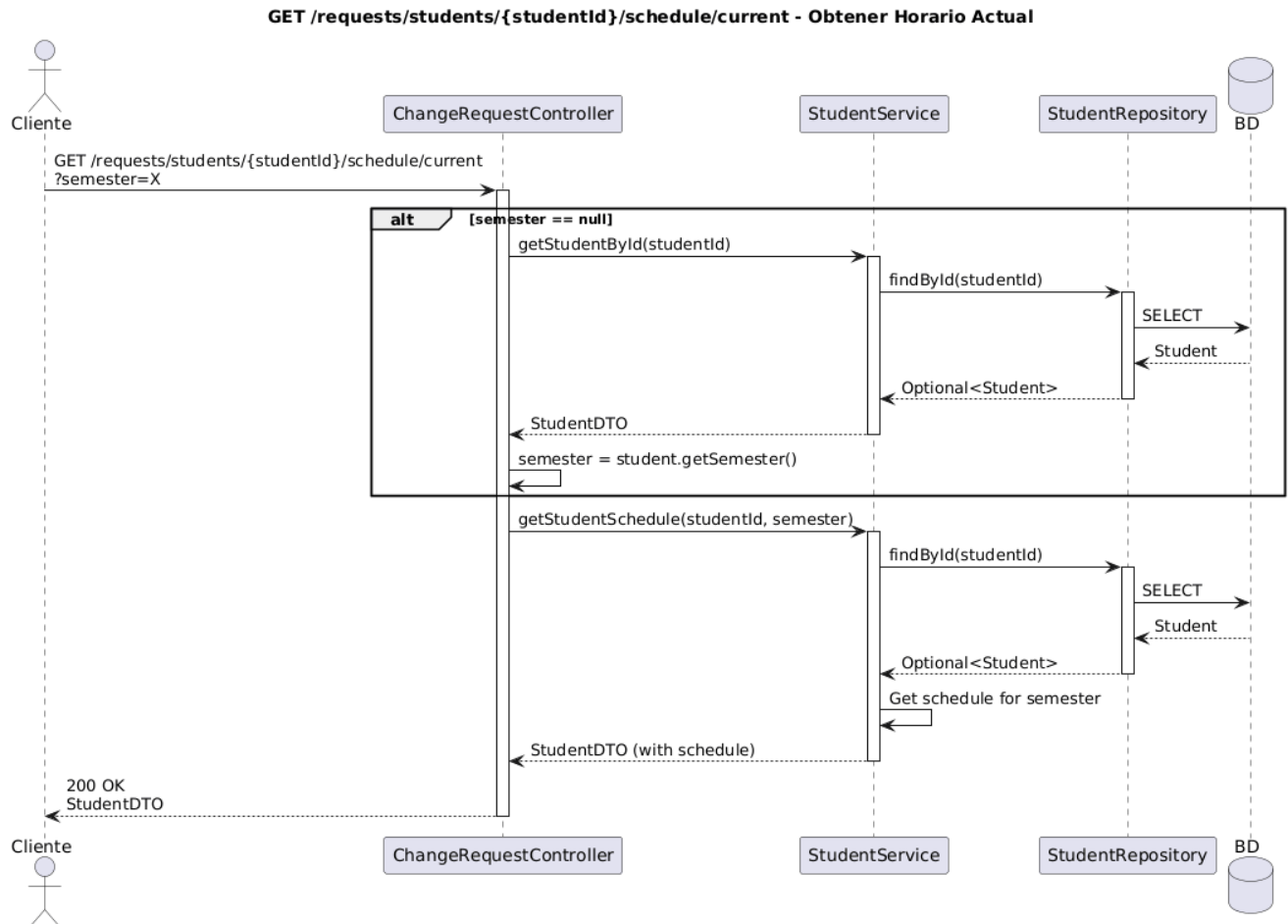
El cliente envía una petición GET con el studentId en la ruta y opcionalmente el parámetro query status (RequestStatus enum), el sistema verifica si el parámetro status es null, si es null invoca

getAllRequestsByStudent() que retorna todas las solicitudes del estudiante, si no es null invoca getStudentRequestsByStatus() que filtra las solicitudes por el estado específico, ambos métodos validan la existencia del estudiante, consultan el repositorio, filtran si es necesario y retornan la lista de ChangeRequestDTO.

- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el studentId no es un número válido. 400 Bad Request: Si el valor de status no corresponde a un valor válido del enum RequestStatus (PENDING, APPROVED, REJECTED, INFO_REQUESTED, REQUEST_ADDITIONAL_INFO, APPROVED_BY_DEANERY, SENT_TO_DEANERY). 500 Internal Server Error: Si ocurre un error durante la consulta o filtrado. Retorna una lista vacía [] si no hay solicitudes que coincidan con el filtro (no es un error).
- **Link Swagger:** GET
/requests/students/{studentId}/status?status={RequestStatus}

27.Funcionalidad: Obtener Horario Actual del Estudiante

- **descripción de la Funcionalidad:** Retorna el horario de clases actual de un estudiante para un semestre específico o su semestre actual si no se especifica. Incluye todas las entradas de horario (schedule entries) con información de materias, grupos, horarios y días de clase. El semestre puede proporcionarse como parámetro query opcional; si no se proporciona, usa el semestre actual registrado en el perfil del estudiante.
- **Diagramas:**



El cliente envía una petición GET con el studentId en la ruta y opcionalmente el parámetro query semester, el sistema verifica si semester es null, si es null consulta el estudiante completo con getStudentById() y extrae su semestre actual del campo semester, con el semestre determinado invoca getStudentSchedule(studentId, semester), este método busca el estudiante, filtra las entradas de schedule donde el campo semester coincida, mapea a ScheduleEntryDTO, construye un StudentDTO con la información

básica y el horario filtrado, y retorna el DTO completo con el horario del semestre especificado.

- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el studentId no es un número válido. 400 Bad Request: Si el parámetro semester no es un número entero válido. 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna un StudentDTO con lista de schedule vacía si el estudiante no tiene horario para ese semestre (no es un error).
- **Link Swagger:** GET
/requests/students/{studentId}/schedule/current?semester={integer}

28.Funcionalidad: Eliminar Solicitud como Secretaría

- **descripción de la Funcionalidad:** Permite a la secretaría eliminar permanentemente una solicitud pendiente del sistema. Valida que la solicitud esté en estado PENDING antes de eliminarla. Si el estudiante estaba en lista de espera del grupo destino, lo remueve automáticamente. Registra el evento de eliminación en el historial antes de borrar la solicitud. Retorna código 204 No Content al completarse exitosamente.
- **Diagramas:**

El cliente envía una petición DELETE con el requestId como parámetro de ruta, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté en estado PENDING mediante ensureRequestPending(), si la solicitud tiene un grupo destino busca el grupo con findByGroupId(), si el grupo tiene lista de espera remueve el studentId de la lista usando removeIf(), persiste los cambios del grupo si hubo modificaciones, registra el evento "DELETED" en el historial con addHistoryEvent(), elimina la solicitud del repositorio con deleteById() y retorna estado HTTP 204 sin contenido.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el requestId no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la eliminación.
- **Parámetros Query:** semester (opcional): Número entero del semestre. Si se omite, usa el semestre actual del estudiante.
- **Link Swagger:** DELETE /secretariat/requests/{requestId}

29.Funcionalidad: Obtener Horario Anterior del Estudiante

- **descripción de la Funcionalidad:** Retorna el horario de clases de un estudiante para un semestre anterior específico que debe ser proporcionado obligatoriamente. Permite consultar horarios históricos de semestres previos para comparación o referencia. El parámetro semester es requerido en este endpoint.
- **Diagramas:**

El cliente envía una petición GET con el studentId en la ruta y el parámetro query obligatorio semester, el sistema invoca directamente `getStudentSchedule(studentId, semester)` con el valor proporcionado, busca el estudiante mediante `findStudentOrThrow()`, filtra las entradas de `schedule` donde el campo `semester` coincida con el valor especificado, mapea a `ScheduleEntryDTO`, construye un `StudentDTO` con información básica y el horario filtrado del semestre anterior, y retorna el DTO completo.
- **Manejo de Errores:** 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el studentId no es un número válido. 400 Bad Request: Si el parámetro semester no es proporcionado (es requerido). 400 Bad Request: Si el parámetro semester no es un número entero válido. 500 Internal Server Error: Si ocurre un error durante la consulta. Retorna un `StudentDTO` con lista de `schedule` vacía si el estudiante no tiene horario para ese semestre anterior (no es un error).
- **Link Swagger:** GET
/requests/students/{studentId}/schedule/previous?semester={integer}

30.Funcionalidad: Solicitar Revisión Excepcional

- **descripción de la Funcionalidad:** Permite a un estudiante marcar una solicitud existente como excepcional, solicitando revisión especial por parte de la decanatura. Esto se usa cuando la solicitud de cambio no cumple con los criterios normales pero el estudiante tiene razones especiales válidas. El sistema marca la solicitud con `exceptional=true`, registra la razón, establece quién solicitó la excepción, marca `timestamp` y calcula una nueva fecha límite de resolución de 5 días hábiles para la decanatura. Registra el evento en el historial para trazabilidad completa.

- **Diagramas:**

El cliente envía una petición POST con `studentId` y `requestId` en la ruta y opcionalmente el parámetro query `reason` explicando por qué solicita revisión excepcional, el sistema busca la solicitud con `findChangeRequestOrThrow()`, valida que la solicitud pertenezca al estudiante mediante `ensureStudentOwnsRequest()`, marca el campo `exceptional` como `true`, registra la razón en `exceptionalReason`, establece `exceptionalRequestedBy` como `"STUDENT:{studentId}"`, marca `exceptionalRequestedAt` con `timestamp` actual, calcula `exceptionalResolutionDeadline` sumando 5 días hábiles con `addBusinessDays()`, persiste la solicitud actualizada con `save()`, registra el evento `"EXCEPTION_REQUESTED"` en el historial con la razón proporcionada y retorna el `ChangeRequestDTO` actualizado.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 404 Not Found: Si el estudiante no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si la solicitud no pertenece al estudiante. Mensaje: "Esta solicitud no pertenece al estudiante con id: {studentId}". 400 Bad Request: Si el `requestId` no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la actualización.

Parámetros Query:

`reason` (opcional): Texto explicando la razón de la solicitud excepcional.

- **Link Swagger:** POST

`/requests/{requestId}/students/{studentId}/requestExceptional?reason={string}`

31.Funcionalidad: Eliminar Solicitud como Secretaría

- **descripción de la Funcionalidad:** Permite a la secretaría eliminar permanentemente una solicitud pendiente del sistema. Valida que la solicitud esté en estado PENDING antes de eliminarla. Si el estudiante estaba en lista de espera del grupo destino, lo remueve automáticamente. Registra el evento de eliminación en el historial antes de borrar la solicitud. Retorna código 204 No Content al completarse exitosamente.
- **Diagramas:**

El cliente envía una petición DELETE con el requestId como parámetro de ruta, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté en estado PENDING mediante ensureRequestPending(), si la solicitud tiene un grupo destino busca el grupo con findByGroupId(), si el grupo tiene lista de espera remueve el studentId de la lista usando removeIf(), persiste los cambios del grupo si hubo modificaciones, registra el evento "DELETED" en el historial con addHistoryEvent(), elimina la solicitud del repositorio con deleteById() y retorna estado HTTP 204 sin contenido.
- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestId}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el requestId no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la eliminación.
- **Link Swagger:** DELETE /secretariat/requests/{requestId}

32.Funcionalidad: Eliminar Solicitud como Secretaría

- **descripción de la Funcionalidad:** Permite a la secretaría eliminar permanentemente una solicitud pendiente del sistema. Valida que la solicitud esté en estado PENDING antes de eliminarla. Si el estudiante estaba en lista de espera del grupo destino, lo remueve automáticamente. Registra el evento de eliminación en el historial antes de borrar la solicitud. Retorna código 204 No Content al completarse exitosamente.
- **Diagramas:**

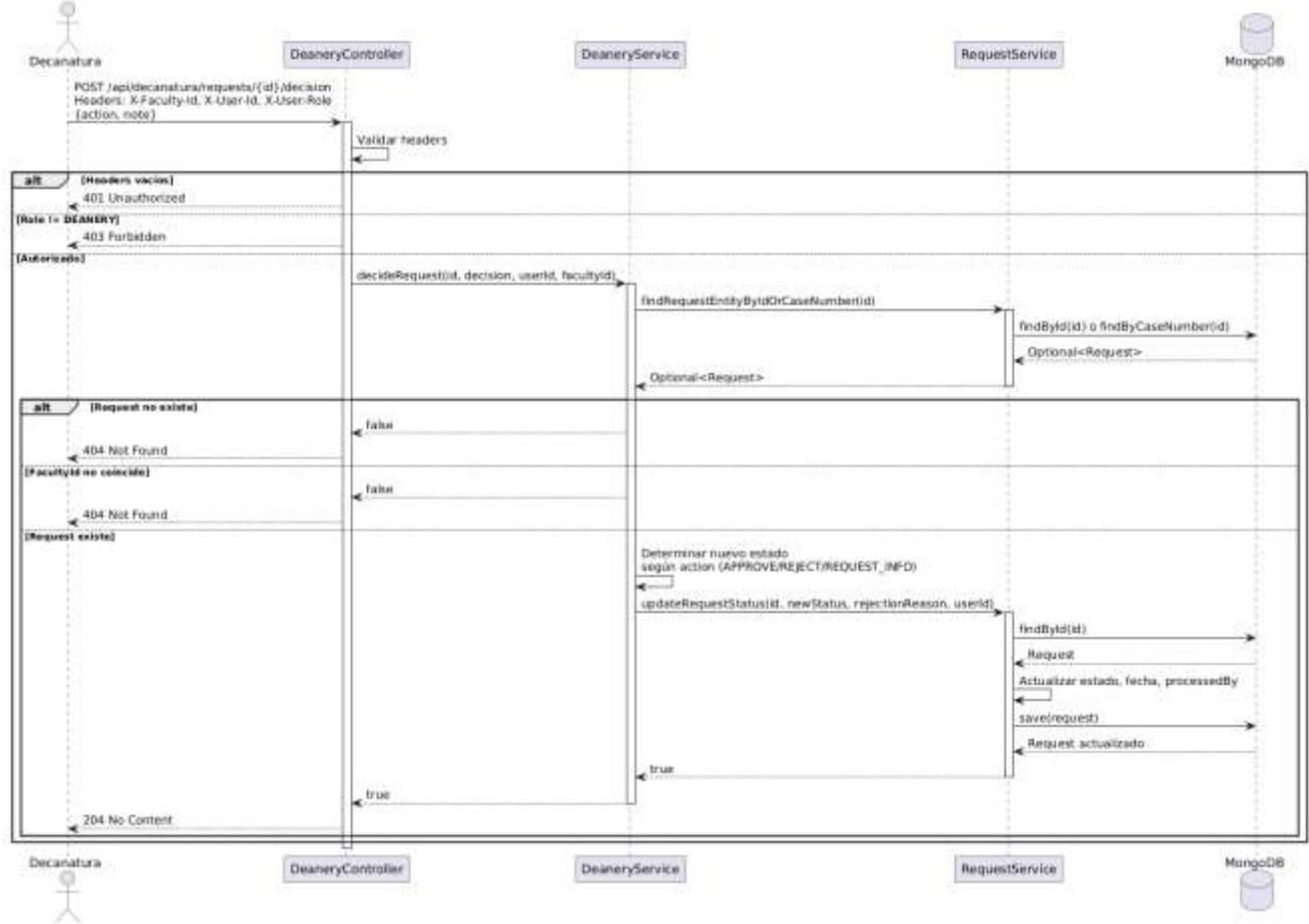
El cliente envía una petición DELETE con el requestId como parámetro de ruta, el sistema busca la solicitud con findChangeRequestOrThrow(), valida que esté en estado PENDING mediante

ensureRequestPending(), si la solicitud tiene un grupo destino busca el grupo con findByGroupld(), si el grupo tiene lista de espera remueve el studentld de la lista usando removelf(), persiste los cambios del grupo si hubo modificaciones, registra el evento "DELETED" en el historial con addHistoryEvent(), elimina la solicitud del repositorio con deleteByld() y retorna estado HTTP 204 sin contenido.

- **Manejo de Errores:** 404 Not Found: Si la solicitud con el UUID especificado no existe. Mensaje: "Solicitud no encontrada: {requestld}". 400 Bad Request: Si la solicitud no está en estado PENDING. Mensaje: "Sólo solicitudes en estado PENDING pueden ser procesadas o actualizadas". 400 Bad Request: Si el requestld no tiene formato UUID válido. 500 Internal Server Error: Si ocurre un error durante la eliminación.
- **Link Swagger:** DELETE /secretariat/requests/{requestld}

33. Funcionalidad: Decisión de Decanatura

- **descripción de la Funcionalidad:** La decanatura revisa las solicitudes de estudiantes de su facultad, consulta información y decide aprobar, rechazar o solicitar información adicional. Requiere autenticación con rol DEANERY.
- **Diagramas:**

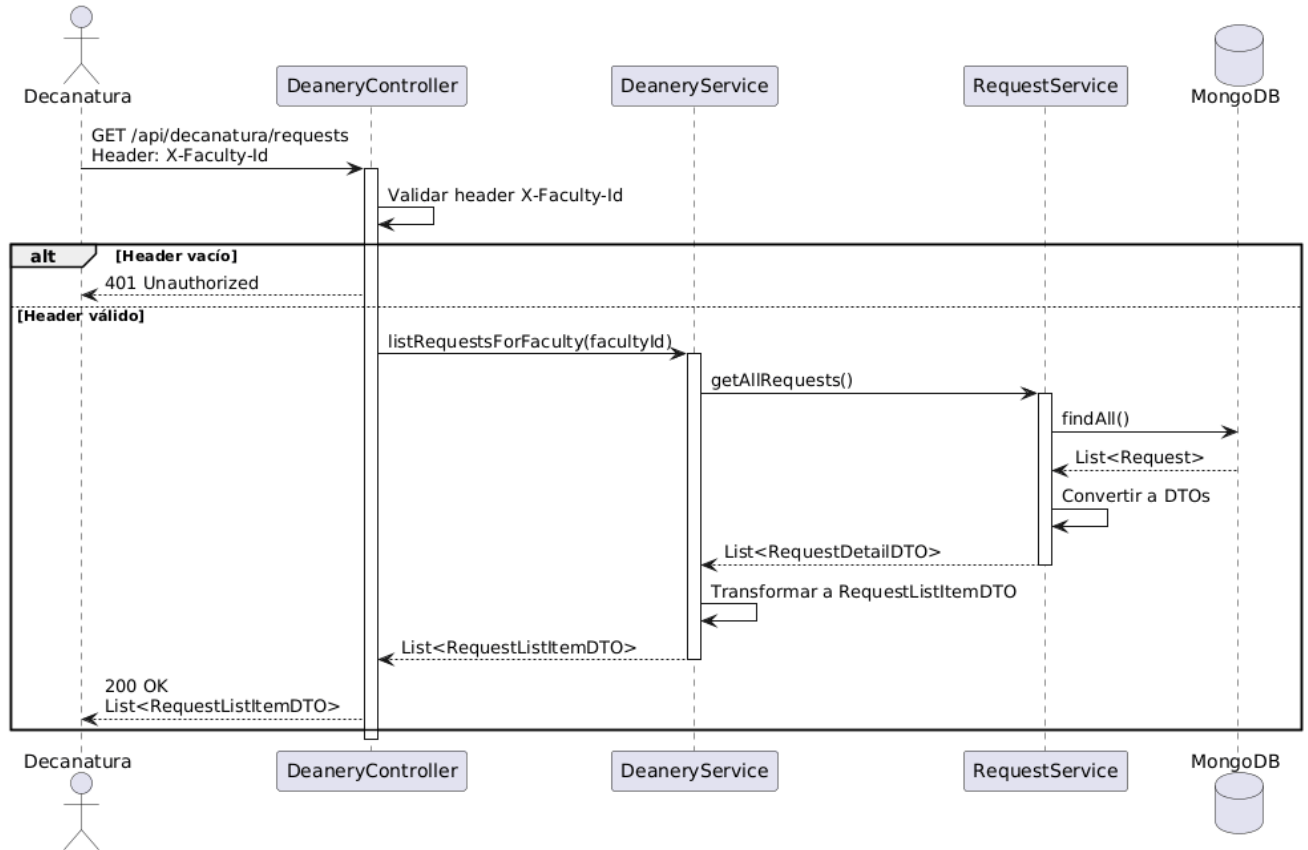


La decanatura envía una decisión (aprobar/rechazar/solicitar info) con headers de autenticación, el sistema valida los permisos, busca la solicitud, verifica que pertenezca a su facultad, actualiza el estado según la acción y registra quién procesó la solicitud.

- **Manejo de Errores: 401 Unauthorized:** Si faltan headers requeridos (X-Faculty-Id, X-User-Id, X-User-Role), **403 Forbidden:** Si el rol del usuario no es DEANERY, **404 Not Found:** Si la solicitud no existe o no pertenece a la facultad del usuario
- **Link Swagger:** POST /api/decanatura/requests/{id}/decision

34. Funcionalidad: Listar Solicitudes de Facultad

- **descripción de la Funcionalidad:** Lista todas las solicitudes relevantes para la decanatura de una facultad específica. Retorna información básica de cada solicitud (número de caso, estudiante, grupos, estado, fecha).
- **Diagramas:**

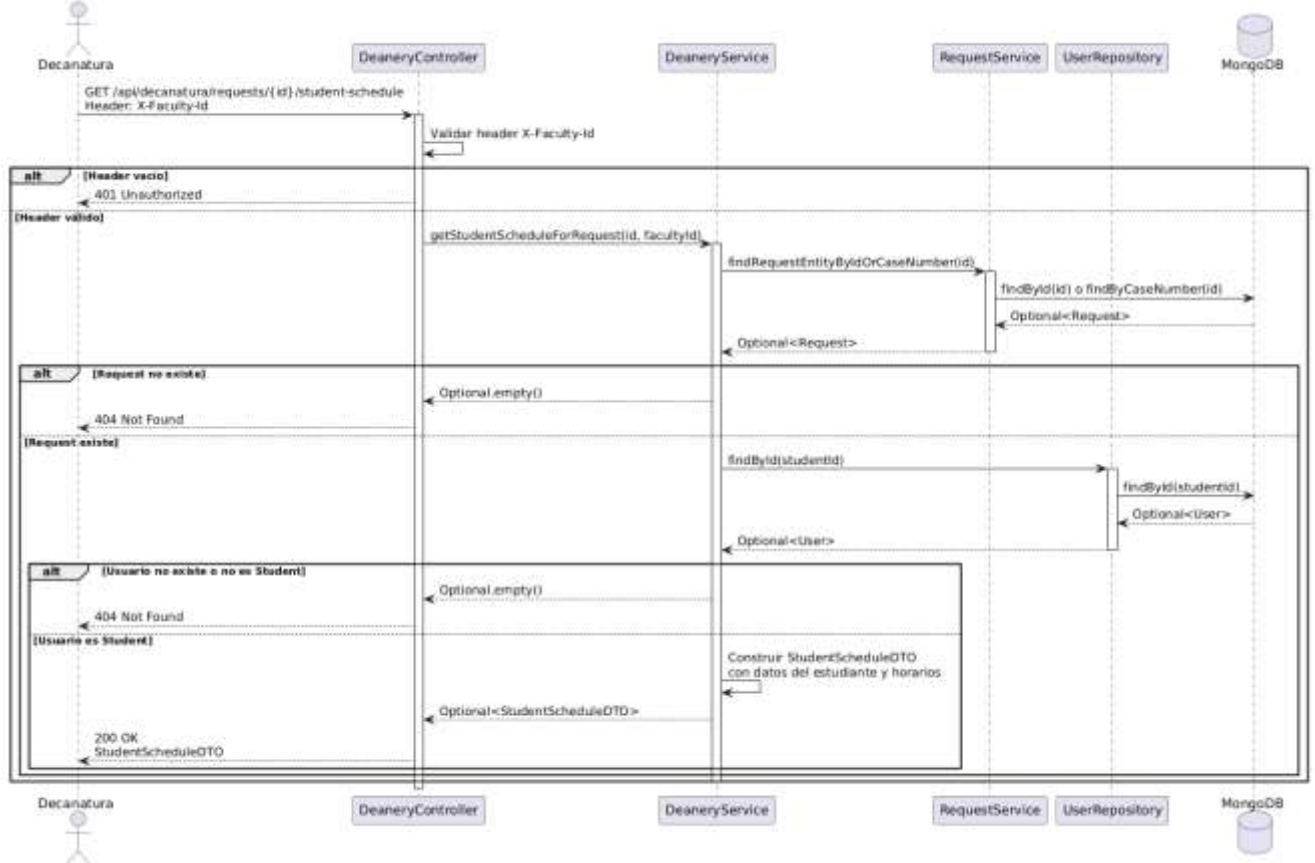


La decanatura solicita listar solicitudes enviando su facultyId en el header, el sistema obtiene todas las solicitudes de la base de datos (sin filtrar por facultad actualmente), las transforma a DTOs simplificados y retorna la lista.

- **Manejo de Errores: 401 Unauthorized:** Si falta el header X-Faculty-Id o está vacío
- **Link Swagger:** GET /api/decanatura/requests

35. Funcionalidad: Horario de Estudiante

- **descripción de la Funcionalidad:** Obtiene el horario completo del estudiante asociado a una solicitud específica, incluyendo información académica (carrera, semestre, semáforo curricular).
- **Diagramas:**

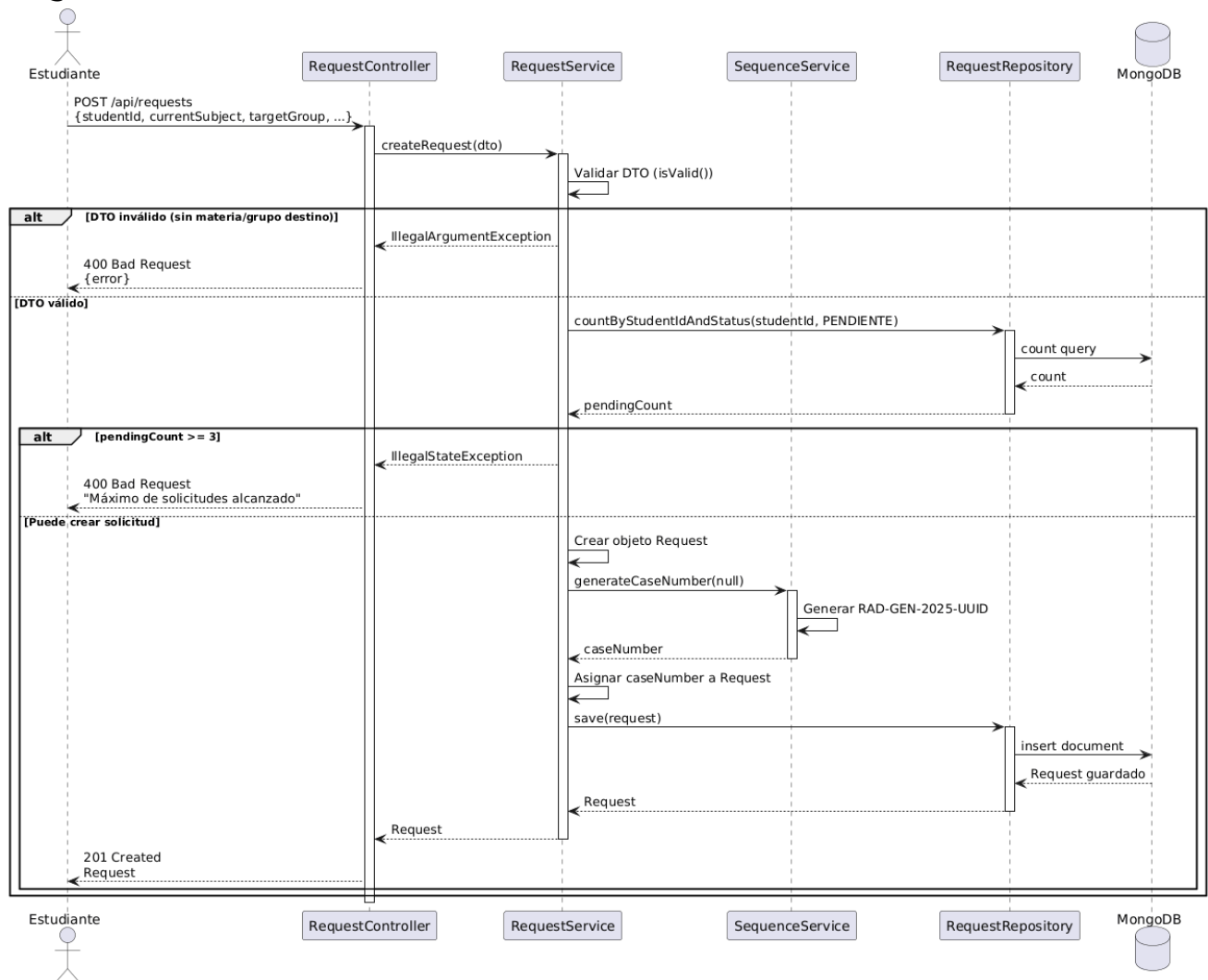


Se busca una solicitud por ID o número de caso, luego se obtiene el estudiante asociado, se extraen sus datos académicos (carrera, semestre, semáforo) y horarios, y se retorna todo en un DTO estructurado.

- **Manejo de Errores: 401 Unauthorized:** Si falta el header X-Faculty-Id o está vacío,
- **404 Not found:** Si la solicitud no existe, el estudiante no existe o el usuario no de tipo Student
- **Link Swagger:** GET /api/decanatura/requests/{id}/student-schedule

36. Funcionalidad: Creación de solicitudes de cambio de materia y grupo

- **descripción de la Funcionalidad:** Permite a un estudiante crear una nueva solicitud de cambio de materia/grupo. Valida que tenga al menos un destino (materia o grupo) y que no exceda el límite de 3 solicitudes pendientes simultáneas.
- **Diagramas:**

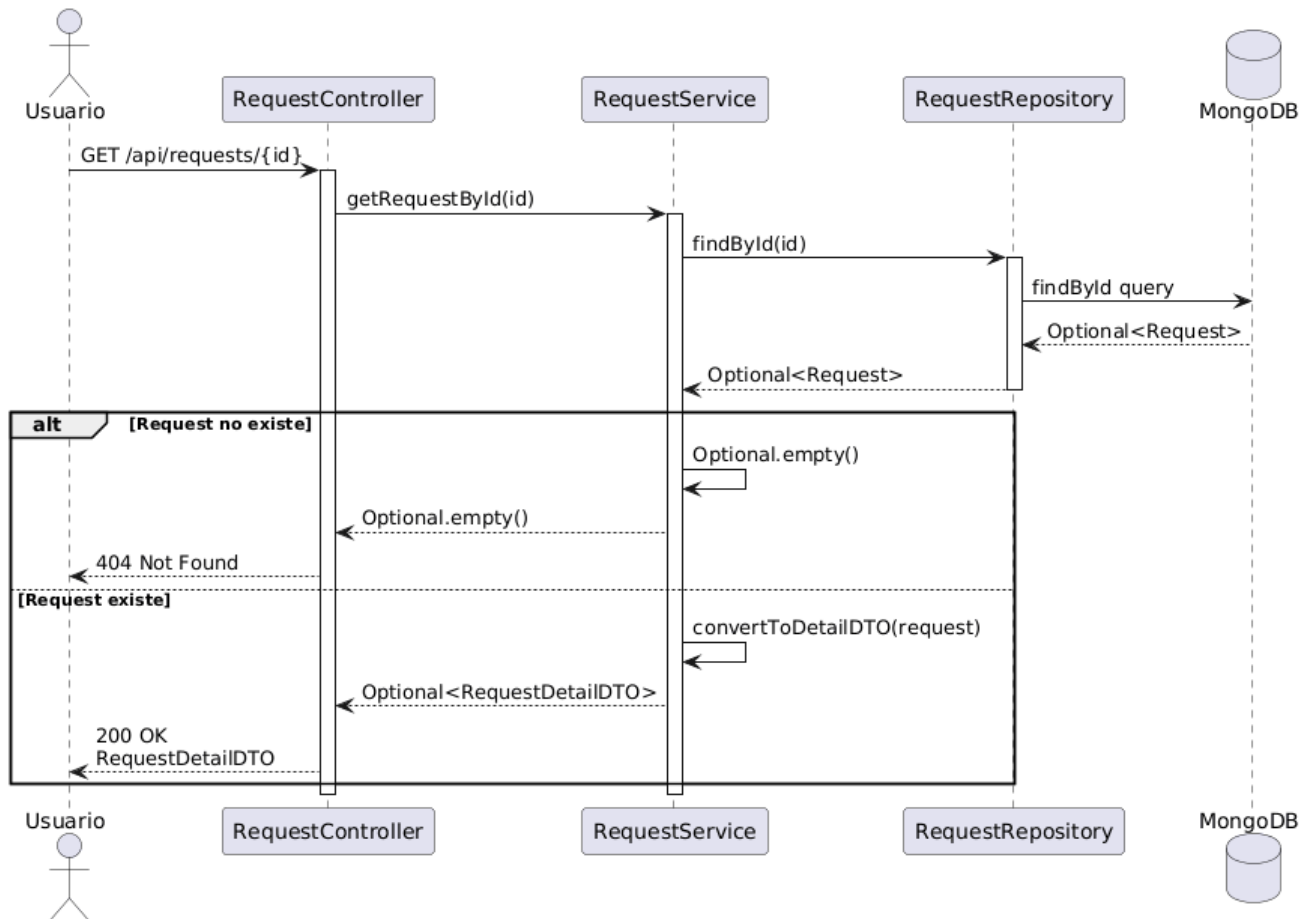


El estudiante envía datos para crear una solicitud, el sistema valida que tenga al menos un destino, verifica que no tenga 3 solicitudes pendientes, genera un número de caso único (formato RAD-XXX-2025-UUID), y guarda la solicitud en la base de datos.

- **Manejo de Errores: 400 Bad Request (IllegalArgumentException):** Si no se especifica materia o grupo destino, **400 Bad Request (IllegalStateException):** Si el estudiante ya tiene 3 solicitudes pendientes
- **Link Swagger:** POST /api/requests

37. Funcionalidad: Obtener Solicitud por ID

- **descripción de la Funcionalidad:** Recupera los detalles completos de una solicitud específica mediante su ID único.
- **Diagramas:**

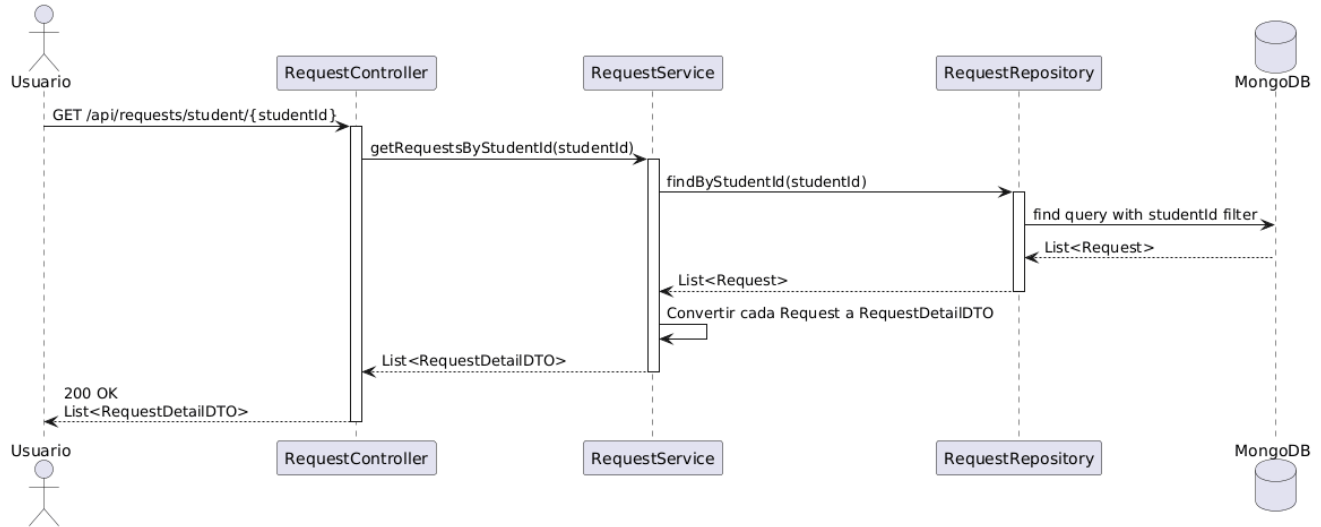


Se busca una solicitud por su ID en MongoDB, si existe se convierte a un DTO con todos los detalles y se retorna, si no existe retorna 404.

- **Manejo de Errores: 404 Not Found:** Si la solicitud con el ID especificado no existe
- **Link Swagger:** GET /api/requests/{id}

38. Funcionalidad: Solicitudes por Estudiante

- **descripción de la Funcionalidad:** Obtiene todas las solicitudes realizadas por un estudiante específico, ordenadas por fecha de creación.
- **Diagramas:**

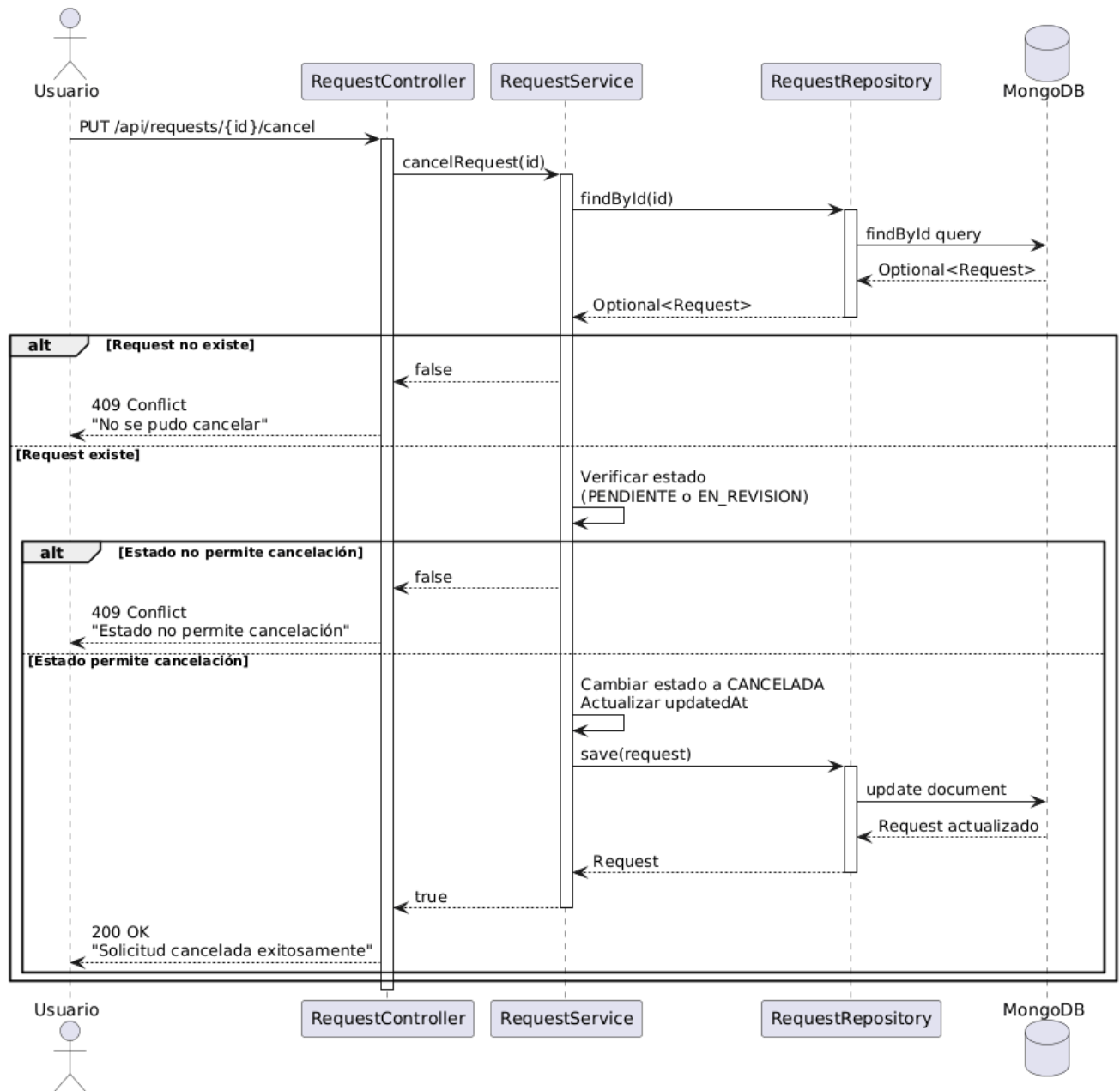


Se consultan todas las solicitudes que tengan ese studentId, se convierten a DTOs detallados y se retorna la lista completa.

- **Manejo de Errores:** Retorna lista vacía si no hay solicitudes (no es error)
- **Link Swagger:** GET /api/requests/student/{studentId}

39.Funcionalidad: Cancelar Solicitud

- **descripción de la Funcionalidad:** Permite cancelar una solicitud existente. Solo se pueden cancelar solicitudes en estado PENDIENTE o EN_REVISION.
- **Diagramas:**

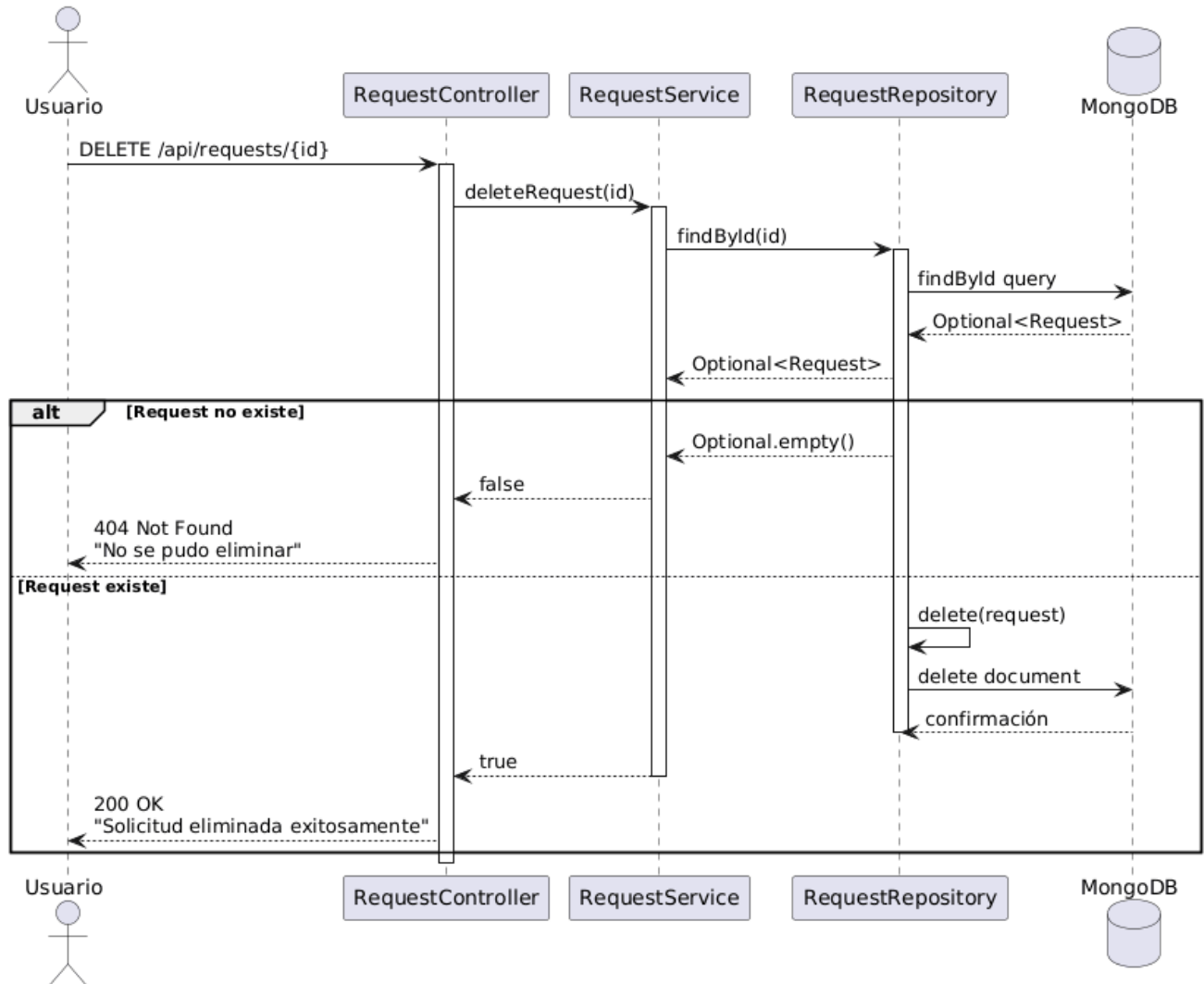


Se busca la solicitud por ID, se verifica que esté en estado PENDIENTE o EN_REVISION, si cumple se cambia el estado a CANCELADA y se actualiza la fecha, si no cumple retorna conflicto.

- **Manejo de Errores: 409 Conflict:** Si la solicitud no existe o no está en estado PENDIENTE/EN_REVISION
- **Link Swagger:** PUT /api/requests/{id}/cancel

40. Funcionalidad: Eliminar Solicitud

- **descripción de la Funcionalidad:** Elimina permanentemente una solicitud del sistema. Esta es una operación destructiva sin posibilidad de recuperación.
- **Diagramas:**

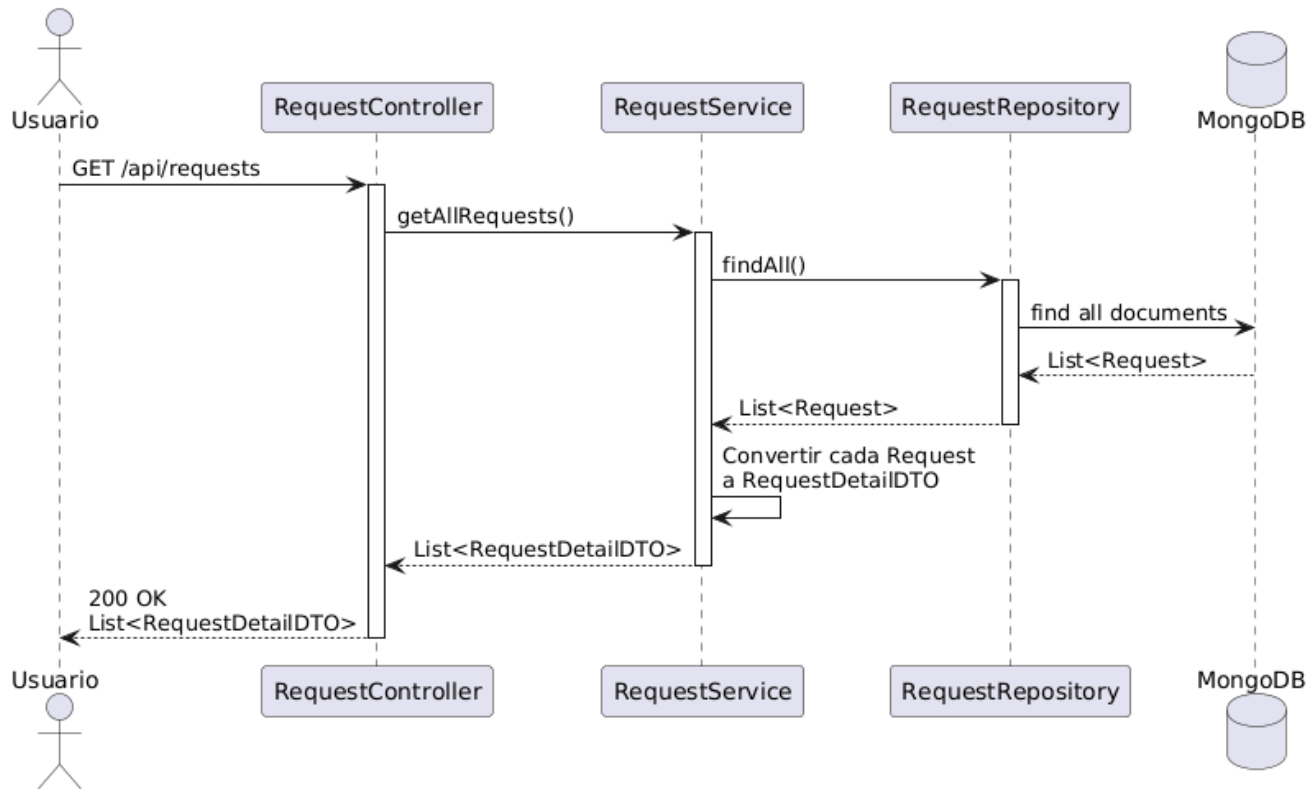


Se busca la solicitud por ID en la base de datos, si existe se elimina permanentemente del sistema, si no existe retorna 404.

- **Manejo de Errores: 404 Not Found:** Si la solicitud no existe
- **Link Swagger:** DELETE /api/requests/{id}

41.Funcionalidad: Listar Todas las Solicitudes

- **descripción de la Funcionalidad:** Obtiene todas las solicitudes del sistema sin filtros. Útil para administradores o vistas generales.
- **Diagramas:**

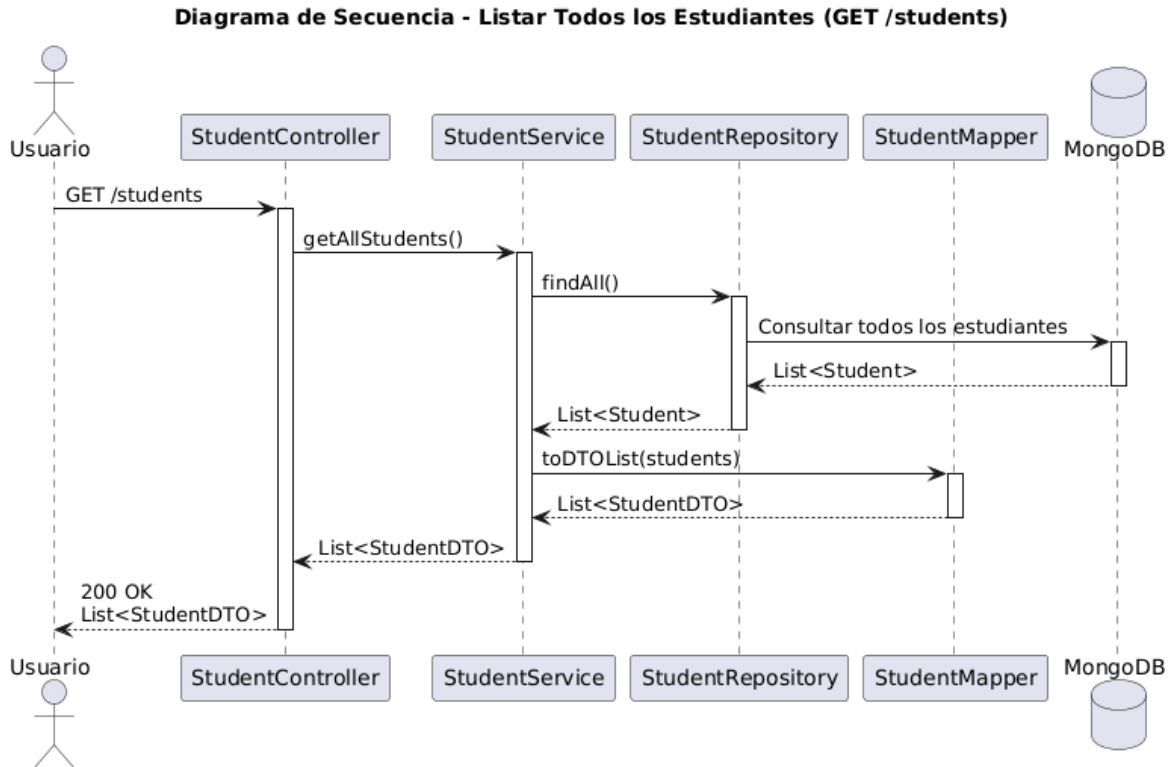


Se ejecuta un `findAll()` en MongoDB sin filtros ni paginación, se obtienen todas las solicitudes del sistema, se convierten a DTOs y se retornan.

- **Manejo de Errores:** No maneja errores específicos
- **Link Swagger:** `GET /api/requests`

42.Funcionalidad: Listar Estudiantes

- **descripción de la Funcionalidad:** Permite obtener una lista completa de todos los estudiantes registrados en el sistema. Este endpoint retorna la información básica de cada estudiante incluyendo ID, nombre, email, carrera, semestre, currículo, semáforo académico y sus listas de materias aprobadas e inscritas.
- **Diagramas:**

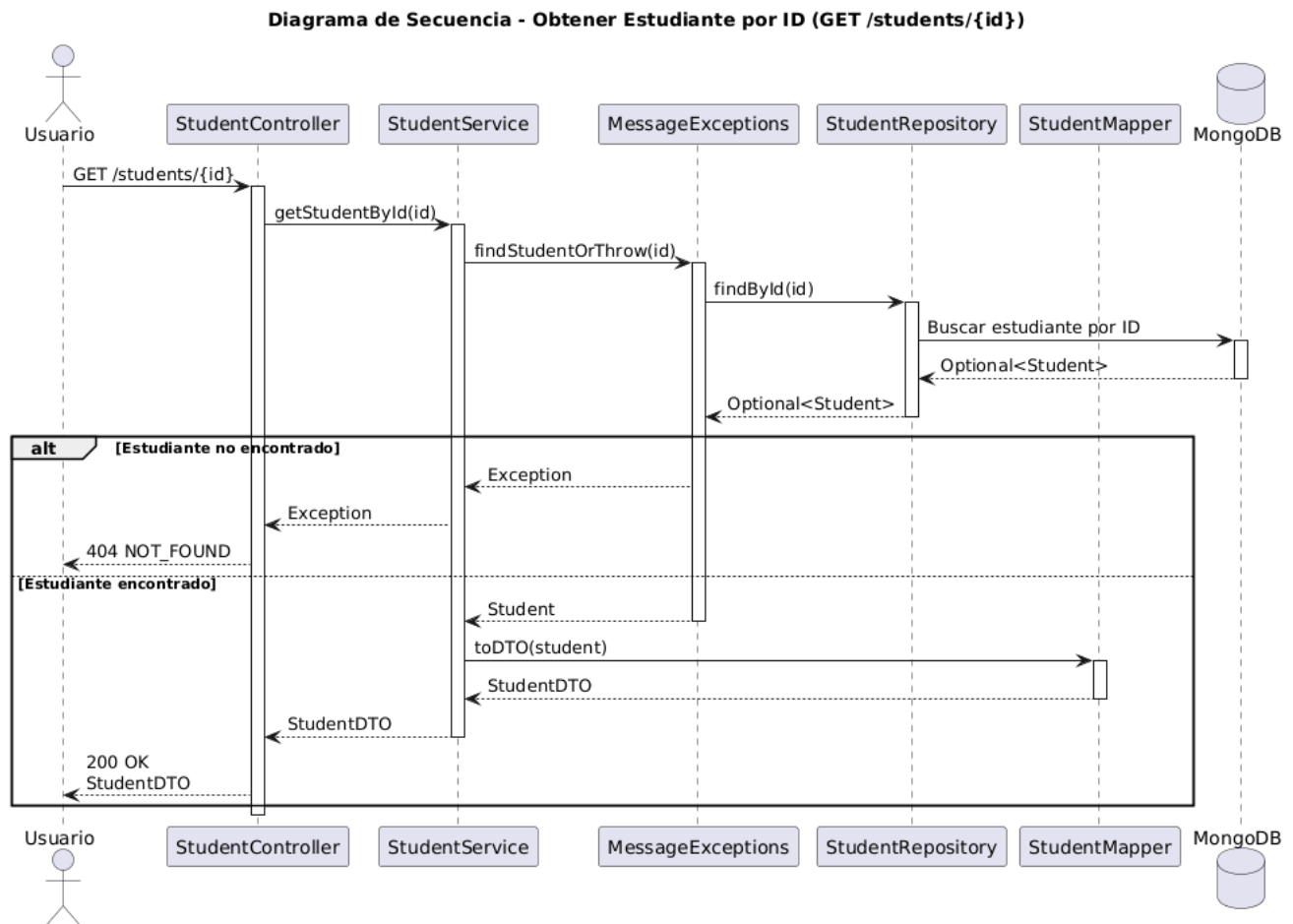


El cliente realiza una petición GET sin parámetros, el sistema consulta el repositorio de estudiantes mediante findAll(), mapea cada entidad Student a StudentDTO usando el mapper con toDTOList() y retorna la lista completa en formato JSON.

- **Manejo de Errores:** 500 Internal Server Error: Si ocurre un error inesperado en el servidor durante la consulta. Retorna una lista vacía [] si no hay estudiantes registrados (no es un error).
- **Link Swagger:** GET /estudiantes

43.Funcionalidad: Obtener Estudiante por ID

- **descripción de la Funcionalidad:** Recupera la información detallada y completa de un estudiante específico mediante su identificador único numérico. Retorna todos los datos del estudiante incluyendo información académica, materias aprobadas, materias inscritas, horario y solicitudes asociadas.
- **Diagramas:**

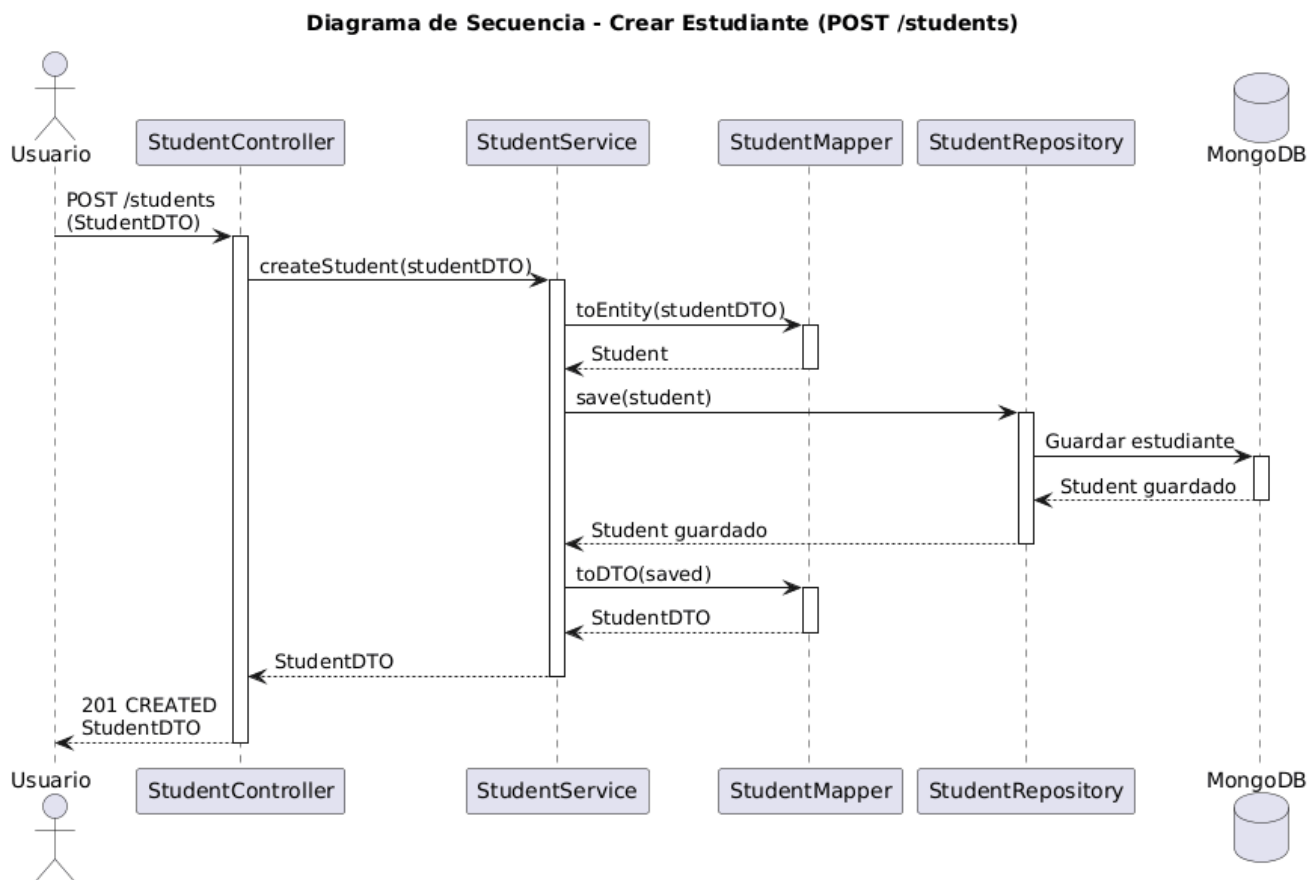


El cliente envía una petición GET con el ID del estudiante como parámetro de ruta, el sistema invoca `findStudentOrThrow(id)` que busca en el repositorio mediante `findById()`, valida la existencia del estudiante lanzando excepción 404 si no existe, mapea la entidad `Student` a `StudentDTO` con toda su información y retorna el DTO completo.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la operación.
- **Link Swagger:** GET /estudiantes/{id}

44.Funcionalidad: Crear Estudiante

- **descripción de la Funcionalidad:** Realiza una actualización completa (PUT) de todos los campos de un estudiante existente. Requiere enviar la información completa y validada del estudiante (nombre, email, carrera, semestre, currículum y semáforo académico). Todos los campos obligatorios deben estar presentes en el request body.
- **Diagramas:**



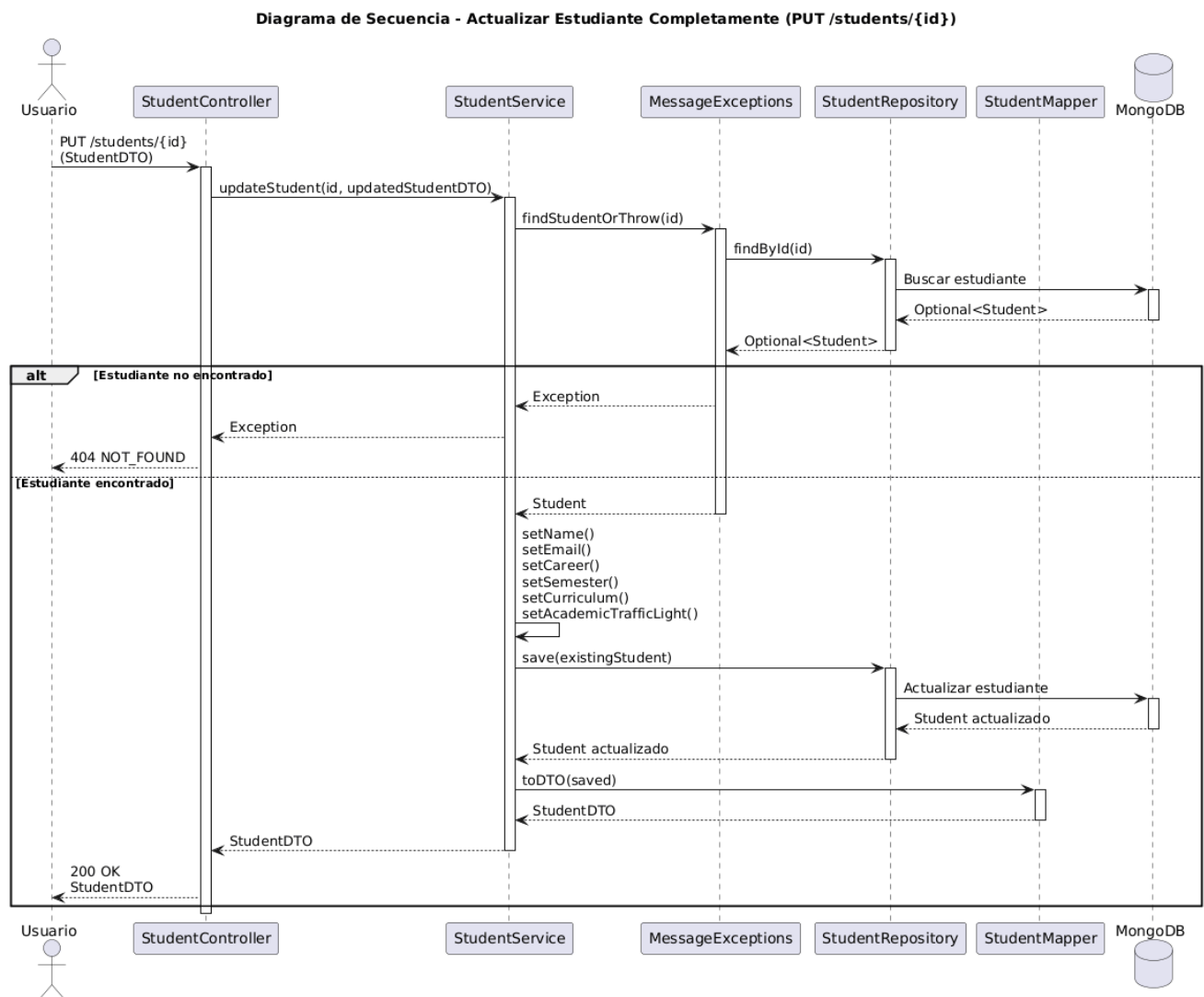
El cliente envía una petición PUT con el ID en la ruta y el cuerpo con todos los datos validados mediante @Valid, el sistema busca el estudiante existente con findStudentOrThrow(id), valida todos los campos del DTO entrante, actualiza todos los campos del estudiante (name, email, career, semester, curriculum, academicTrafficLight),

persiste los cambios con `save()` y retorna el `StudentDTO` actualizado completo.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la operación.
- **Link Swagger:** POST /estudiantes

45.Funcionalidad: Actualizar Estudiante

- **descripción de la Funcionalidad:** Realiza una actualización completa (PUT) de todos los campos de un estudiante existente. Requiere enviar la información completa y validada del estudiante (nombre, email, carrera, semestre, currículo y semáforo académico). Todos los campos obligatorios deben estar presentes en el request body.
- **Diagramas:**

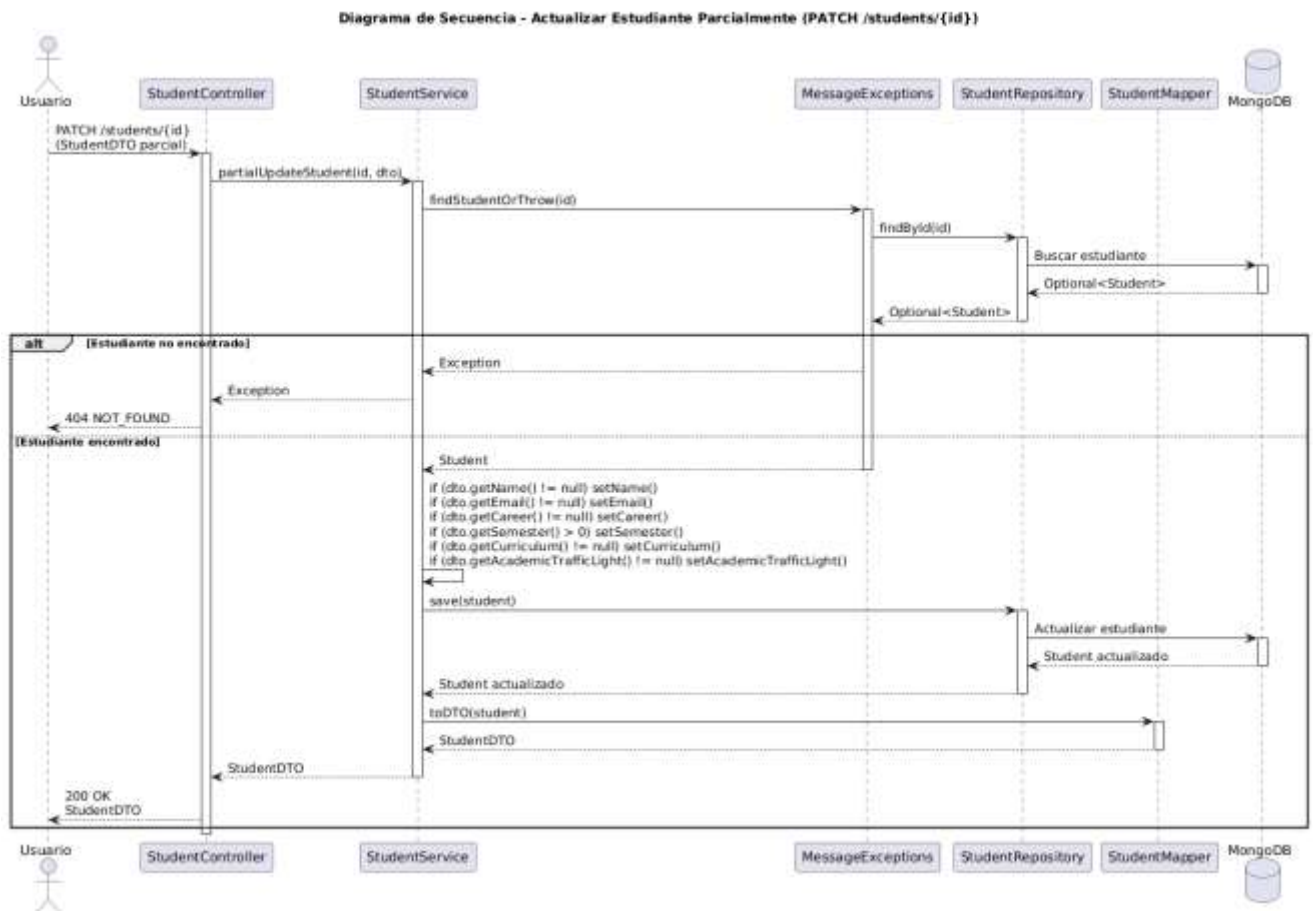


Se busca el estudiante por ID, si existe se actualizan todos sus campos con los nuevos valores del DTO incluyendo contraseña en texto plano, se guarda y se retorna el estudiante actualizado.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si faltan campos obligatorios (nombre, email, carrera, semestre, currículo). Mensaje: "Validation failed" con detalles. 400 Bad Request: Si el formato del email es inválido. 400 Bad Request: Si los valores de enums no son válidos. 500 Internal Server Error: Si ocurre un error durante la actualización.
- **Link Swagger:** PUT /estudiantes/{id}

46.Funcionalidad: Actualizar Estudiante Parcialmente

- **descripción de la Funcionalidad:** Permite actualizar uno o varios campos específicos de un estudiante (PATCH) sin necesidad de enviar toda la información. Solo los campos presentes y no nulos en el request body serán actualizados. Los campos ausentes o null se ignoran y mantienen su valor actual. No requiere validación @Valid ya que acepta datos parciales.
- **Diagramas:**



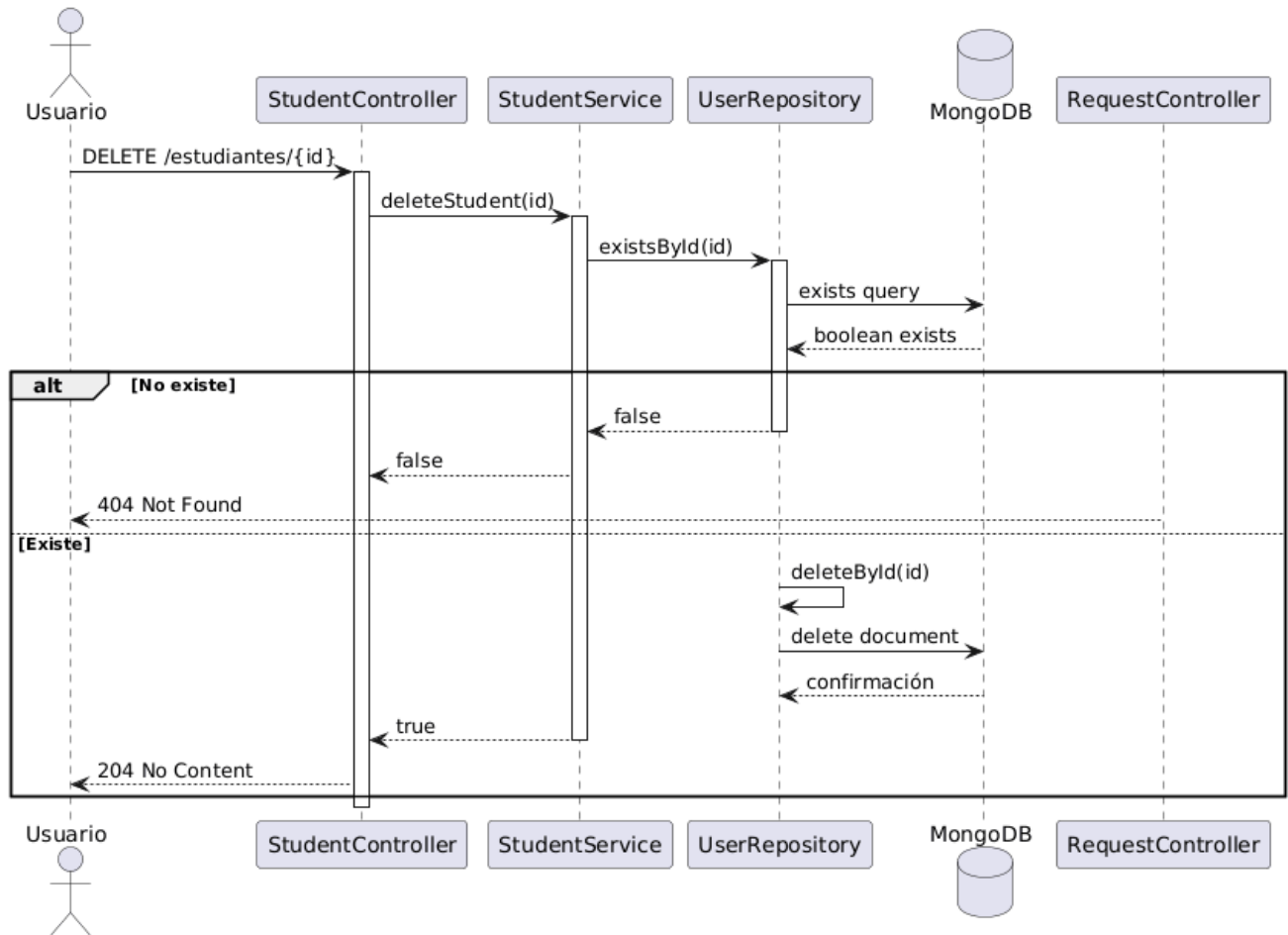
El cliente envía una petición PATCH con el ID en la ruta y el cuerpo con solo los campos a actualizar, el sistema busca el

estudiante con `findStudentOrThrow(id)`, verifica campo por campo si el DTO contiene valores no nulos (`name`, `email`, `career`, `semester > 0`, `curriculum`, `academicTrafficLight`), actualiza solo los campos presentes manteniendo los demás sin cambios, persiste con `save()` y retorna el estudiante completo actualizado.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el formato del email proporcionado es inválido. 400 Bad Request: Si los valores de enums proporcionados no son válidos. 500 Internal Server Error: Si ocurre un error durante la actualización parcial.
- **Link Swagger:** `PATCH /students/{id}`

47.Funcionalidad: Eliminar Estudiante

- **descripción de la Funcionalidad:** Elimina permanentemente un estudiante del sistema mediante su ID, incluyendo toda su información académica, horarios y solicitudes asociadas. Antes de eliminar, valida la existencia del estudiante. Retorna código de estado 204 No Content al completarse exitosamente la eliminación sin cuerpo en la respuesta.
- **Diagramas:**

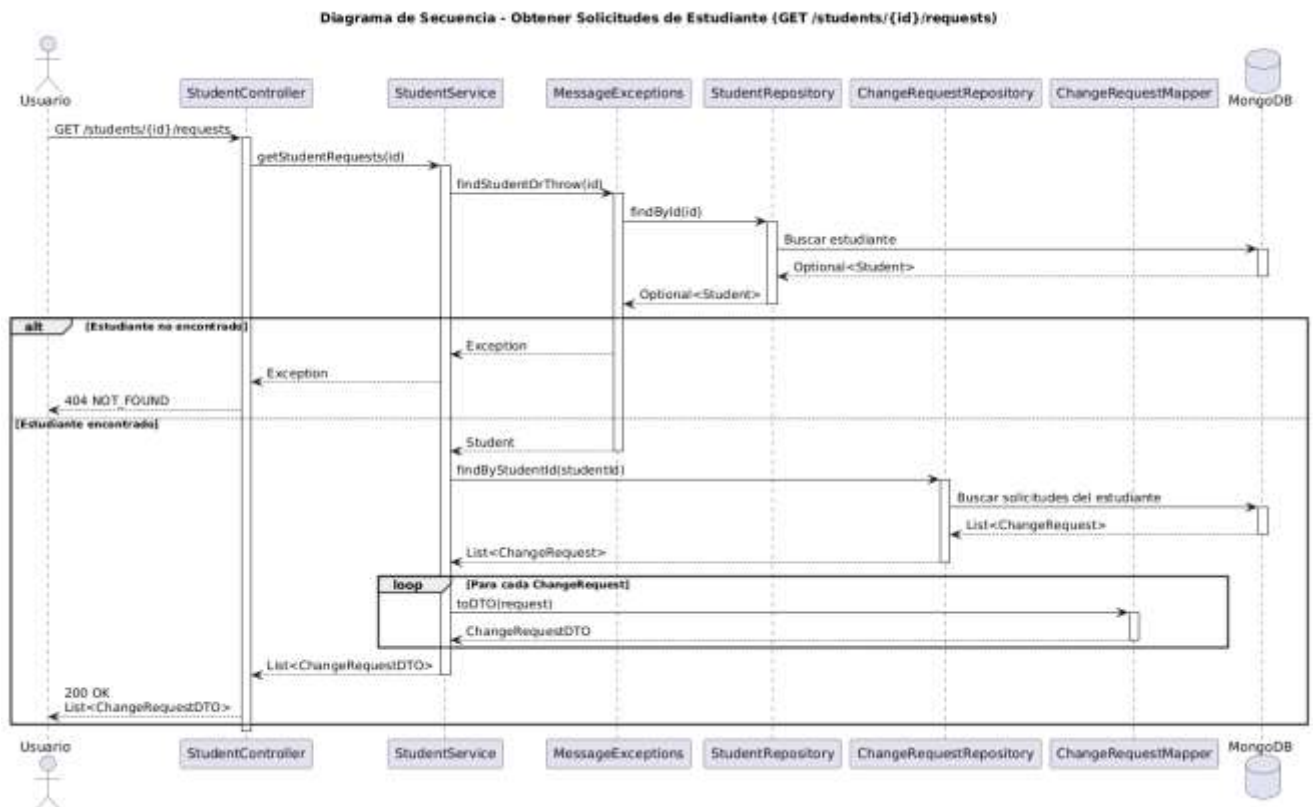


El cliente envía una petición DELETE con el ID del estudiante en la ruta, el sistema busca y valida la existencia del estudiante mediante `findStudentOrThrow(id)`, si existe procede a eliminarlo del repositorio con `delete(student)` eliminando en cascada sus relaciones, confirma la eliminación y retorna estado HTTP 204 sin contenido en el body.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe en el sistema. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la eliminación (por ejemplo, restricciones de integridad referencial si hay datos dependientes).
- **Link Swagger:** DELETE /estudiantes/{id}

48.Funcionalidad: Obtener Todas las Solicitudes del Estudiante

- **descripción de la Funcionalidad:** Retorna el listado completo de todas las solicitudes de cambio de materia o grupo realizadas por un estudiante específico, independientemente de su estado (PENDING, APPROVED, REJECTED, INFO_REQUESTED, APPROVED_BY_DEANERY). Incluye información detallada de cada solicitud como ID, tipo de cambio, materia origen y destino, grupo origen y destino, estado actual, fechas, observaciones y si es excepcional.
- **Diagramas:**



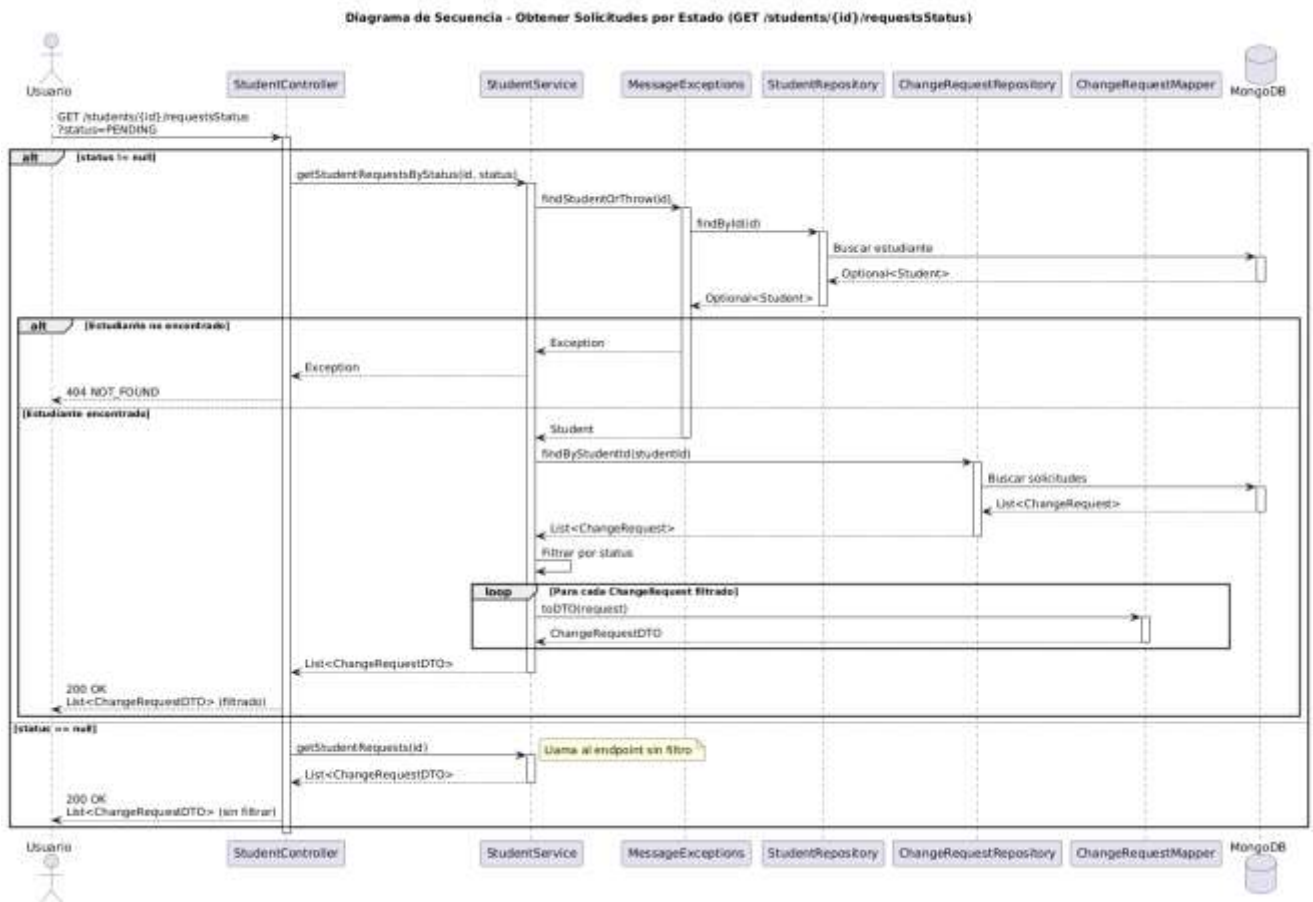
El cliente envía una petición GET con el ID del estudiante como parámetro de ruta, el sistema valida la existencia del estudiante mediante `findStudentOrThrow()`, consulta el repositorio de `ChangeRequest` filtrando por `studentId` mediante

findByStudentId(), mapea cada entidad ChangeRequest a ChangeRequestDTO usando el mapper en un stream y retorna la lista completa de solicitudes en formato JSON.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error durante la consulta de solicitudes. Retorna una lista vacía [] si el estudiante no tiene solicitudes registradas (no es un error).
- **Link Swagger:** GET /students/{id}/requests

49. Funcionalidad: Obtener Solicitudes Filtradas por Estado

- **descripción de la Funcionalidad:** Permite obtener las solicitudes de cambio de un estudiante específico filtradas opcionalmente por su estado (PENDING, APPROVED, REJECTED, INFO_REQUESTED, APPROVED_BY_DEANERY). Si se proporciona el parámetro query status, retorna solo las solicitudes en ese estado; si no se proporciona o es null, funciona igual que el endpoint /students/{id}/requests retornando todas las solicitudes del estudiante sin filtro.
- **Diagramas:**



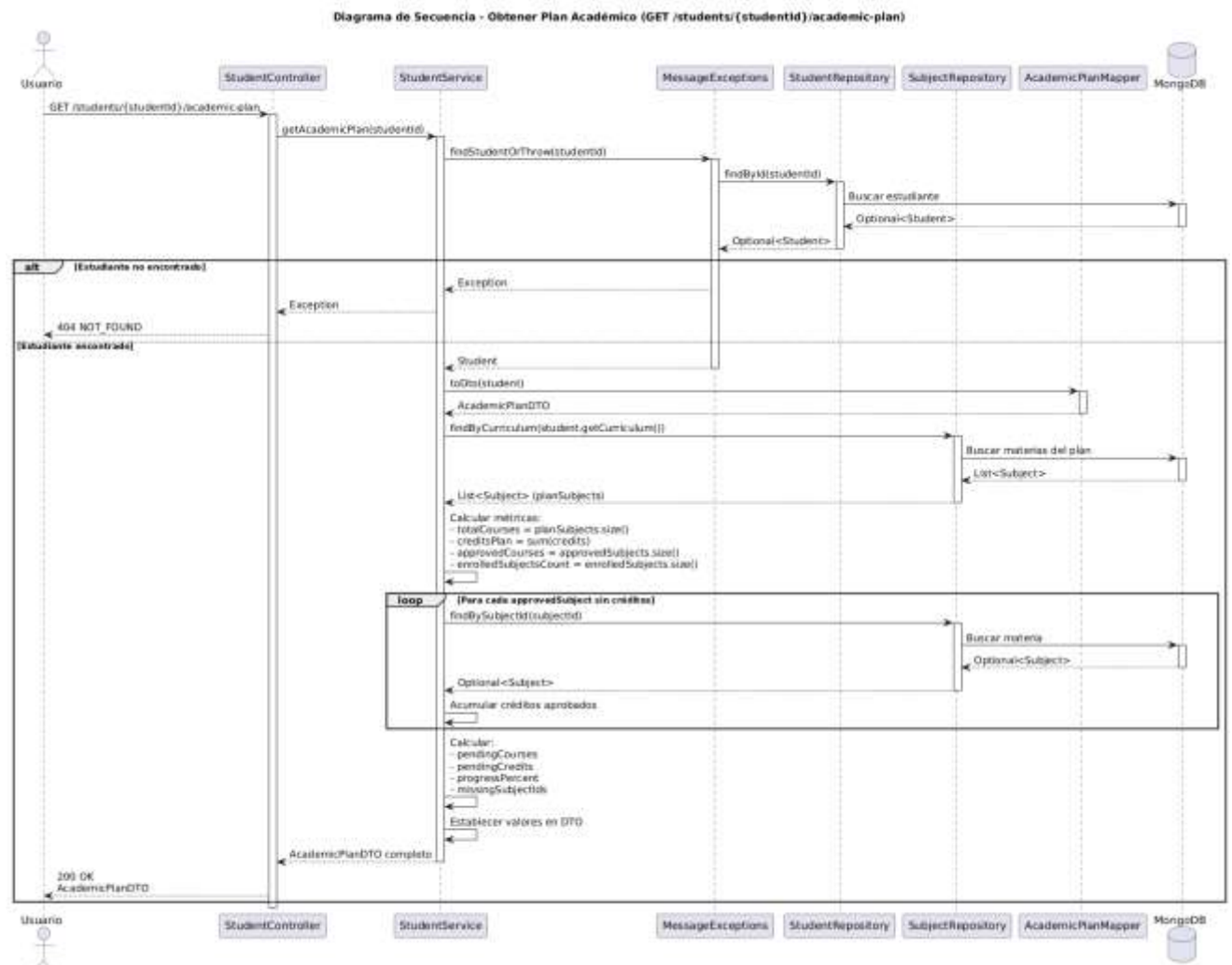
El cliente envía una petición GET con el ID del estudiante en la ruta y opcionalmente el parámetro query status, el sistema valida la existencia del estudiante mediante findStudentOrThrow(), consulta todas las solicitudes del estudiante con findByStudentId(), verifica si el parámetro status es diferente de null mediante condicional, si está presente filtra las solicitudes en memoria usando stream().filter() comparando el estado de cada request con el RequestStatus proporcionado, si es null retorna todas sin filtrar, mapea las solicitudes resultantes a

ChangeRequestDTO y retorna la lista filtrada o completa según corresponda.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 400 Bad Request: Si el valor del parámetro status no corresponde a un valor válido del enum RequestStatus (valores válidos: PENDING, APPROVED, REJECTED, INFO_REQUESTED, APPROVED_BY_DEANERY). 500 Internal Server Error: Si ocurre un error durante la consulta o filtrado de solicitudes. Retorna una lista vacía [] si no hay solicitudes que coincidan con el filtro (no es un error).
- **Link Swagger:** GET /students/{id}/requestsStatus?status={RequestStatus}

50.Funcionalidad: Obtener Plan Académico del Estudiante

- **descripción de la Funcionalidad:** Genera y retorna un resumen completo y consolidado del plan académico del estudiante, incluyendo el total de materias del plan de estudios según su currículo, créditos totales del plan, materias aprobadas y sus créditos acumulados, materias inscritas actualmente con sus IDs, materias y créditos pendientes, porcentaje de progreso académico calculado sobre materias inscritas y listado detallado de IDs de materias faltantes. Este endpoint consolida información de múltiples fuentes (Student, Subject) para proporcionar una vista integral y calculada del avance académico del estudiante.
- **Diagramas:**



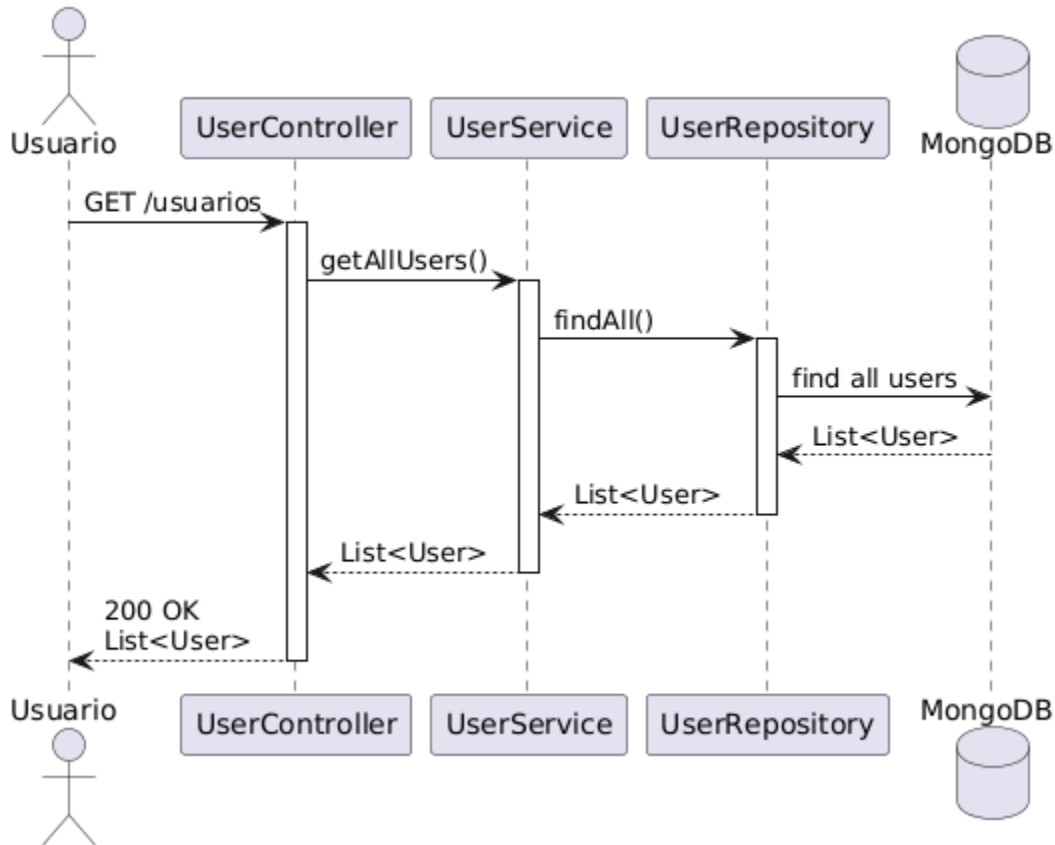
El cliente envía una petición GET con el studentId como parámetro de ruta, el sistema valida la existencia del estudiante mediante findStudentOrThrow(), consulta todas las materias del currículo del estudiante en el repositorio de Subject usando findByCurriculum(), calcula el total de materias y suma los

créditos del plan completo, obtiene las listas de materias aprobadas (approvedSubjects) e inscritas (enrolledSubjects) del estudiante, calcula los créditos aprobados desde el campo approvedCredits o recalculando si es null consultando cada materia, determina las materias pendientes restando aprobadas del total del plan, calcula los créditos pendientes restando aprobados del total del plan, calcula el porcentaje de progreso usando la fórmula $(\text{materias inscritas} / \text{total materias del plan}) \times 100$ redondeado a 2 decimales, genera la lista de materias faltantes excluyendo las que están en aprobadas o inscritas usando streams y Set, mapea toda la información calculada a AcademicPlanDTO con todos los campos poblados y retorna el resumen completo.

- **Manejo de Errores:** 404 Not Found: Si el estudiante con el ID especificado no existe. Mensaje: "Estudiante no encontrado con id: {studentId}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error durante el cálculo del plan académico, la consulta de materias del currículo o el mapeo de datos.
- **Link Swagger:** GET /students/{studentId}/academic-plan

51.Funcionalidad: Listar Usuarios

- **descripción de la Funcionalidad:** Obtiene la lista completa de todos los usuarios del sistema sin importar su rol (estudiantes, secretaría, decanatura).
- **Diagramas:**

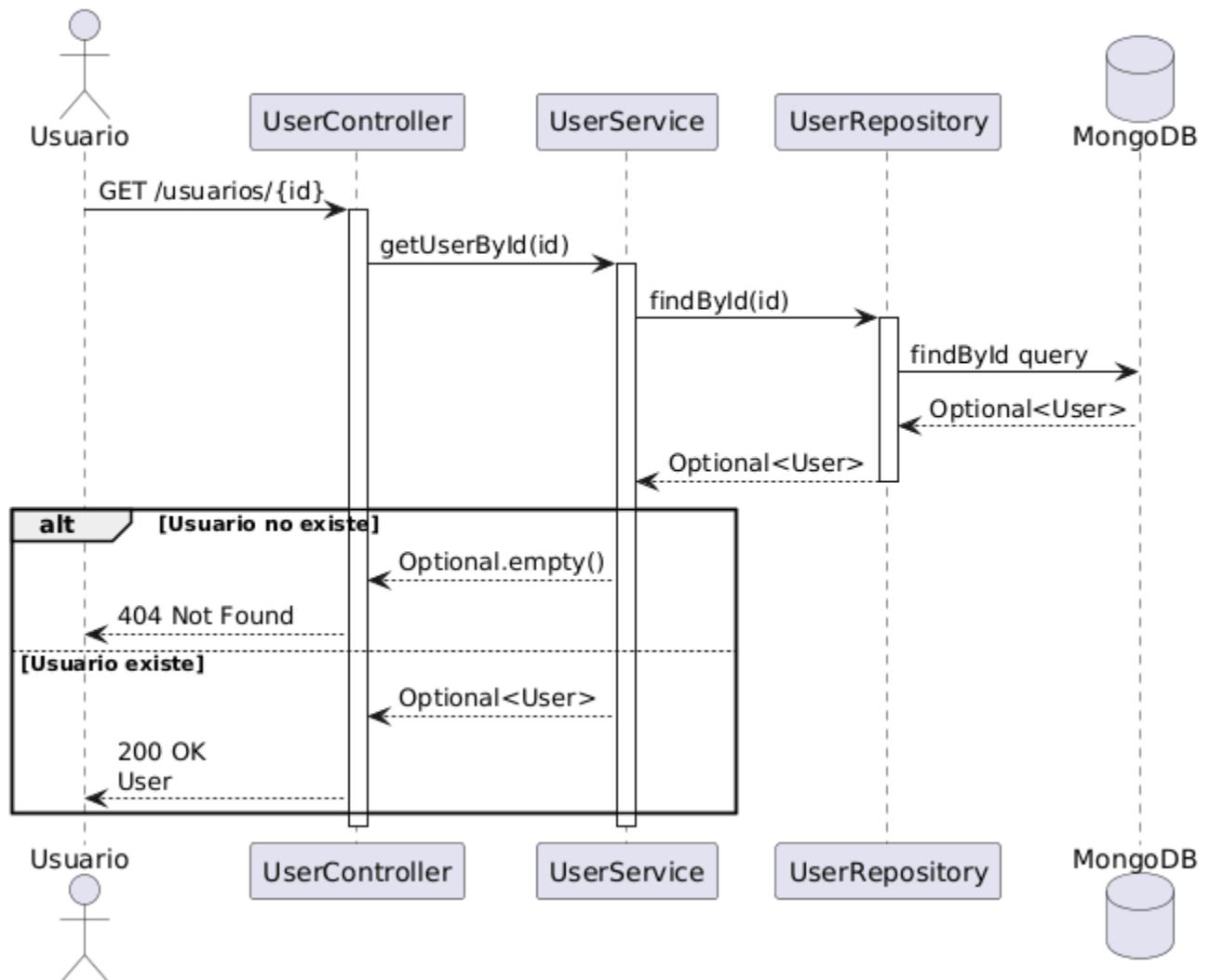


Se ejecuta findAll() en el repositorio de usuarios sin ningún filtro, retorna absolutamente todos los usuarios del sistema (estudiantes, secretaría, decanatura) con contraseñas incluidas.

- **Manejo de Errores:** No maneja errores específicos
- **Link Swagger:** GET /usuarios

52.Funcionalidad: Obtener Usuario por ID

- **descripción de la Funcionalidad:** Recupera la información completa de un usuario específico mediante su ID, independientemente de su rol.
- **Diagramas:**

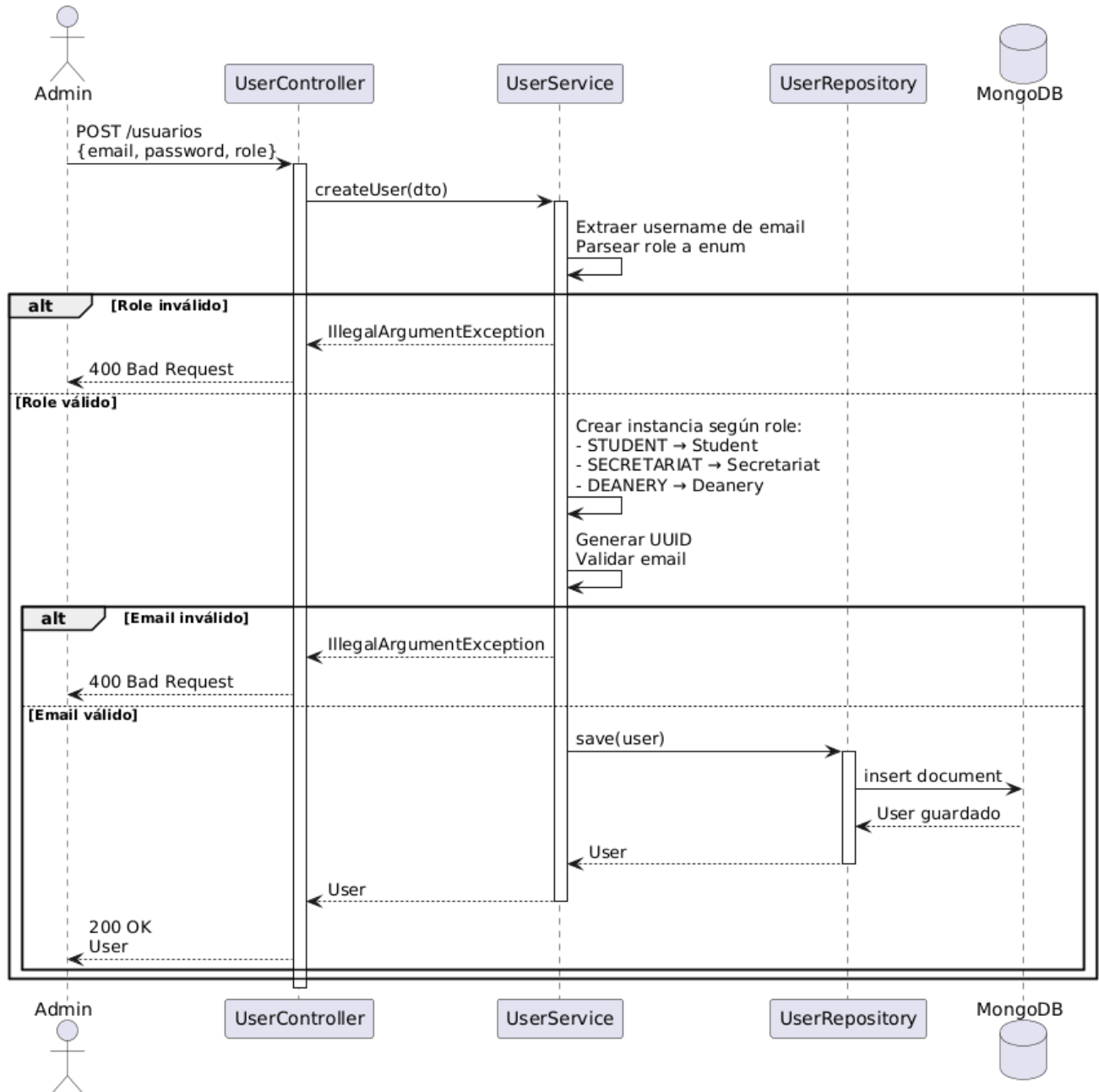


Se busca cualquier usuario por su ID sin importar el rol, si existe se retorna con toda su información incluyendo contraseña, si no existe retorna 404.

- **Manejo de Errores: 404 Not Found:** Si el usuario no existe
- **Link Swagger:** GET /usuarios/{id}

53.Funcionalidad: POST /usuarios - Crear Usuario

- **descripción de la Funcionalidad:** Crea un nuevo usuario en el sistema. Dependiendo del rol especificado en el DTO, crea una instancia de Student, Secretariat o Deanery.
- **Diagramas:**

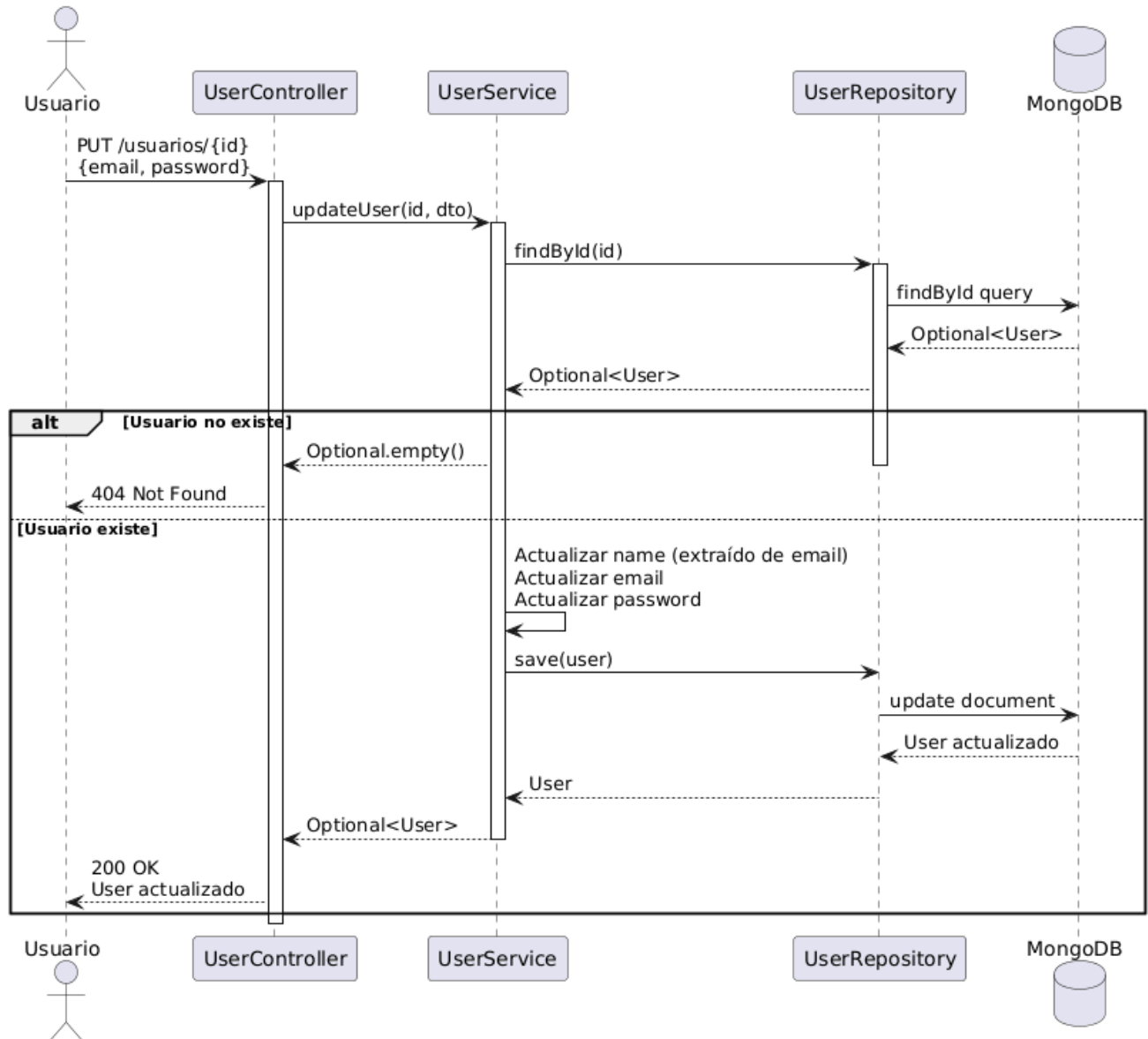


Se recibe email, password y role, se extrae el username del email, se parsea el role a enum, según el role se crea una instancia de Student/Secretariat/Deanery con UUID, se valida el email y se guarda con contraseña en texto plano.

- **Manejo de Errores: 400 Bad Request:** Si el rol es inválido o el email no es válido.
- **Link Swagger:** POST /usuarios

54.Funcionalidad: Actualizar Usuario

- **descripción de la Funcionalidad:** Actualiza la información de un usuario existente (email, contraseña).
- **Diagramas:**

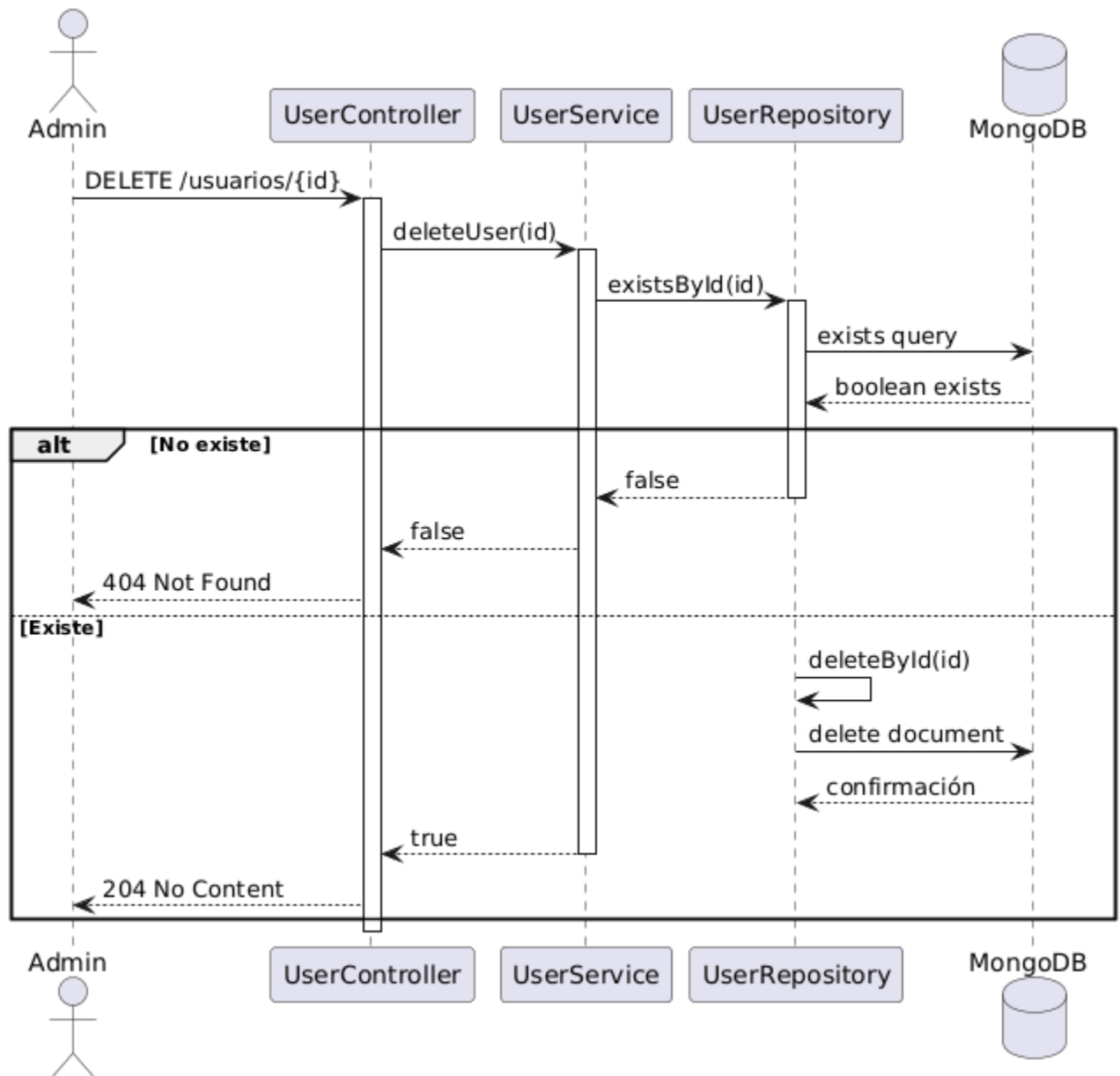


Se busca el usuario por ID, si existe se actualiza su name (extraído del email), email y password sin encriptar, se guarda y retorna el usuario actualizado.

- **Manejo de Errores: 404 Not Found:** Si el usuario no existe
- **Link Swagger:** PUT /usuarios/{id}

55.Funcionalidad: Eliminar Usuario

- **descripción de la Funcionalidad:** Elimina permanentemente un usuario del sistema.
- **Diagramas:**

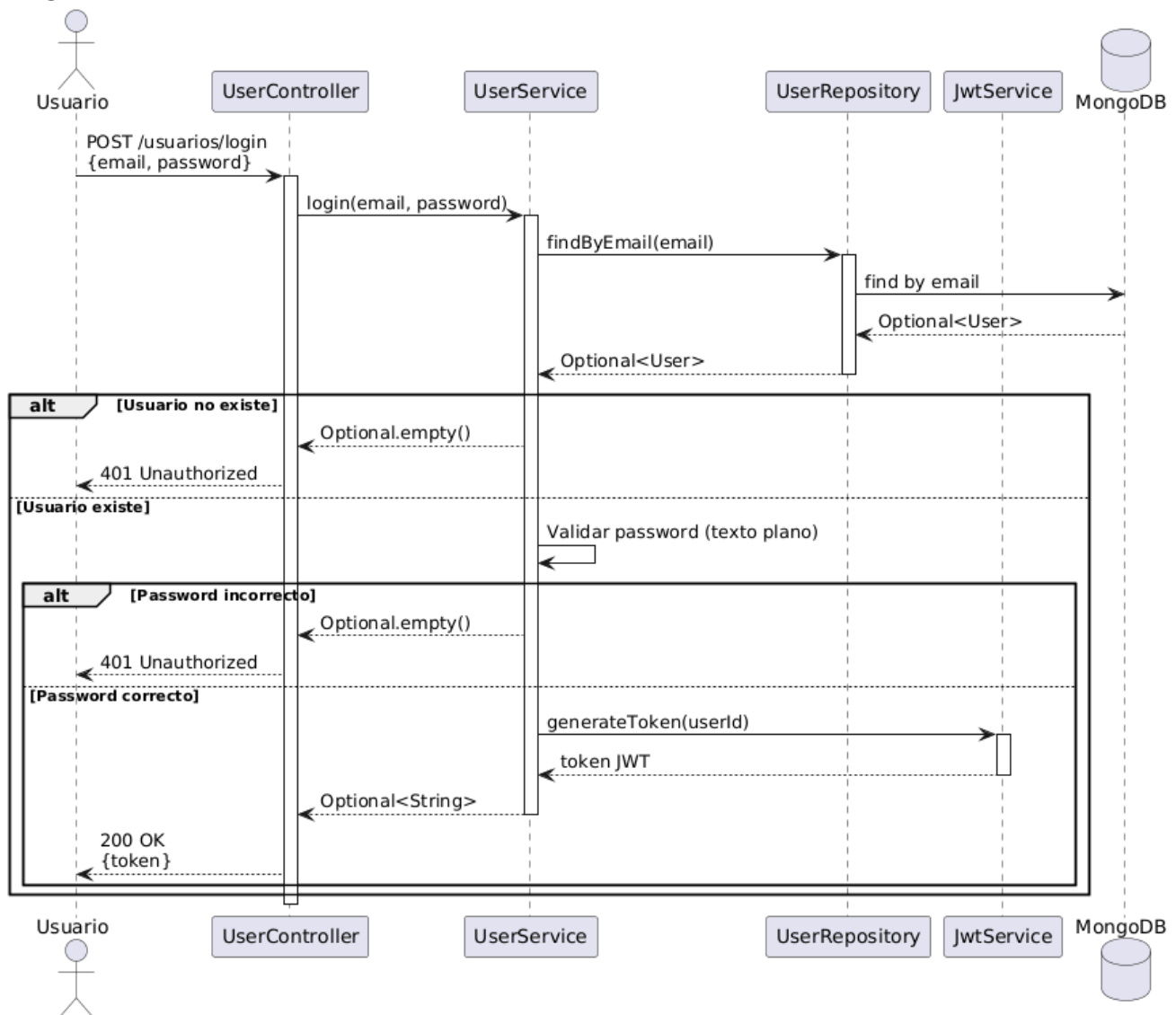


Se verifica existencia del usuario por ID, si existe se ejecuta deleteById() eliminándolo permanentemente, si no existe retorna 404.

- **Manejo de Errores: 404 Not Found:** Si el usuario no existe.
- **Link Swagger:** DELETE /usuarios/{id}

56.Funcionalidad: Login Alternativo

- **descripción de la Funcionalidad:** Endpoint de login duplicado que usa UserService en lugar de AuthController. Valida credenciales y retorna token JWT.
- **Diagramas:**

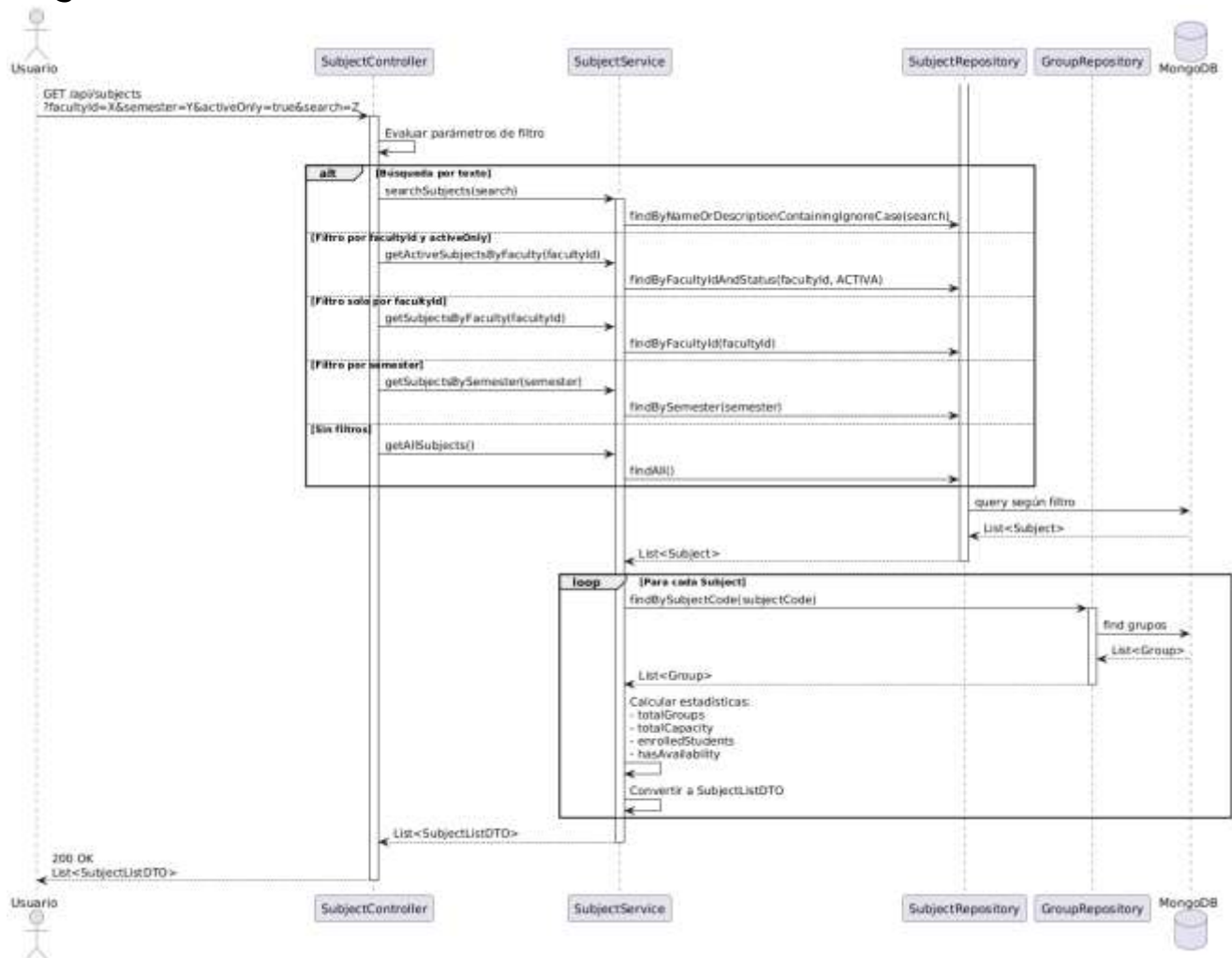


Duplicado de /auth/login, busca usuario por email en el repositorio, compara contraseña en texto plano, si coincide genera token JWT con el userId y lo retorna, si no retorna 401.

- **Manejo de Errores: 401 Unauthorized:** Si las credenciales son incorrectas
- **Link Swagger:** POST /usuarios/login

57.Funcionalidad: Listar Materias

- **descripción de la Funcionalidad:** Lista materias con múltiples opciones de filtrado: por facultad, semestre, solo activas, o búsqueda por texto. También calcula estadísticas de grupos (capacidad total, estudiantes inscritos, disponibilidad).
- **Diagramas:**

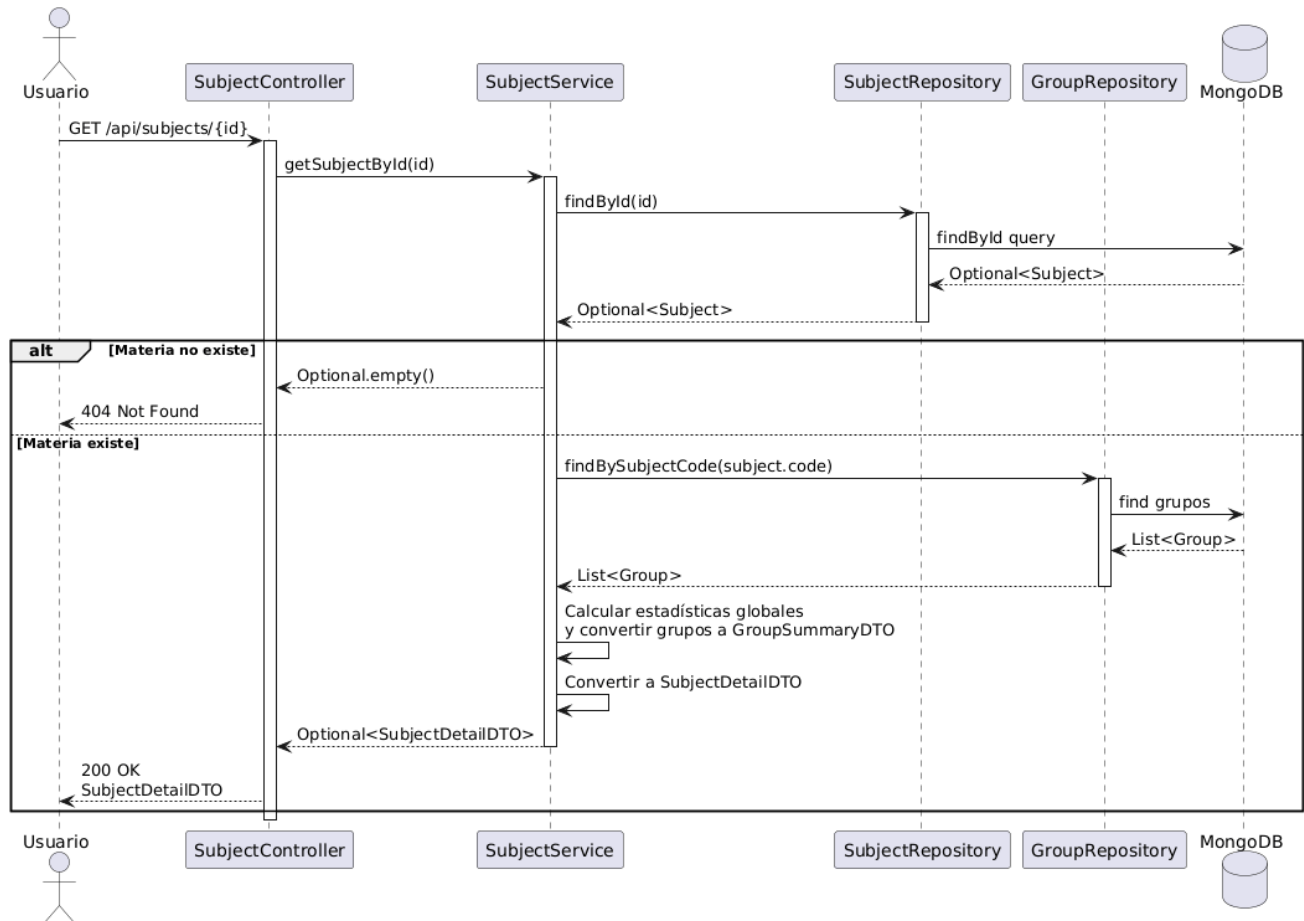


Según los parámetros recibidos (search, facultyId, semester, activeOnly) ejecuta diferentes consultas al repositorio de materias, para cada materia encontrada consulta sus grupos en otra query, calcula estadísticas agregadas (total grupos, capacidad, inscritos, disponibilidad) y retorna la lista de DTOs.

- **Manejo de Errores:** No maneja errores específicos
- **Link Swagger:** GET /api/subjects

58.Funcionalidad: Obtener Materia por ID

- **descripción de la Funcionalidad:** Obtiene los detalles completos de una materia específica, incluyendo información de todos sus grupos con estadísticas.
- **Diagramas:**

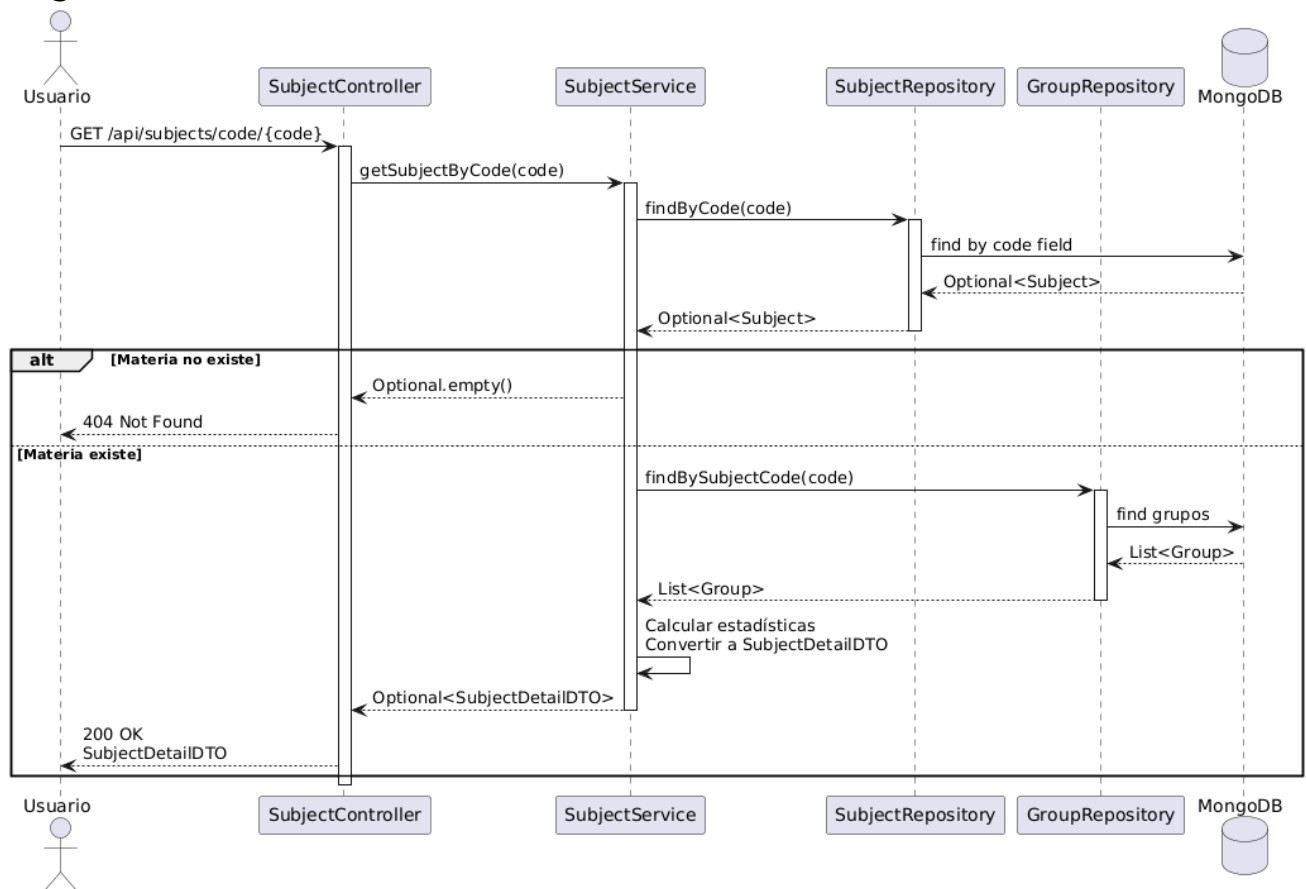


Busca una materia por su ID de MongoDB, luego consulta todos sus grupos, calcula estadísticas globales, convierte cada grupo a DTO resumido, construye un DTO detallado con toda la información y lista de grupos, y lo retorna.

- **Manejo de Errores: 404 Not Found:** Si la materia no existe
- **Link Swagger:** GET /api/subjects/{id}

59.Funcionalidad: Obtener Materia por Código

- **descripción de la Funcionalidad:** Obtiene una materia utilizando su código único (ej: "ISOFT-101") en lugar de su ID de MongoDB.
- **Diagramas:**

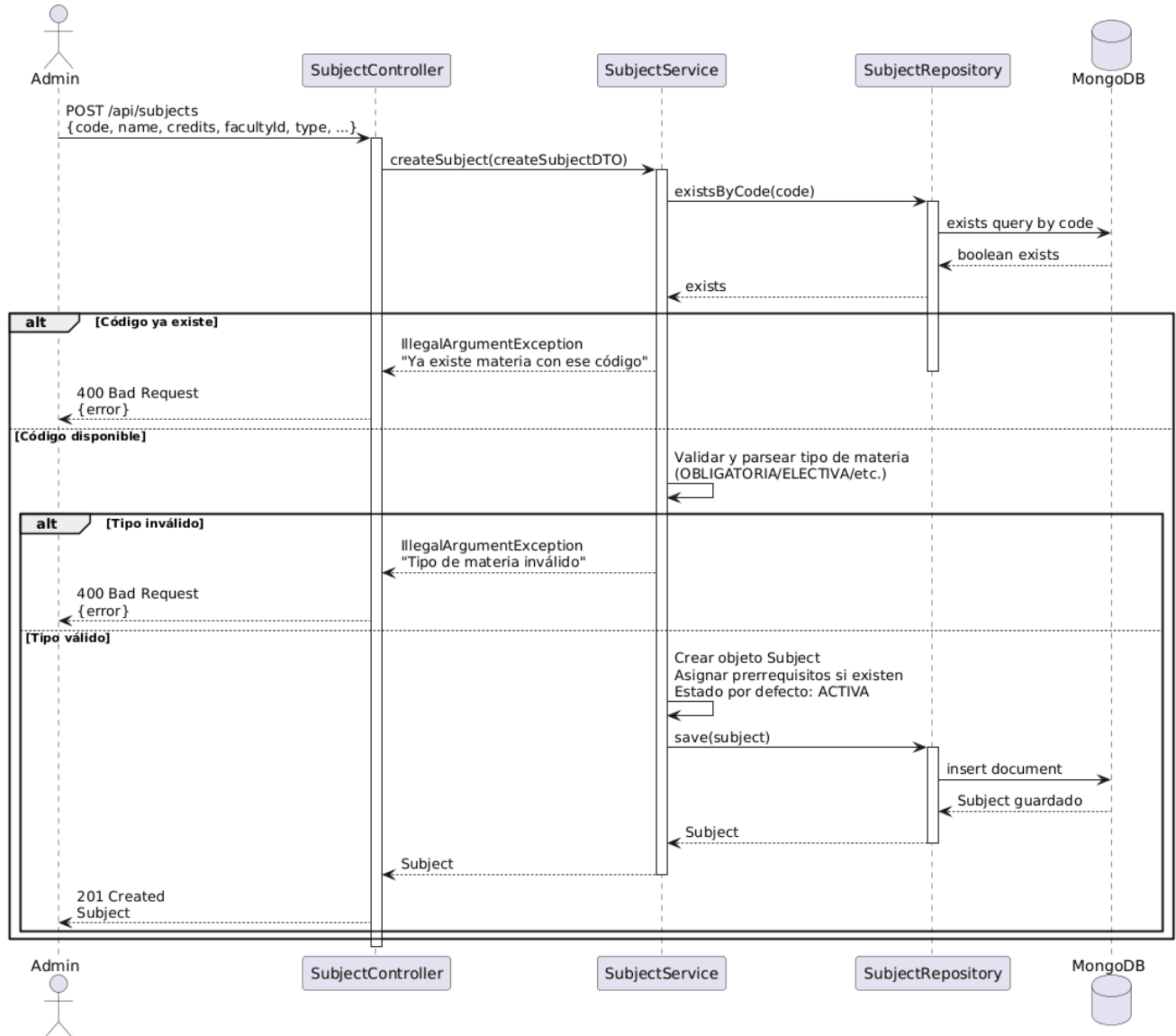


Similar al anterior pero busca la materia usando su código único (ej: "ISOFT-101") en lugar del ID, luego sigue el mismo proceso de obtener grupos, calcular estadísticas y construir el DTO detallado.

- **Manejo de Errores: 404 Not Found:** Si no existe materia con ese código
- **Link Swagger:** GET /api/subjects/code/{code}

60.Funcionalidad: Crear Materia

- **descripción de la Funcionalidad:** Crea una nueva materia en el sistema con toda su información académica (código, nombre, créditos, facultad, tipo, prerequisites).
- **Diagramas:**

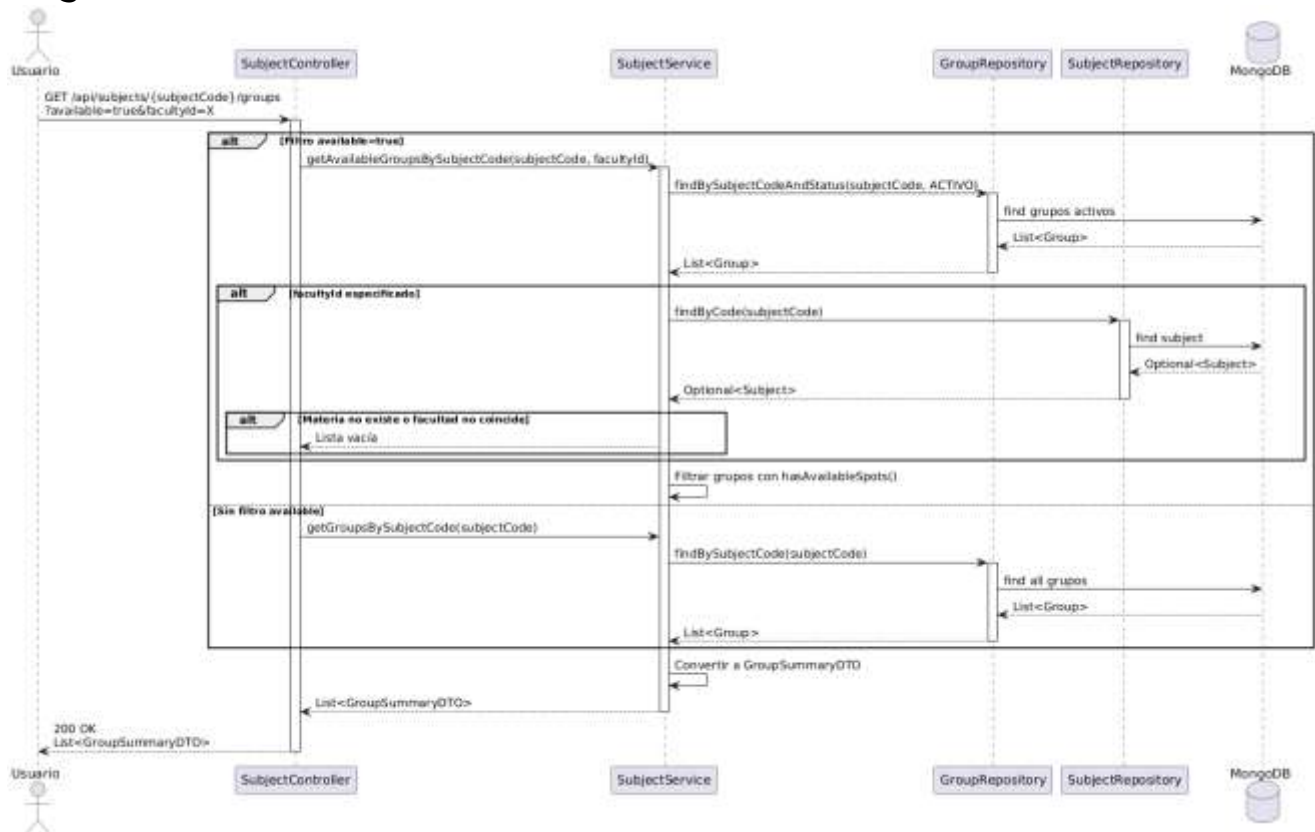


Recibe datos de una nueva materia, verifica que el código no exista ya en la base de datos, valida y parsea el tipo de materia (OBLIGATORIA/ELECTIVA), crea el objeto Subject con estado ACTIVA por defecto, asigna prerequisites si los hay, y guarda en MongoDB.

- **Manejo de Errores: 400 Bad Request:** Si el código ya existe o el tipo es inválido
- **Link Swagger:** POST /api/subjects

61. Funcionalidad: Listar Grupos de Materia

- **descripción de la Funcionalidad:** Lista todos los grupos de una materia específica, con opción de filtrar solo grupos disponibles (con cupos) para una facultad.
- **Diagramas:**

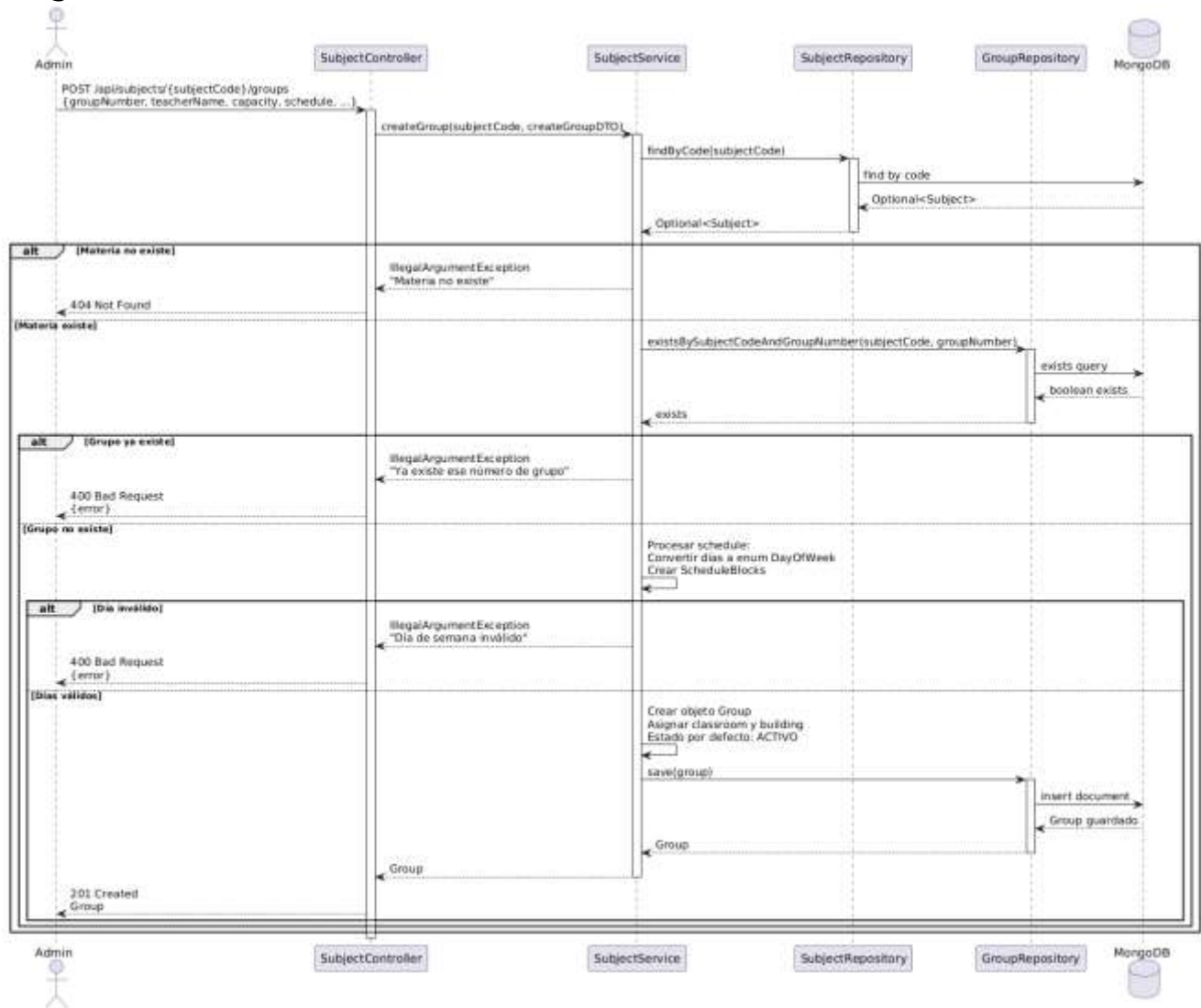


Si se solicitan grupos disponibles, busca grupos activos del código de materia, opcionalmente valida que la materia pertenezca a la facultad especificada, filtra solo grupos con cupos disponibles y retorna la lista; si no, simplemente retorna todos los grupos de esa materia.

- **Manejo de Errores:** Retorna lista vacía si no hay grupos (no es error)
- **Link Swagger:** GET /api/subjects/{subjectCode}/groups

62.Funcionalidad: Crear Grupo

- **descripción de la Funcionalidad:** Crea un nuevo grupo para una materia existente, incluyendo horarios, profesor, capacidad y ubicación.
- **Diagramas:**

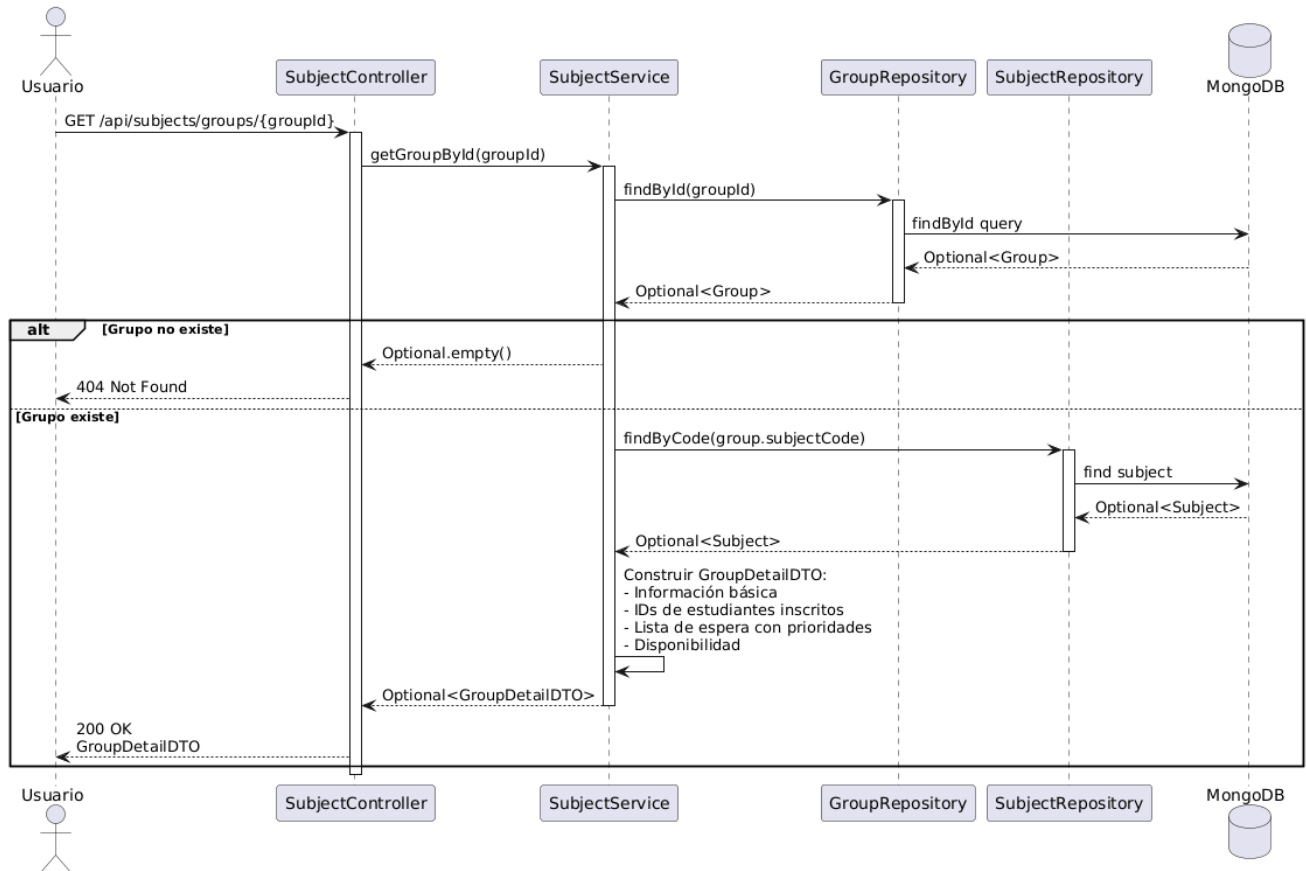


Verifica que la materia exista, valida que no exista ya ese número de grupo para esa materia, procesa el schedule convirtiendo los días a enum DayOfWeek y creando ScheduleBlocks, crea el objeto Group con estado ACTIVO, asigna classroom/building, y guarda en la base de datos.

- **Manejo de Errores: 404 Not Found:** Si la materia no existe, **400 Bad Request:** Si ya existe el número de grupo o los días son inválidos.
- **Link Swagger:** POST /api/subjects/{subjectCode}/groups

63.Funcionalidad: Obtener Detalle de Grupo

- **descripción de la Funcionalidad:** Obtiene información completa de un grupo específico, incluyendo lista de estudiantes inscritos, lista de espera con prioridades, y horarios.
- **Diagramas:**

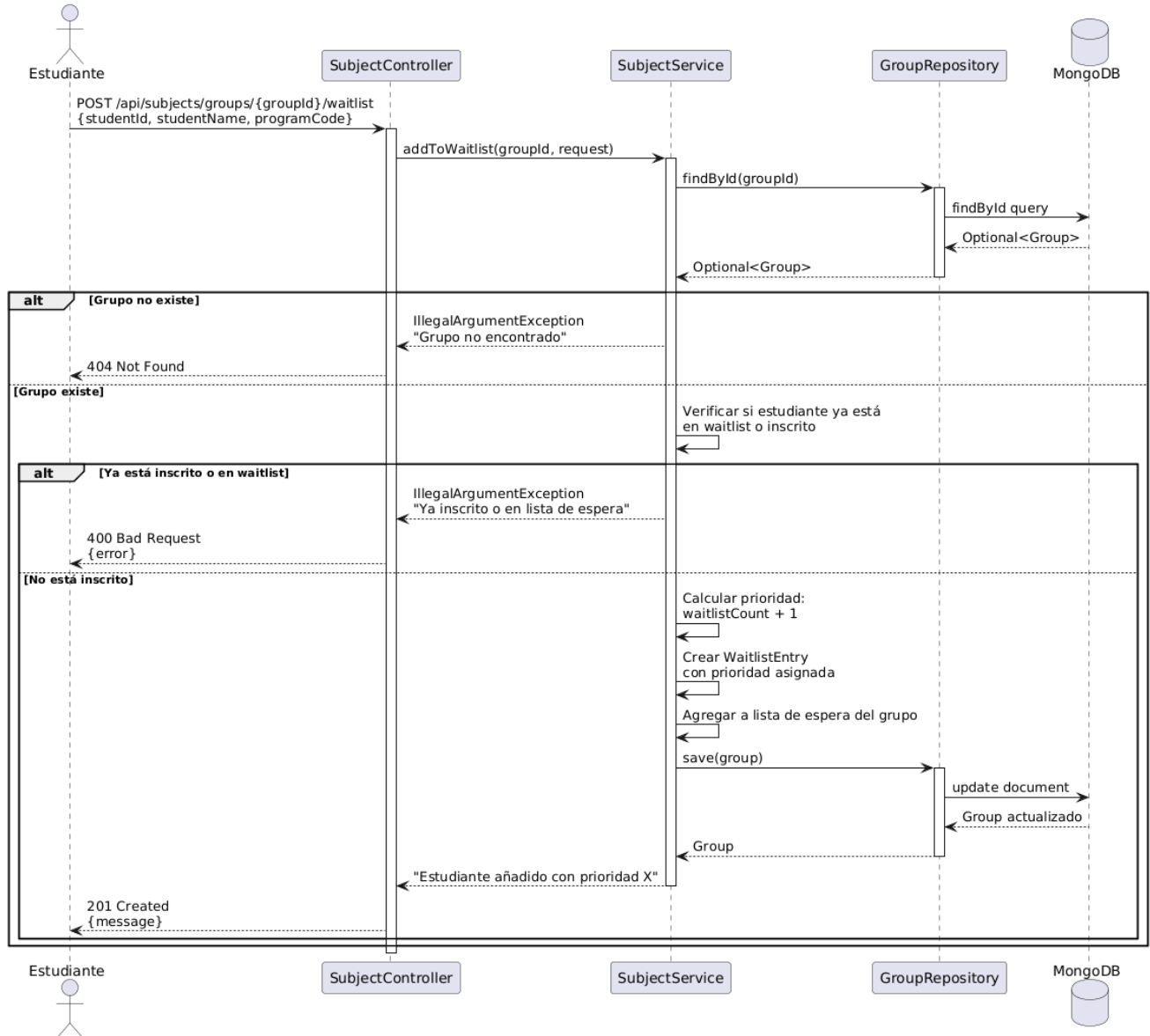


Busca un grupo por su ID, luego busca la materia asociada por su código para obtener el nombre, construye un DTO detallado con información del grupo, lista de IDs de estudiantes inscritos, lista de espera completa con prioridades, y toda la información de disponibilidad.

- **Manejo de Errores: 404 Not Found:** Si el grupo no existe
- **Link Swagger:** GET /api/subjects/groups/{groupId}

64.Funcionalidad: Agregar a Lista de Espera

- **descripción de la Funcionalidad:** Añade un estudiante a la lista de espera de un grupo que está lleno. Asigna prioridad automática basada en el orden de llegada.
- **Diagramas:**

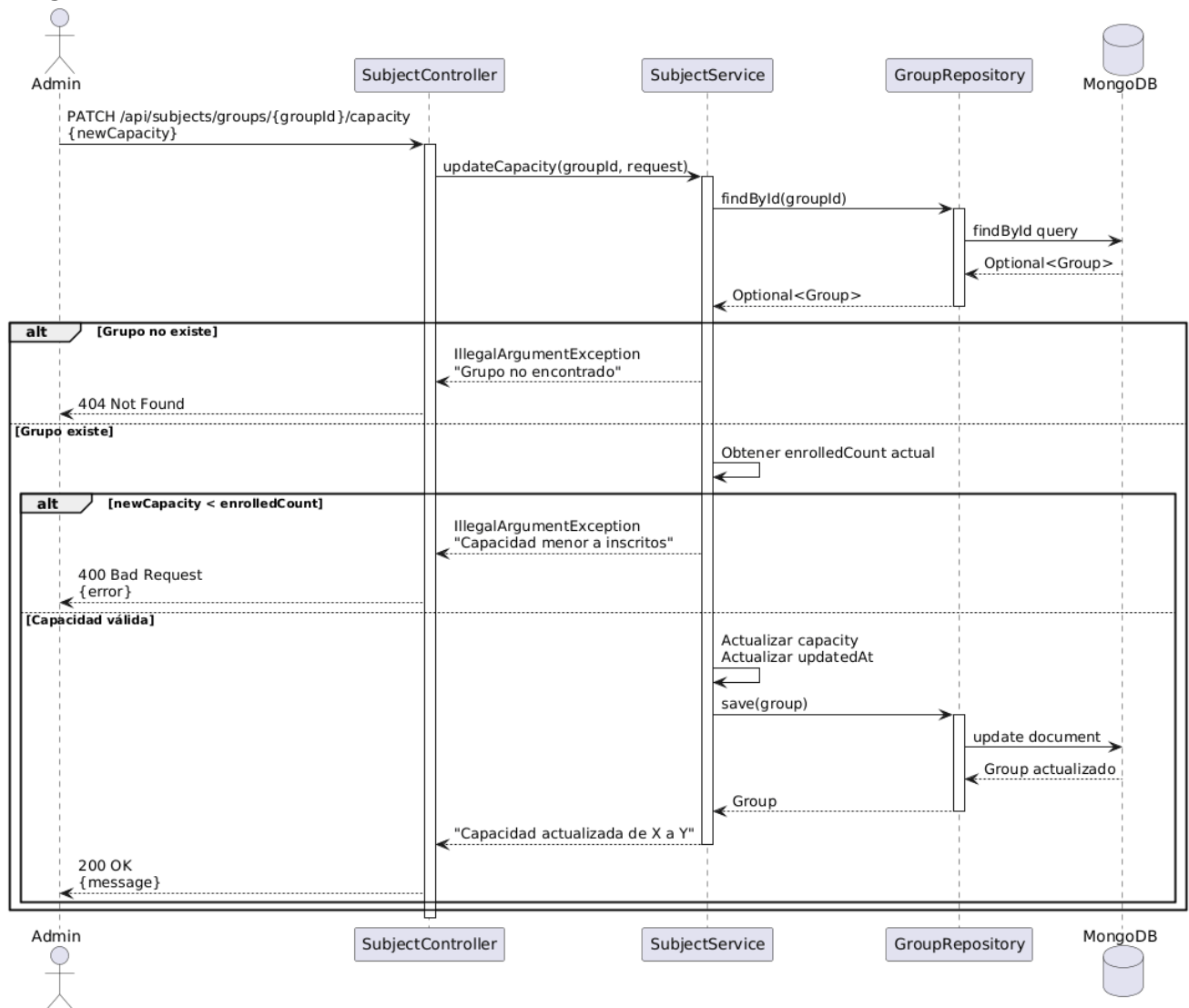


Busca el grupo por ID, verifica que el estudiante no esté ya inscrito ni en la lista de espera, calcula la prioridad automática (posición en la cola = waitlistCount + 1), crea una WaitlistEntry con los datos del estudiante y su prioridad, la añade a la lista del grupo y guarda.

- **Manejo de Errores: 404 Not Found:** Si el grupo no existe, **400 Bad Request:** Si el estudiante ya está inscrito o en la lista de espera.
- **Link Swagger:** POST /api/subjects/groups/{groupId}/waitlist

65.Funcionalidad: Actualizar Capacidad

- **descripción de la Funcionalidad:** Permite modificar la capacidad máxima de un grupo. Valida que la nueva capacidad no sea menor al número de estudiantes ya inscritos.
- **Diagramas:**



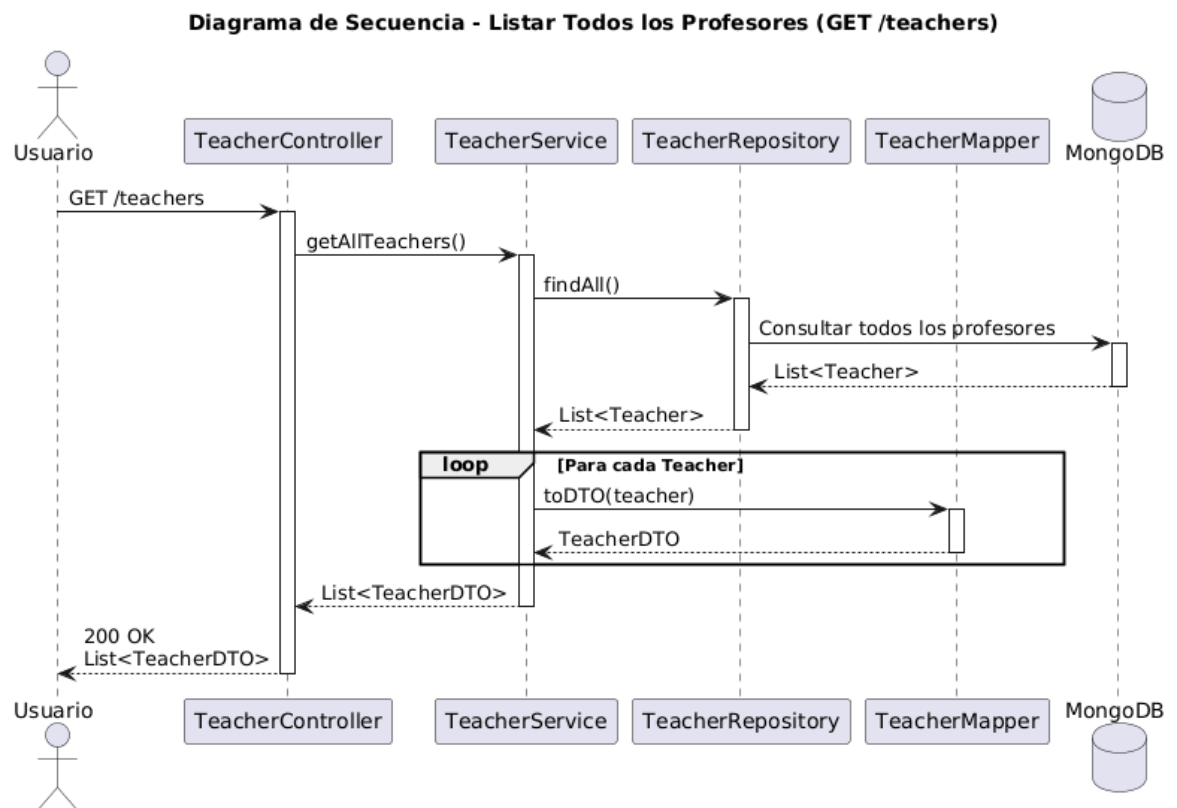
Busca el grupo por ID, obtiene el número actual de estudiantes inscritos, valida que la nueva capacidad sea mayor o igual a los inscritos (no se puede reducir por debajo), si es válida actualiza el campo

capacity y updatedAt del grupo, guarda en MongoDB y retorna mensaje de confirmación.

- **Manejo de Errores: 404 Not Found:** Si el grupo no existe, **400 Bad Request:** Si la nueva capacidad es menor al número de estudiantes inscritos.
- **Link Swagger:** PATCH /api/subjects/groups/{groupId}/capacity

66.Funcionalidad: Listar Todos los Profesores

- **descripción de la Funcionalidad:** Permite obtener una lista completa de todos los profesores registrados en el sistema. Este endpoint retorna la información básica de cada profesor incluyendo ID, nombre y correo electrónico.
- **Diagramas:**

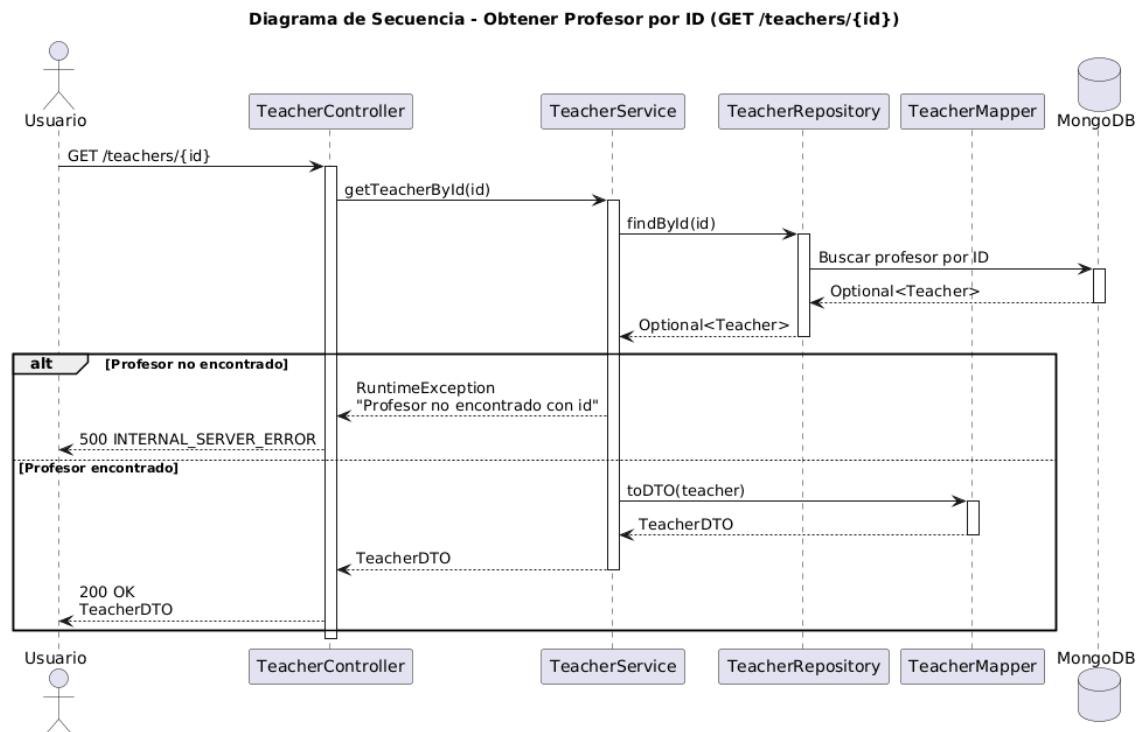


El cliente realiza una petición GET sin parámetros, el sistema consulta el repositorio de profesores, obtiene todos los registros, los mapea a DTOs y retorna la lista completa de profesores.

- **Manejo de Errores:** 500 Internal Server Error: Si ocurre un error inesperado en el servidor durante la consulta. Retorna una lista vacía [] si no hay profesores registrados (no es un error).
- **Link Swagger:** GET /teachers

67.Funcionalidad: Obtener Profesor por ID

- **descripción de la Funcionalidad:** Recupera la información detallada de un profesor específico mediante su identificador único numérico. Retorna los datos completos del profesor incluyendo ID, nombre y correo electrónico.
- **Diagramas:**

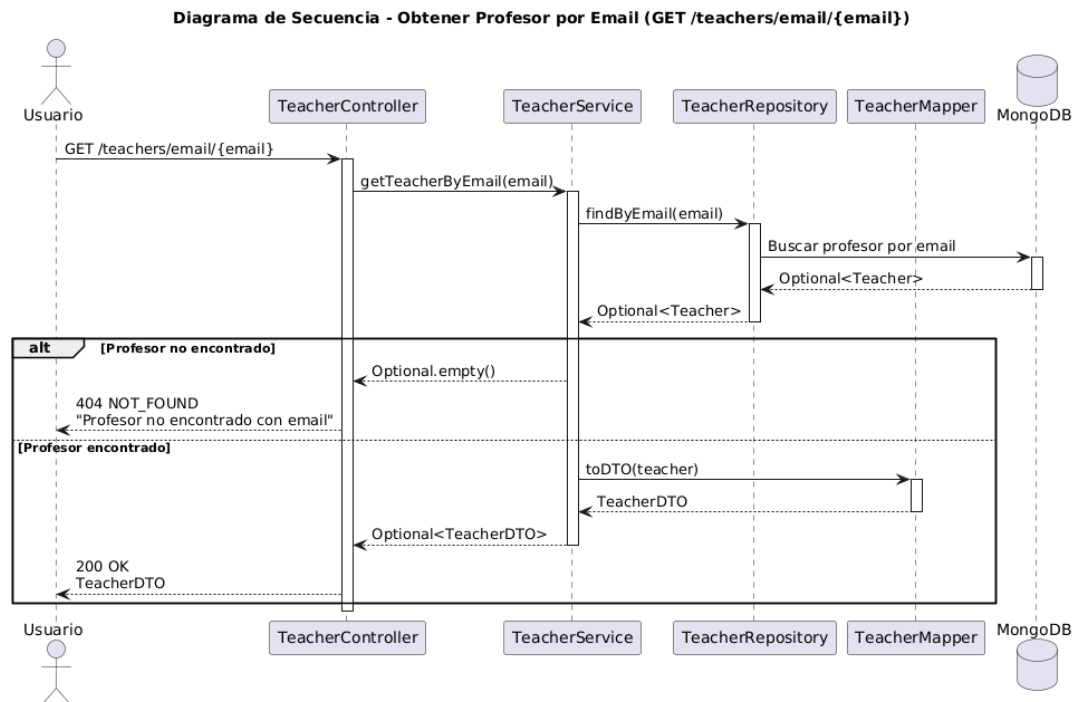


El cliente envía una petición GET con el ID del profesor como parámetro de ruta, el sistema invoca `findTeacherOrThrow(id)` que busca en el repositorio mediante `findById()`, valida la existencia del profesor, mapea la entidad a DTO y retorna la información.

- **Manejo de Errores: 404 Not Found:** 404 Not Found: Si el profesor con el ID especificado no existe. Mensaje: "Profesor no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la operación.
- **Link Swagger:** GET /teachers/{id}

68.Funcionalidad: Obtener Profesor por Email

- **descripción de la Funcionalidad:** Busca y retorna la información de un profesor utilizando su dirección de correo electrónico como criterio de búsqueda único. Es útil para validaciones de existencia, búsquedas alternativas al ID o integraciones con sistemas de autenticación.
- **Diagramas:**

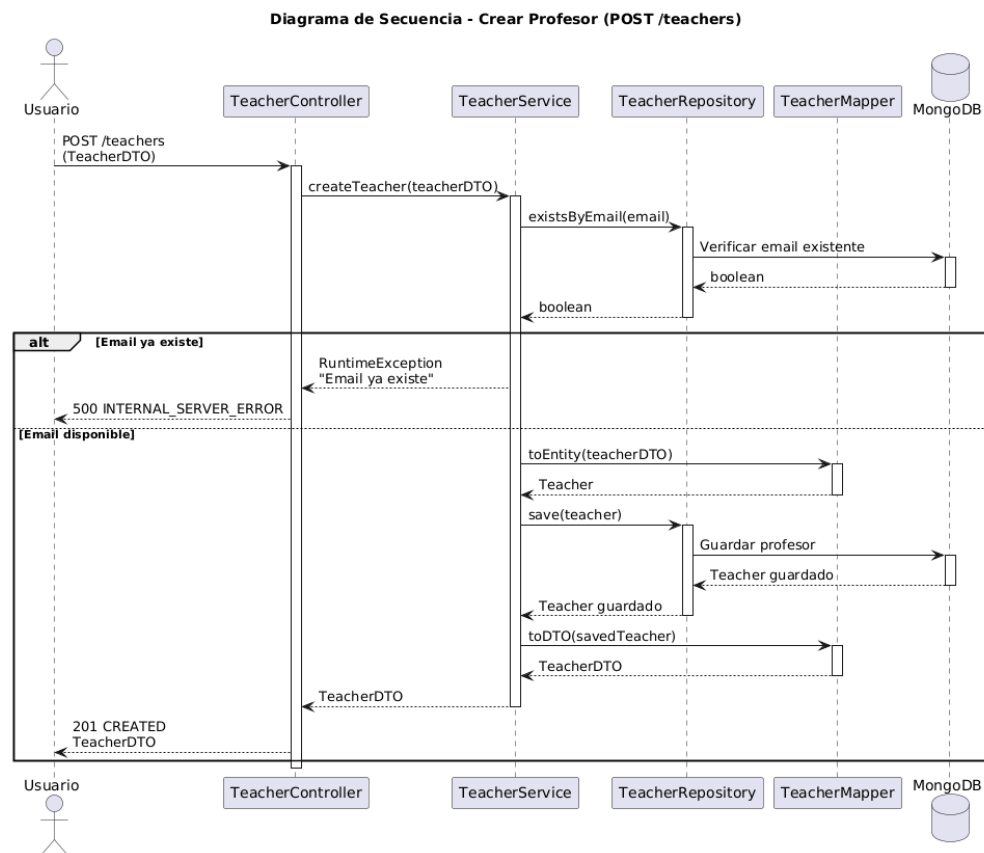


El cliente envía una petición GET con el email del profesor como parámetro de ruta, el sistema busca en el repositorio mediante `findByEmail(email)` que retorna un `Optional`, valida la presencia del resultado y lanza excepción si no existe, o mapea y retorna el `TeacherDTO`.

- **Manejo de Errores:** 404 Not Found: Si no existe ningún profesor registrado con el email especificado. Mensaje: "Profesor no encontrado con email: {email}". 500 Internal Server Error: Si ocurre un error inesperado durante la búsqueda.
- **Link Swagger:** GET /teachers/email/{email}

69.Funcionalidad: Crear Nuevo Profesor

- **descripción de la Funcionalidad:** Permite registrar un nuevo profesor en el sistema con los datos básicos (nombre y email). Antes de crear, valida que el correo electrónico no esté previamente registrado mediante ensureTeacherEmailNotRegisteredForCreate(). Retorna el profesor creado con su ID autogenerated y código de estado 201 Created.
- **Diagramas:**



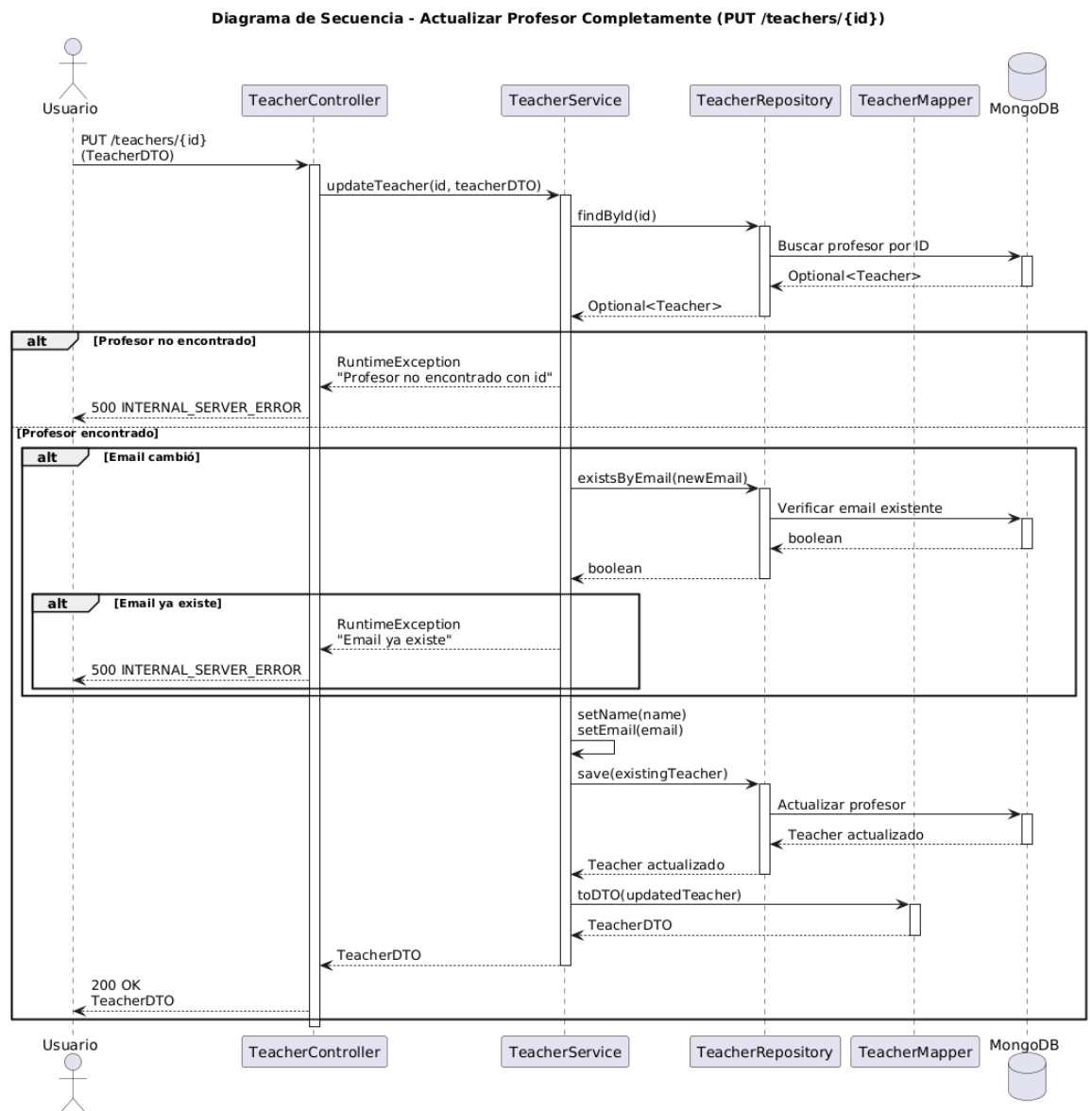
El cliente envía una petición POST con el cuerpo JSON conteniendo los datos del profesor, el sistema valida que el email no esté registrado mediante existsByEmail(), mapea el DTO a entidad Teacher, persiste en

la base de datos con save() generando el ID automáticamente, mapea la entidad guardada a DTO y retorna con estado HTTP 201.

- **Manejo de Errores:** 400 Bad Request: Si el email ya está registrado en el sistema. Mensaje: "Email ya existe: {email}". 400 Bad Request: Si faltan campos obligatorios (nombre o email) o están vacíos. Mensaje: "Validation failed" con detalles de los campos. 400 Bad Request: Si el formato del email es inválido según las validaciones de Jakarta/Bean Validation. 500 Internal Server Error: Si ocurre un error inesperado durante la persistencia.
- **Link Swagger:** POST /teachers

70.Funcionalidad: Actualizar Profesor Completamente

- **descripción de la Funcionalidad:** Realiza una actualización completa (PUT) de todos los campos de un profesor existente. Requiere enviar la información completa del profesor (nombre y email obligatorios). Si se cambia el email, valida mediante ensureTeacherEmailNotRegisteredForUpdate() que el nuevo correo no esté registrado por otro profesor diferente.
- **Diagramas:**



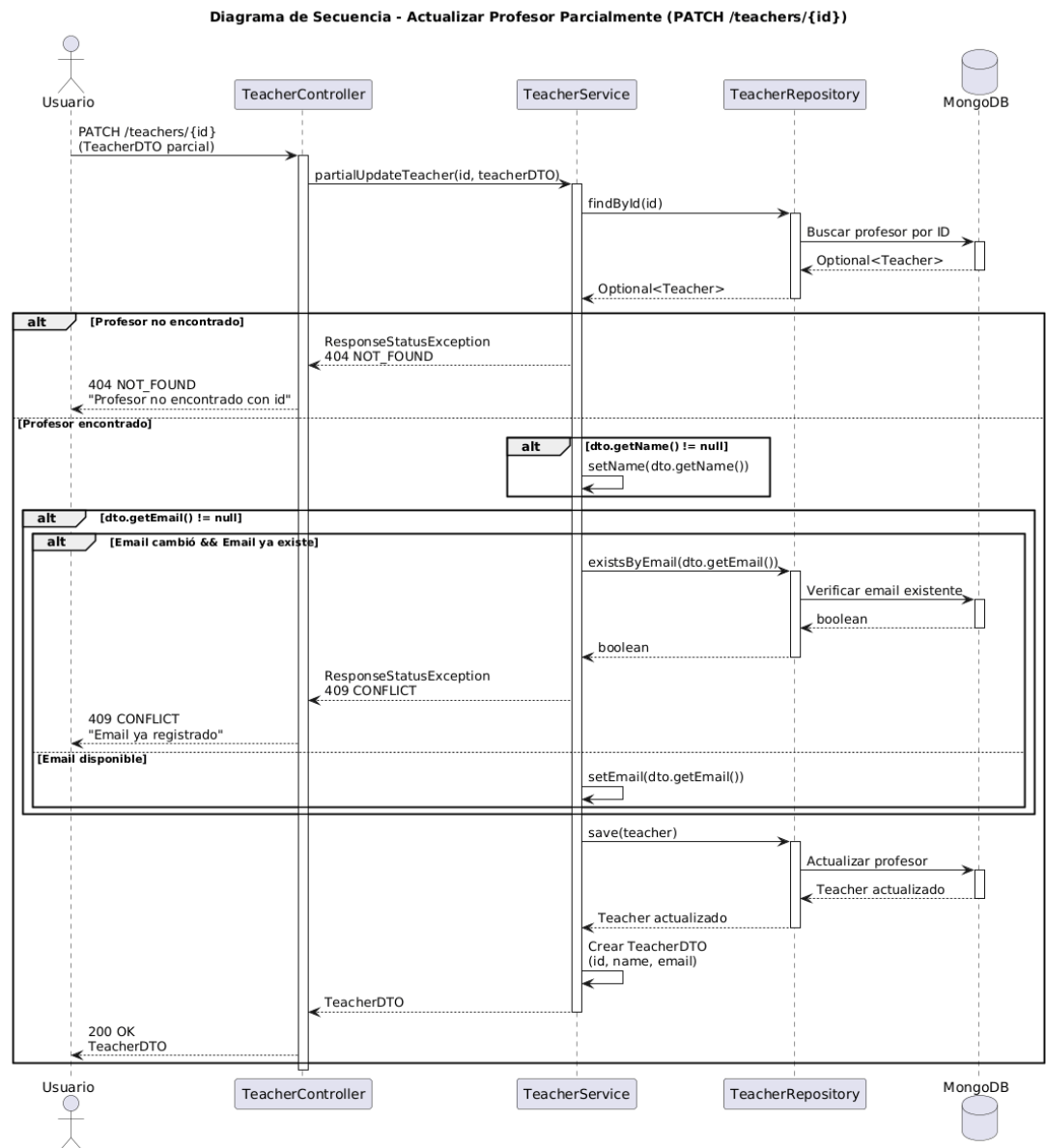
El cliente envía una petición PUT con el ID en la ruta y el cuerpo con todos los datos, el sistema busca el profesor existente con findTeacherOrThrow(id), compara el email actual con el nuevo, si son diferentes valida que el nuevo email no exista o pertenezca al mismo

profesor, actualiza los campos nombre y email, persiste con save() y retorna el profesor actualizado.

- **Manejo de Errores:** 404 Not Found: Si el profesor con el ID especificado no existe. Mensaje: "Profesor no encontrado con id: {id}". 400 Bad Request: Si el nuevo email ya está registrado por otro profesor. Mensaje: "Email ya registrado: {email}". 400 Bad Request: Si faltan campos obligatorios (nombre o email). 400 Bad Request: Si el formato del email es inválido. 500 Internal Server Error: Si ocurre un error durante la actualización.
- **Link Swagger:** PUT /teachers/{id}

71.Funcionalidad: Actualizar Profesor Parcialmente

- **descripción de la Funcionalidad:** Permite actualizar uno o varios campos específicos de un profesor (PATCH) sin necesidad de enviar toda la información. Solo los campos presentes en el request body serán actualizados, los campos null o ausentes se ignoran. Si se incluye el email, valida mediante `ensureTeacherEmailNotRegisteredForUpdate()` que no esté registrado por otro profesor.
- **Diagramas:**



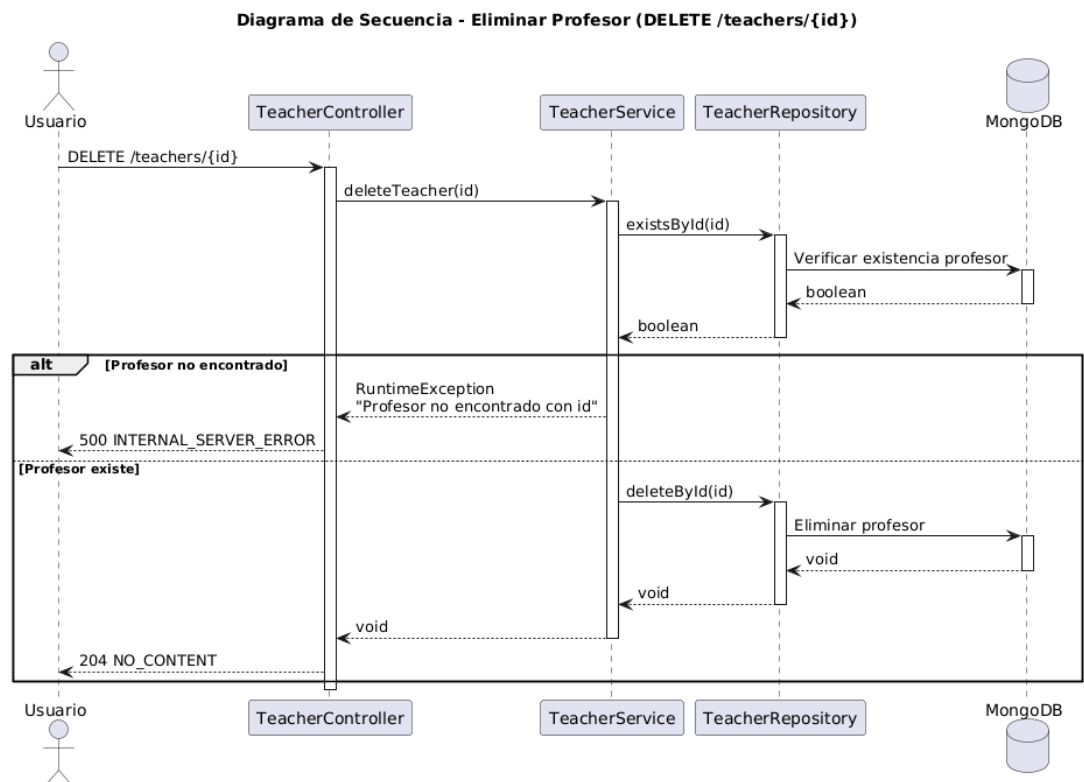
El cliente envía una petición PATCH con el ID en la ruta y el cuerpo con solo los campos a actualizar, el sistema busca el profesor con `findTeacherOrThrow(id)`, verifica campo por campo si el DTO contiene

valores no nulos (nombre y/o email), para el email valida si cambi6 y si el nuevo no est1 registrado por otro profesor, actualiza solo los campos presentes, persiste con save() y retorna el profesor completo actualizado.

- **Manejo de Errores:** 404 Not Found: Si el profesor con el ID especificado no existe. Mensaje: "Profesor no encontrado con id: {id}". 400 Bad Request: Si el email enviado ya est1 registrado por otro profesor. Mensaje: "Email ya registrado: {email}". 400 Bad Request: Si el formato del email proporcionado es inv1lido. 500 Internal Server Error: Si ocurre un error durante la actualizaci6n parcial.
- **Link Swagger:** PATCH /teachers/{id}

72.Funcionalidad: Eliminar Profesor

- **descripci6n de la Funcionalidad:** Elimina permanentemente un profesor del sistema mediante su ID. Antes de eliminar, valida la existencia del profesor con findTeacherOrThrow(). Retorna c6digo de estado 204 No Content al completarse exitosamente la eliminaci6n sin cuerpo en la respuesta.
- **Diagramas:**



El cliente envía una petición DELETE con el ID del profesor en la ruta, el sistema busca y valida la existencia del profesor mediante `findTeacherOrThrow(id)`, si existe procede a eliminarlo del repositorio con `deleteById(id)`, confirma la eliminación y retorna estado HTTP 204 sin contenido en el body.

- **Manejo de Errores:** 404 Not Found: Si el profesor con el ID especificado no existe en el sistema. Mensaje: "Profesor no encontrado con id: {id}". 400 Bad Request: Si el ID proporcionado no es un número válido. 500 Internal Server Error: Si ocurre un error inesperado durante la eliminación (por ejemplo, restricciones de integridad referencial si el profesor está asignado a grupos).
- **Link Swagger:** DELETE /teachers/{id}

Referencias bibliográficas

Frameworks y Tecnologías Principales

Spring Framework

- Spring Boot Documentation. (2024). *Spring Boot Reference Documentation Version 3.5.3*. Spring.io. Recuperado de <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- Spring Data MongoDB Documentation. (2024). *Spring Data MongoDB - Reference Documentation*. Spring.io. Recuperado de <https://docs.spring.io/spring-data/mongodb/docs/current/reference/html/>
- Spring Web MVC Documentation. (2024). *Web on Servlet Stack*. Spring.io. Recuperado de <https://docs.spring.io/spring-framework/docs/current/reference/html/web.html>

Lenguaje de Programación

- Oracle Corporation. (2024). *Java Platform, Standard Edition 21 API Specification*. Oracle.com. Recuperado de <https://docs.oracle.com/en/java/javase/21/docs/api/>

Base de Datos

- MongoDB, Inc. (2024). *MongoDB Manual*. MongoDB.com. Recuperado de <https://www.mongodb.com/docs/manual/>
- MongoDB, Inc. (2024). *MongoDB Java Driver Documentation*. MongoDB.com. Recuperado de <https://www.mongodb.com/docs/drivers/java/sync/current/>

Gestión de Dependencias

- Apache Software Foundation. (2024). *Apache Maven Project Documentation*. Maven.apache.org. Recuperado de <https://maven.apache.org/guides/>

Seguridad y Autenticación

- Auth0. (2024). *JSON Web Tokens - jwt.io*. JWT.io. Recuperado de <https://jwt.io/introduction>
- jwtk/jjwt. (2024). *Java JWT: JSON Web Token for Java and Android*. GitHub. Recuperado de <https://github.com/jwtk/jjwt>

Documentación de API

- SpringDoc. (2024). *SpringDoc OpenAPI Documentation Version 2.6.0*. Springdoc.org. Recuperado de <https://springdoc.org/>
- OpenAPI Initiative. (2024). *OpenAPI Specification v3.1.0*. OpenAPI.org. Recuperado de <https://spec.openapis.org/oas/latest.html>
- SmartBear Software. (2024). *Swagger UI Documentation*. Swagger.io. Recuperado de <https://swagger.io/tools/swagger-ui/>

Validación y Calidad de Código

- Hibernate Validator Documentation. (2024). *Hibernate Validator Reference Guide*. Hibernate.org. Recuperado de https://docs.jboss.org/hibernate/stable/validator/reference/en-US/html_single/
- Project Lombok. (2024). *Lombok Features Documentation Version 1.18.32*. Projectlombok.org. Recuperado de <https://projectlombok.org/features/>

Testing y Análisis de Código

- JUnit Team. (2024). *JUnit 5 User Guide*. JUnit.org. Recuperado de <https://junit.org/junit5/docs/current/user-guide/>
- Mockito. (2024). *Mockito Framework Documentation*. Mockito.org. Recuperado de <https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html>

- JaCoCo. (2024). *JaCoCo Java Code Coverage Library Version 0.8.13*. EcEmma.org. Recuperado de <https://www.jacoco.org/jacoco/trunk/doc/>
- SonarSource. (2024). *SonarQube Documentation*. SonarQube.org. Recuperado de <https://docs.sonarsource.com/sonarqube/latest/>

Modelado y Arquitectura de Software

- Object Management Group (OMG). (2017). *OMG Unified Modeling Language (UML) Specification Version 2.5.1*. OMG.org. Recuperado de <https://www.omg.org/spec/UML/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional.

Ingeniería de Software

- Pressman, R. S., & Maxim, B. R. (2014). *Ingeniería de Software: Un Enfoque Práctico* (8.ª ed.). McGraw-Hill Education.
- Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson Education.
- Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.

Arquitectura REST y APIs

- Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures* [Tesis doctoral]. University of California, Irvine. Recuperado de <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Richardson, L., Amundsen, M., & Ruby, S. (2013). *RESTful Web APIs*. O'Reilly Media.

Base de Datos NoSQL

- Banker, K., Garrett, D., Bakkum, P., & Verch, S. (2016). *MongoDB in Action* (2nd ed.). Manning Publications.